

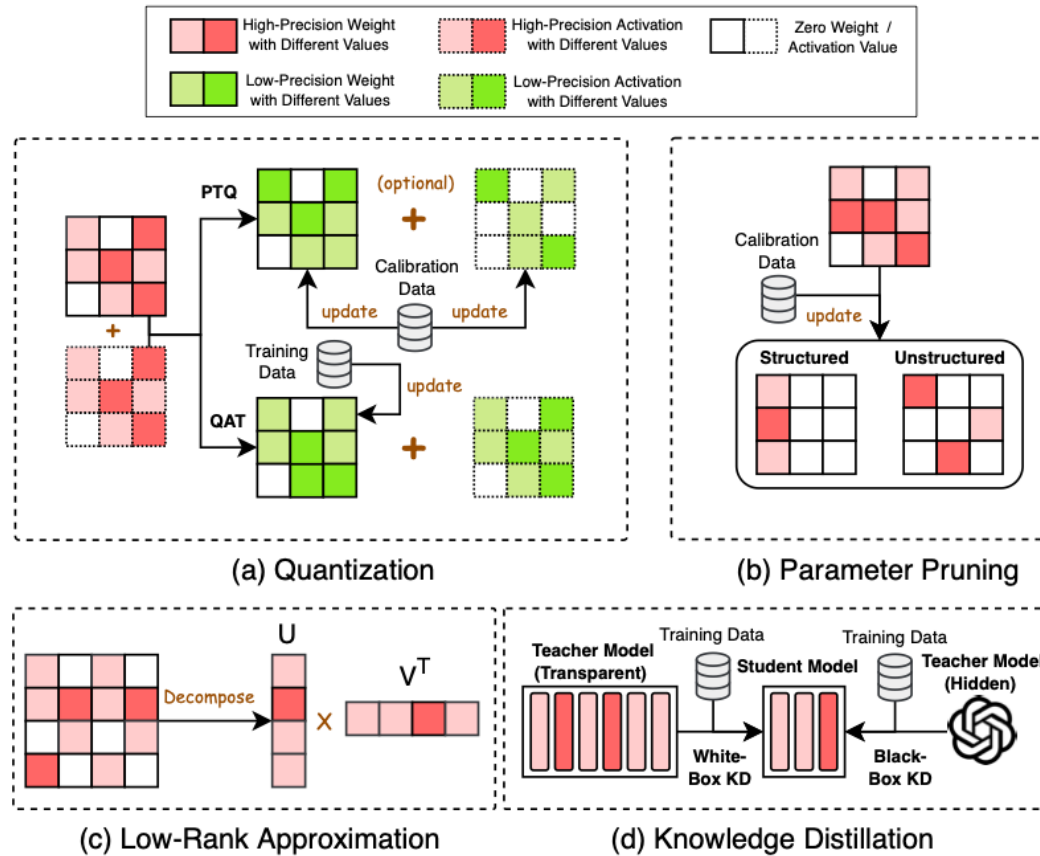
Introduction to High- Performance Machine Learning

Lecture 9

Dr. Kaoutar El Maghraoui

Efficient Transformers/LLMs: Flash Attention

Illustration of Model Compression Techniques for LLMs



Addressing Challenges of RNNs and LSTMs with Transformers

1. Long-Range Dependencies

- Efficiently learns dependencies regardless of sequence length.
- Solves the vanishing gradient problem common in RNNs and LSTMs.

• Enhanced Parallelization

- Processes entire sequences in parallel, not sequentially.
- Leads to faster training and inference.

1. Scalability and Efficiency

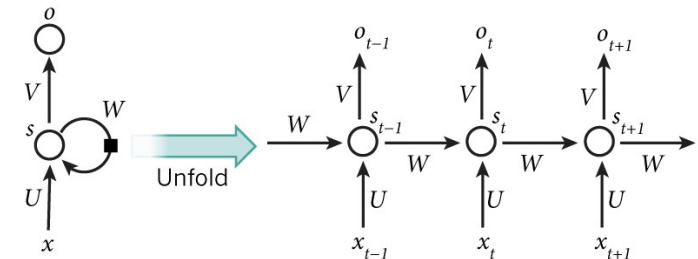
- Ideal for large datasets and model training.
- Powers large-scale models like GPT and BERT.

2. Context Modeling

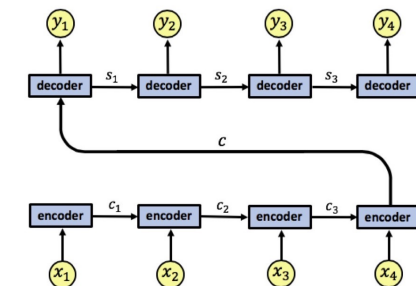
- Captures bidirectional context effectively.
- Improves performance in classification, and sentiment analysis.

3. Simplified yet Generalized

- Less complex than LSTMs, but highly effective.
- Adaptable to a range of tasks beyond translation.



Parameters to be learned:
U, V, W



Simple representation of traditional Encoder-Decoder architecture using RNN/LSTM

The paper that started it all!

- The "Attention is All You Need" paper, which introduced the Transformer architecture, had a profound impact on the field of natural language processing (NLP) and deep learning in general:
- Efficiency in Training
- Parallelization
- Scalability
- State-of-the-Art Performance
- Transferability
- Pre-training and Transfer Learning
- Wider Adoption Beyond NLP
- Open-Source Implementations

HPML

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

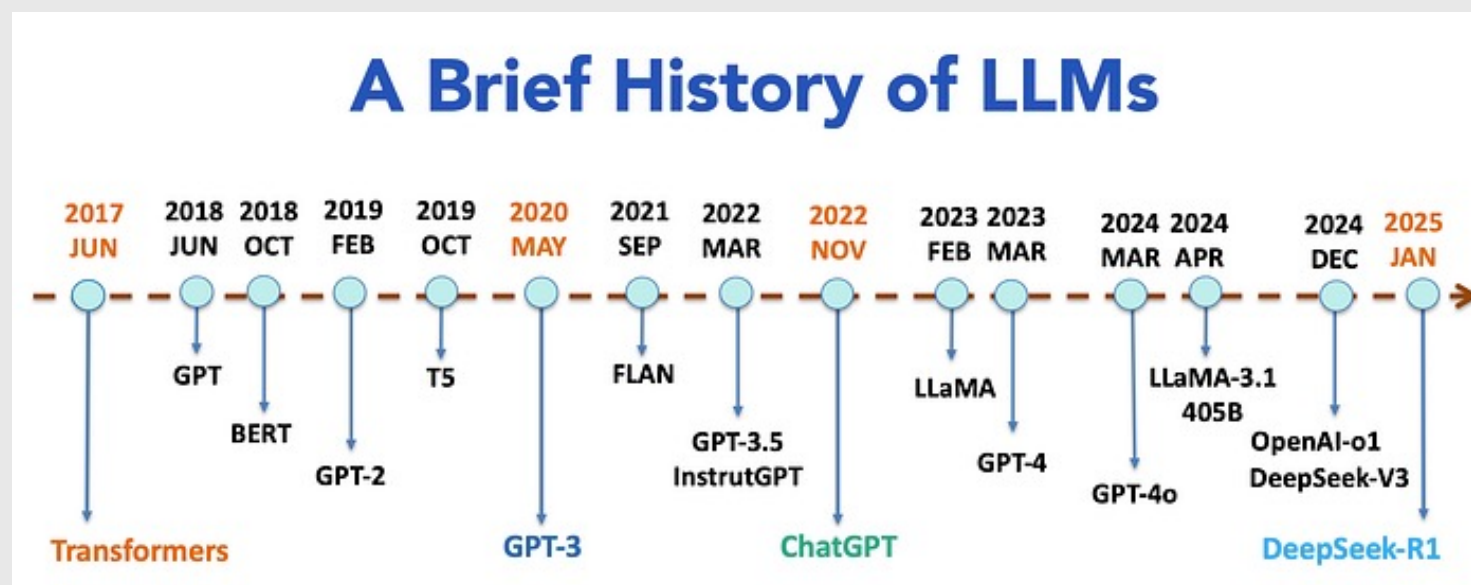
Łukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

Evolution from Transformer Architecture to Modern LLMs



Architecture → Scaling → Alignment/Reasoning → democratization/efficiency

Source: <https://medium.com/@lmpo/a-brief-history-of-lmms-from-transformers-2017-to-deepseek-r1-2025-dae75dd3f59a>

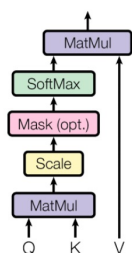
HPML

What is Self-Attention

- "The **animal** didn't cross the street because it was too **tired**"
- "The animal didn't cross the **street** because it was too **wide**"

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

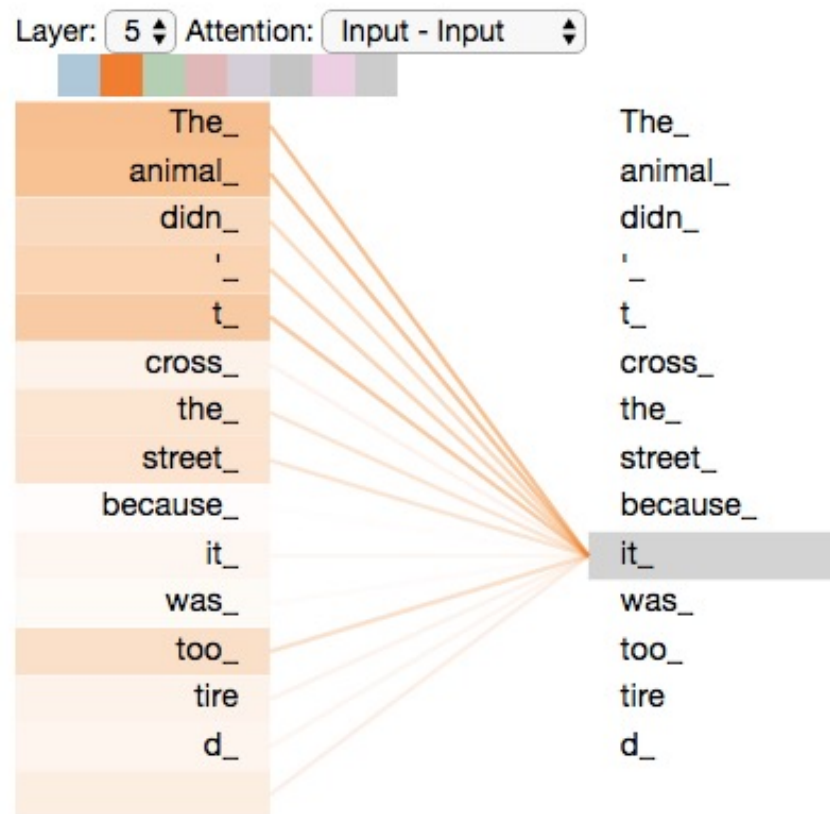
Scaled Dot-Product Attention



As we encode the word "it" in encoder #5 (the top encoder in the stack), part of the attention mechanism focused on "The Animal" and baked a part of its representation into the encoding of "it."

HPML

When the model is processing the word "it", self-attention allows it to associate "it" with "animal".



You start with a Query (Q), and you check against a (Key, Value) pair

Overall Process in Attention

1. Input Transformation:

- Each token in the sequence is transformed into three vectors: Query (Q), Key (K), and Value (V).
- These are computed as:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

Here:

- X : The input sequence (e.g., word embeddings).
- W_Q, W_K, W_V : Learnable weight matrices for transforming inputs into Queries, Keys, and Values.

2. Attention Scores:

- For each Query, compute the similarity scores with all Keys:

$$\text{Score}_{ij} = \frac{Q_i \cdot K_j^T}{\sqrt{d_k}}$$

These scores represent the relevance of Key j to Query i .

3. Softmax Normalization:

- Convert the scores into probabilities using a softmax function:

$$\alpha_{ij} = \text{softmax}(\text{Score}_{ij})$$

4. Weighted Sum:

- Use the normalized attention weights (α_{ij}) to compute a weighted sum of Values (V):

$$\text{Output}_i = \sum_j \alpha_{ij} V_j$$

Intuition:

- **Q (Query)**: "What am I looking for?"
- **K (Key)**: "What information do I have?"
- **V (Value)**: "What content should I extract if it matches?"

This mechanism allows the model to focus dynamically on the most relevant parts of the input sequence, which is especially useful for handling long-range dependencies and context.

Analogy with a Database: Query, Key & Value

Database

{Key, Value store}

```
{"order_100": {"items": "a1", "delivery_date": "a2", ...}},  
{"order_101": {"items": "b1", "delivery_date": "b2", ...}},  
{"order_102": {"items": "c1", "delivery_date": "c2", ...}},  
{"order_103": {"items": "d1", "delivery_date": "d2", ...}},  
{"order_104": {"items": "e1", "delivery_date": "e2", ...}},  
{"order_105": {"items": "f1", "delivery_date": "f2", ...}},  
{"order_106": {"items": "g1", "delivery_date": "g2", ...}},  
{"order_107": {"items": "h1", "delivery_date": "h2", ...}},  
{"order_108": {"items": "i1", "delivery_date": "i2", ...}},  
{"order_109": {"items": "j1", "delivery_date": "j2", ...}},  
{"order_110": {"items": "k1", "delivery_date": "k2", ...}}
```

Self Attention

– Step 1

Item	Shape
Input embedding x_i	1×512
Query projection matrix W^Q	512×64
Key projection matrix W^K	512×64
Value projection matrix W^V	512×64
Query vector q_i	1×64
Key vector k_i	1×64
Value vector v_i	1×64

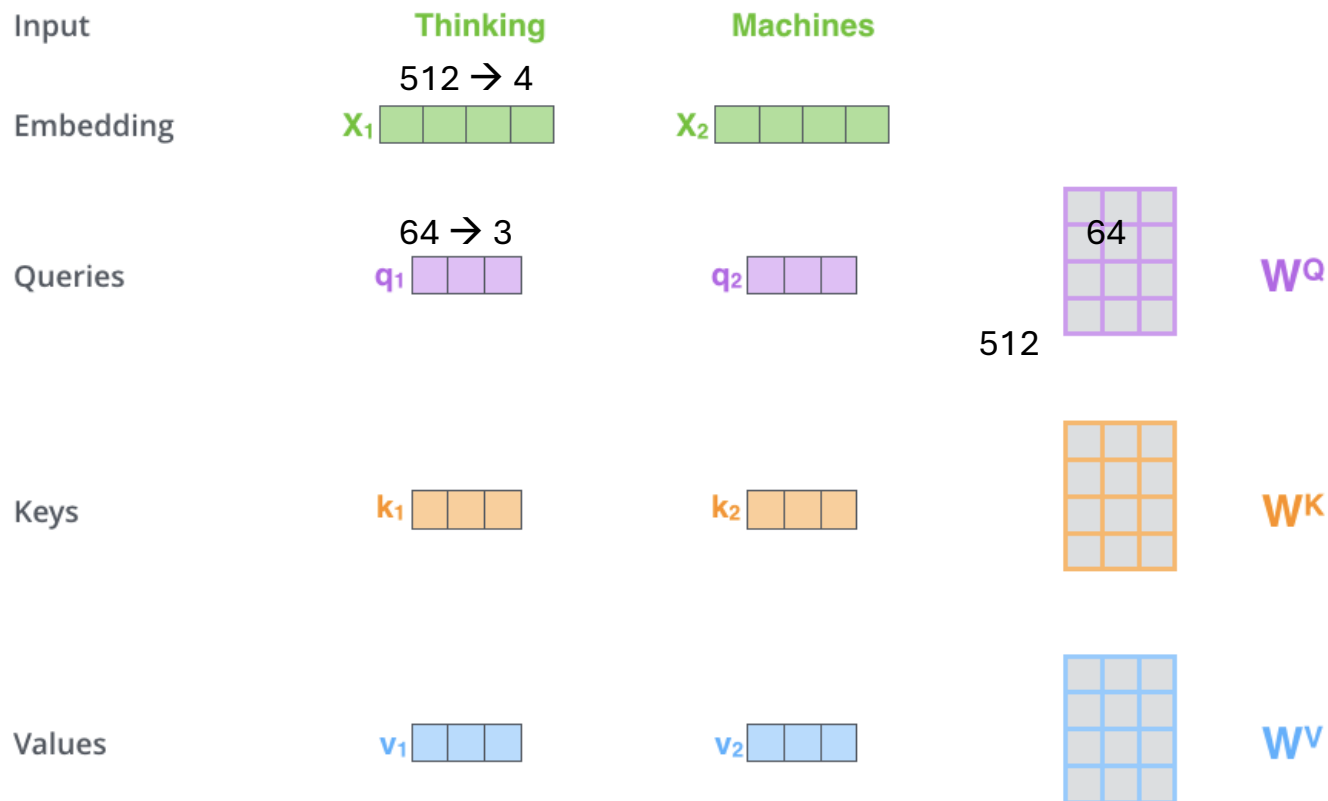
First, we create three vectors by multiplying input embedding (1×512)

x_i with three matrices (64×512):

$$q_i = x_i W^Q$$

$$k_i = x_i W^K$$

$$v_i = x_i W^V$$



Multiplying x_1 by the W^Q weight matrix produces q_1 , the "query" vector associated with that word. We end up creating a "query", a "key", and a "value" projection of each word in the input sentence.

Self Attention

– Step 3 and 4

Formula

$$\text{softmax} \left(\frac{\begin{matrix} 64 \times 64 \\ Q \end{matrix} \times \begin{matrix} 64 \times 512 \\ K^T \end{matrix}}{\sqrt{d_k}} \right) \begin{matrix} 64 \times 512 \\ V \end{matrix}$$

$$= \begin{matrix} Z \end{matrix}$$

$d_k=64$ is dimension of key vector

Divide the scores by 8 (the square root of the dimension of the key vectors). Then pass the results through a softmax.

Input

Embedding

Queries

Keys

Values

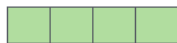
Score

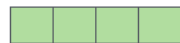
Divide by 8 ($\sqrt{d_k}$)

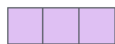
Softmax

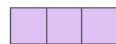
Thinking

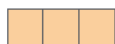
Machines

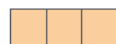
x_1 

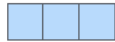
x_2 

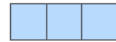
q_1 

q_2 

k_1 

k_2 

v_1 

v_2 

$q_1 \cdot k_1 = 112$

$q_1 \cdot k_2 = 96$

14

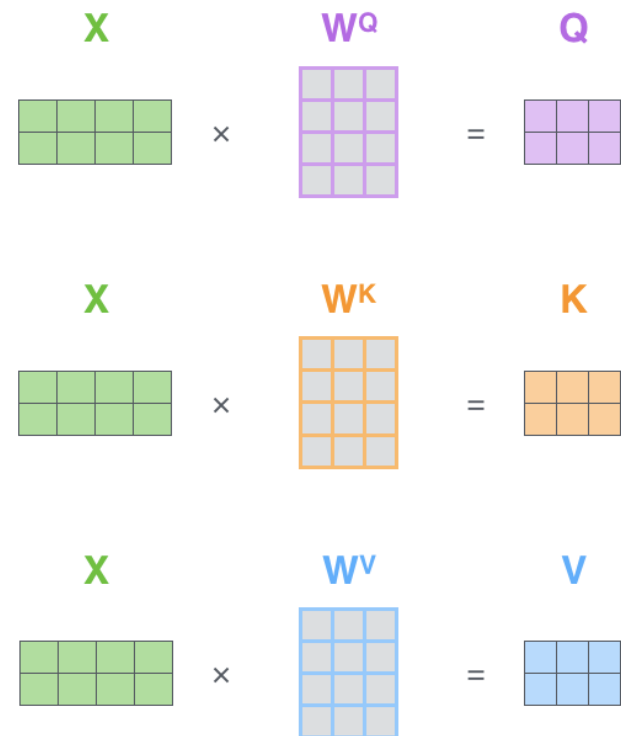
12

0.88

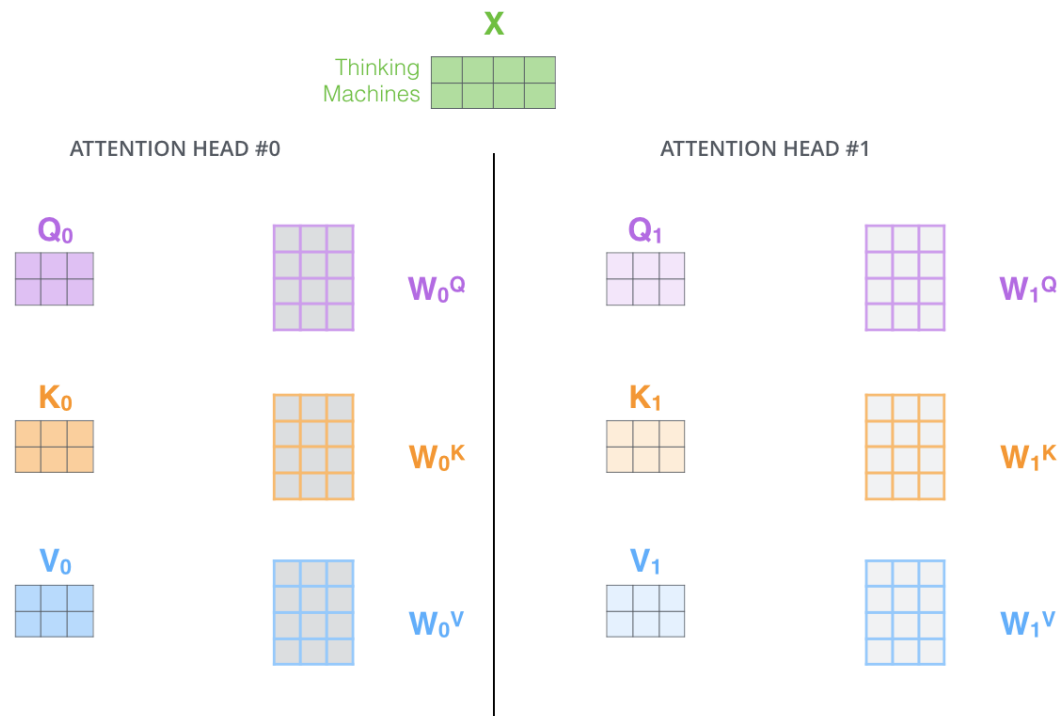
0.12

Matrix Calculation of Self-Attention

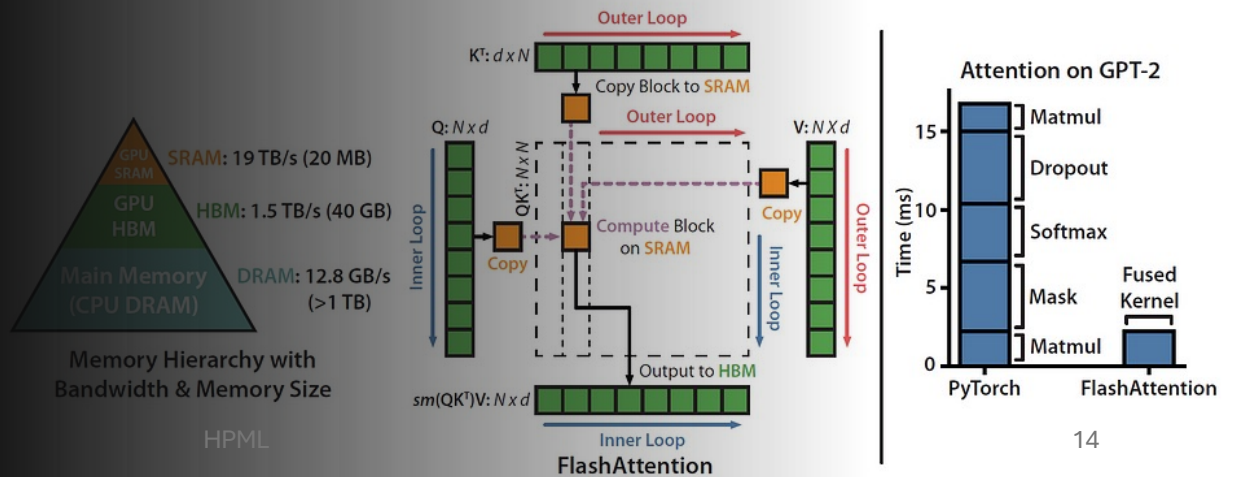
- **The first step** is to calculate the Query, Key, and Value matrices. We do that by packing our embeddings into a matrix **X** and multiplying it by the weight matrices we've trained (**W_Q**, **W_K**, **W_V**).
- Every row in the X matrix corresponds to a word in the input sentence.
- We can see the difference in the size of the embedding vector (512, or 4 boxes in the figure), and the q/k/v vectors (64, or 3 boxes in the figure)



Multiple Heads



FlashAttention



Motivation: Need to Model Longer Sequences



New Capabilities

Large context is required to understand books, plays, instruction manuals, long videos, etc.



More realistic outcomes

In computer vision, higher resolution can lead to better and more robust insights



Unleashing new applications

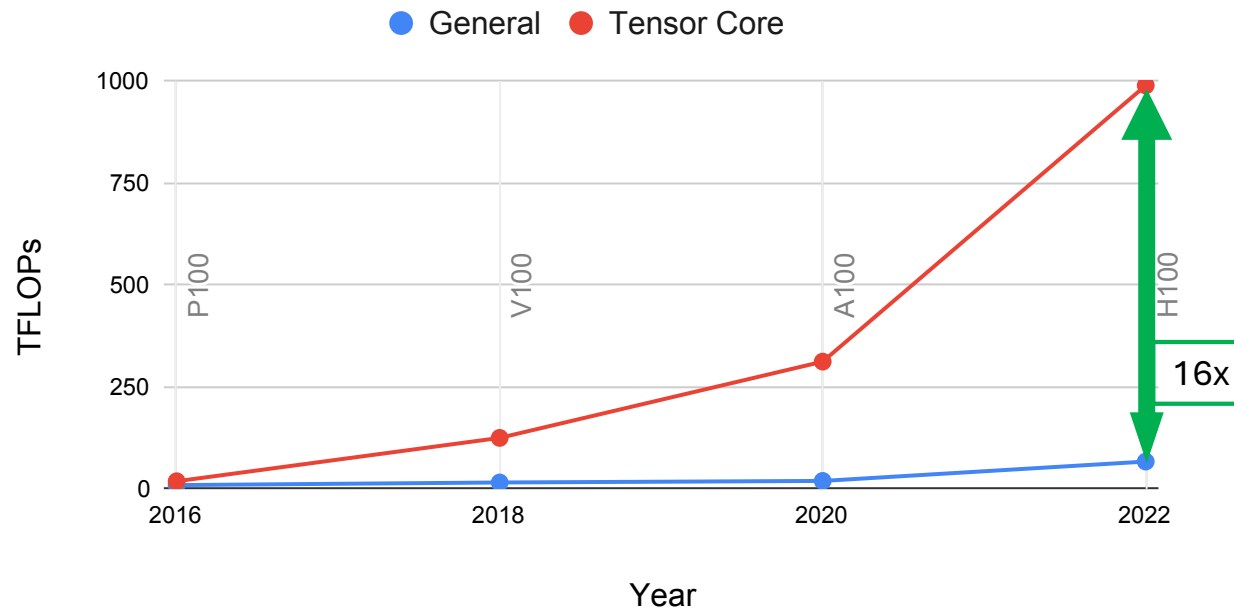
Time series, audio, video, medical imaging data are naturally modeled as very long sequences (sequences of millions of steps)

Challenge: How do we scale Transformers to longer sequences?

Modern Accelerators Compute Capabilities (e.g., NVidia Tensor Cores)

Tensor cores multiply 16x16 matrices (very roughly).

GPU TFLOPs over Time



Speed difference with tensor cores is **increasing**

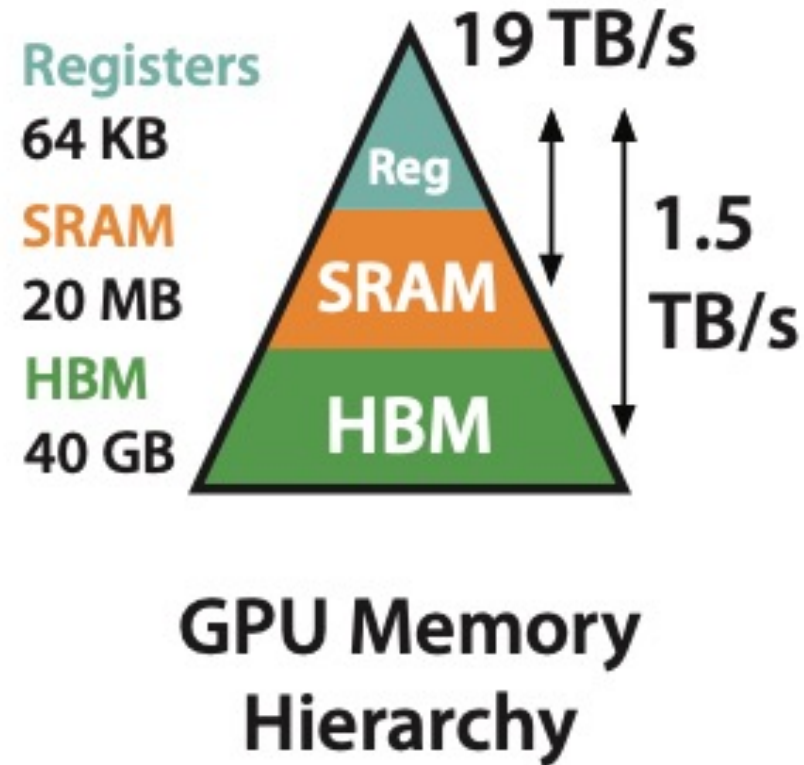
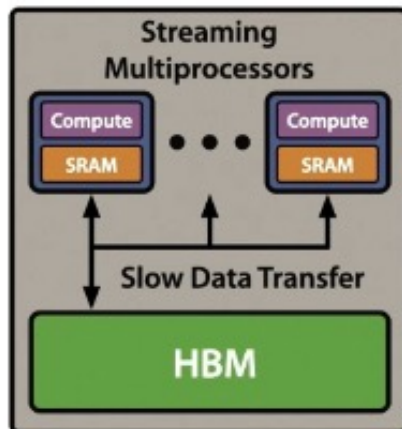
- 4x on V100,
- 8x on A100, and
- 16x on H100

With tensor cores versus without (across precisions).

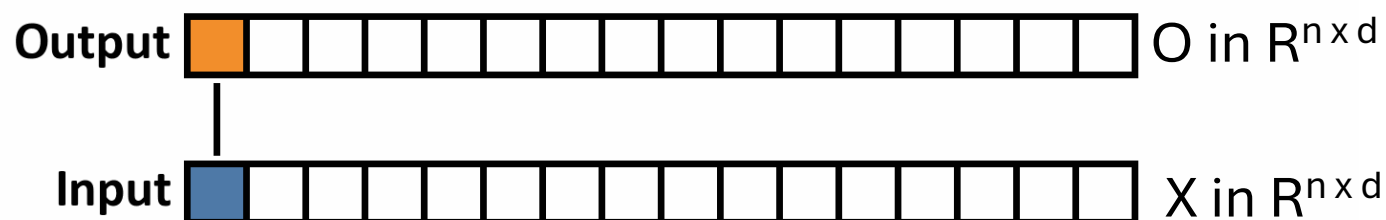
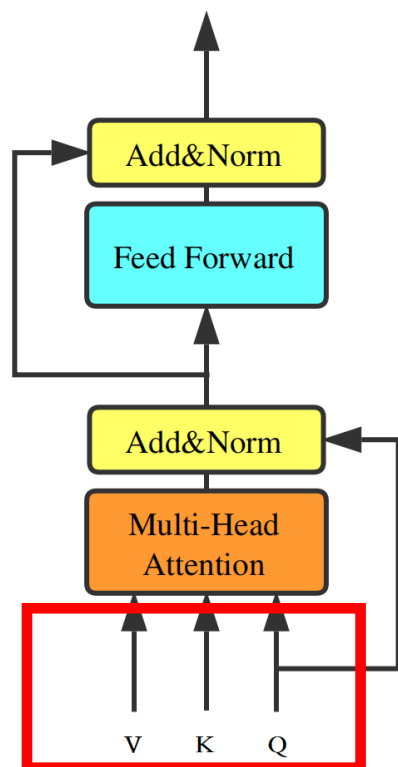
Modern GPU: Memory Hierarchy

Memory hierarchy

- Small and Fast
 - Registers
 - SRAM (Cache)
- Big and Slow
 - Memory



Attention (Visual Version of Single Head)

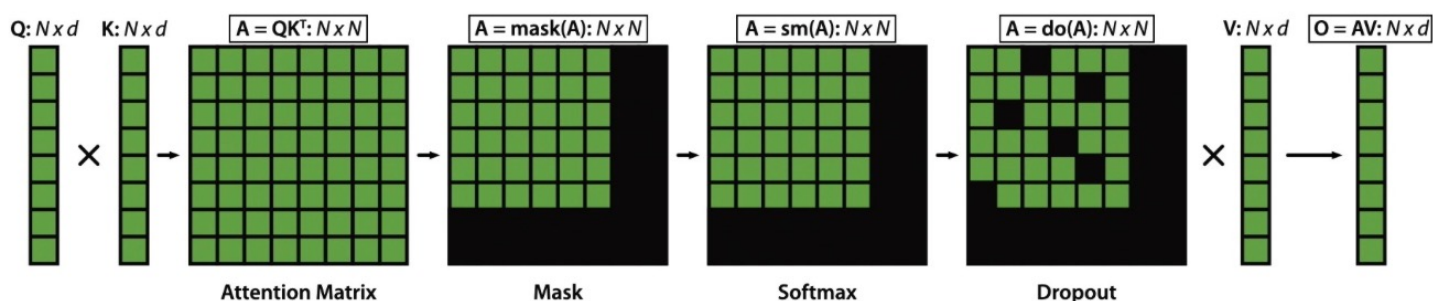


Each output “attends” to input positions.

- Causal (e.g., GPT) attends to preceding positions,
- Non-causal (e.g., BERT) to all positions.

Efficiency: running time **quadratic** (n^2) in sequence length (n)

Attention Bottleneck: Memory Reads/Writes



$$\mathbf{O} = \text{Dropout}(\text{Softmax}(\text{Mask}(\mathbf{QK}^T)))\mathbf{V}$$

**Naive implementation requires
repeated R/W from slow GPU HBM**

[FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness](#)

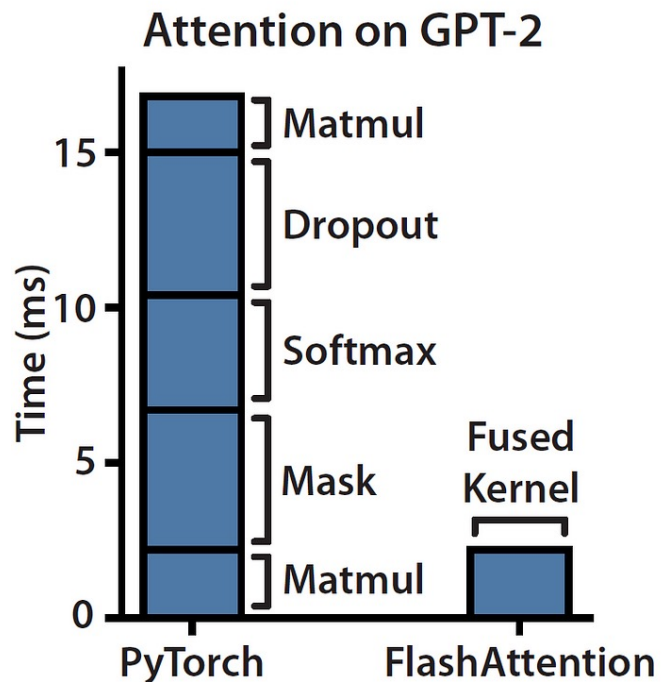
DATA MOVEMENT IS ALL YOU NEED: A CASE STUDY ON OPTIMIZING TRANSFORMERS

Andrei Ivanov^{*1} Nikoli Dryden^{*1} Tal Ben-Nun¹ Shigang Li¹ Torsten Hoefler¹

ABSTRACT

Transformers are one of the most important machine learning workloads today. Training one is a very compute-intensive task, often taking days or weeks, and significant attention has been given to optimizing transformers. Despite this, existing implementations do not efficiently utilize GPUs. We find that data movement is the key bottleneck when training. Due to Amdahl's Law and massive improvements in compute performance, training has now become memory-bound. Further, existing frameworks use suboptimal data layouts. Using these insights, we present a recipe for globally optimizing data movement in transformers. We reduce data movement by up to 22.91% and overall achieve a 1.30 $\times$ performance improvement over state-of-the-art frameworks when training a BERT encoder layer and 1.19 $\times$ for the entire BERT. Our approach is applicable more broadly to optimizing deep neural networks, and offers insight into how to tackle emerging performance bottlenecks.

The memory-bound problem



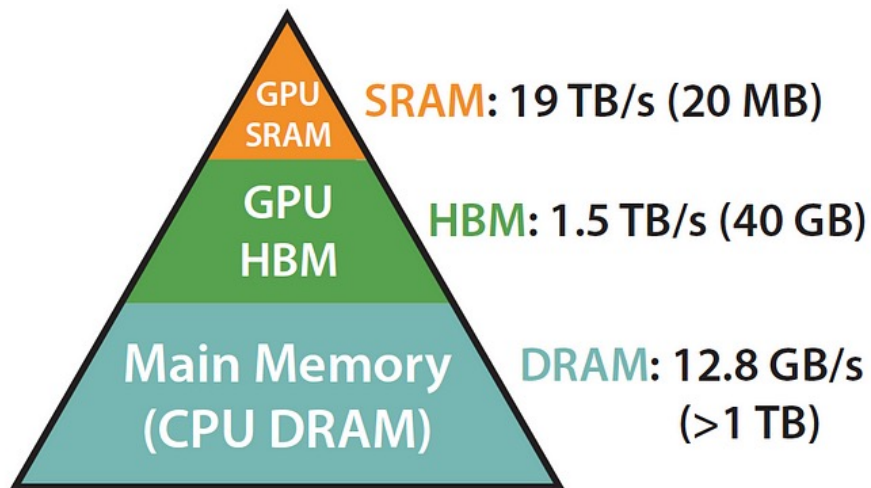
On Current AI Accelerators, **Attention is memory-bound.**

mostly consists of elementwise ops

Low arithmetic density

masking, softmax & dropout are the ops that are taking the bulk of the time and not matrix multiplication

Exploiting the Memory Hierarchy



Memory Hierarchy with
Bandwidth & Memory Size

Being “IO-aware” in practice boils down to exploiting the fact that SRAM is so much faster than HBM (“high bandwidth memory” — unfortunate name) by making sure to reduce the communication between the two.

Algorithm 0 Standard Attention Implementation

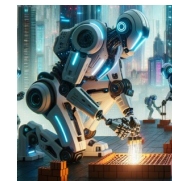
Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

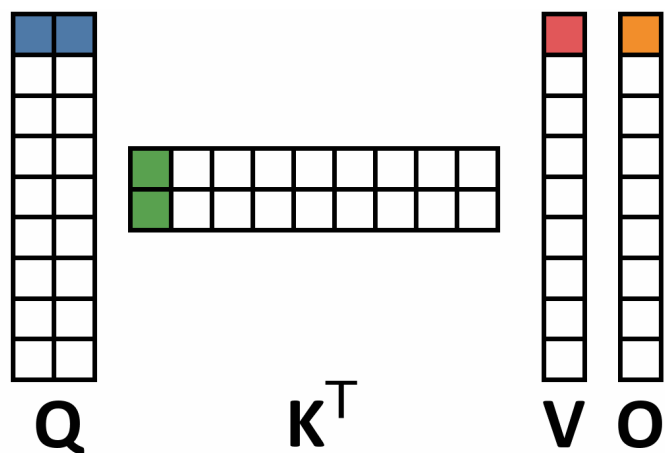
It’s basically treating HBM load/store ops as 0 cost (it’s not “IO-aware”).

Need to remove redundant HBM reads/writes.

- Minimize IO (HBM to SRAM) – not FLOPs
- Aggressive **fusion**: when you pull in data use it.



Minimize IO to HBM in Flash Attention

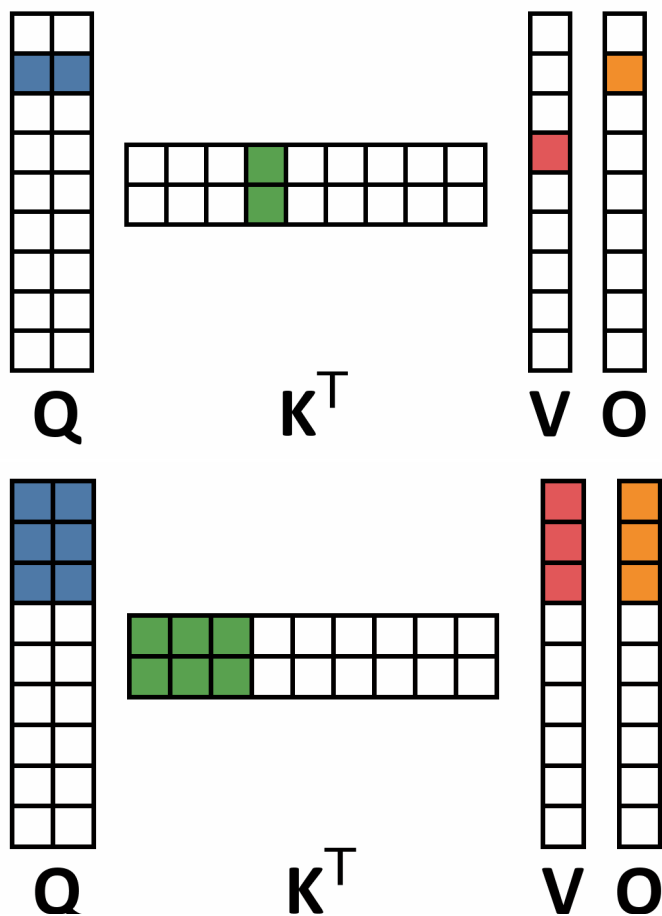


$r(Q) = 9$ is the rows of Q, $|Q|$ is number of tiles.

$$|Q| + r(Q)(|K| + |V|) + |O| = 18 + 9 \times (18 + 9) + 9 = 270 \text{ IO}$$

- Minimize IO (HBM to SRAM) – not FLOPs
- Aggressive **fusion**: when you pull in data use it.

Minimize IO to HBM in Flash Attention



$r(Q) = 9$ is the rows of Q , $|Q|$ is number of tiles.

$$|Q| + r(Q)(|K| + |V|) + |O| = 18 + 9 \times (18 + 9) + 9 = 270 \text{ IO}$$

Read 3 blocks at once, so $b(Q) = 9/3 = 3$.

$$|Q| + \mathbf{b(Q)}(|K| + |V|) + |O| = 18 + 3 \times (18 + 9) + 9 = 108 \text{ IO}$$

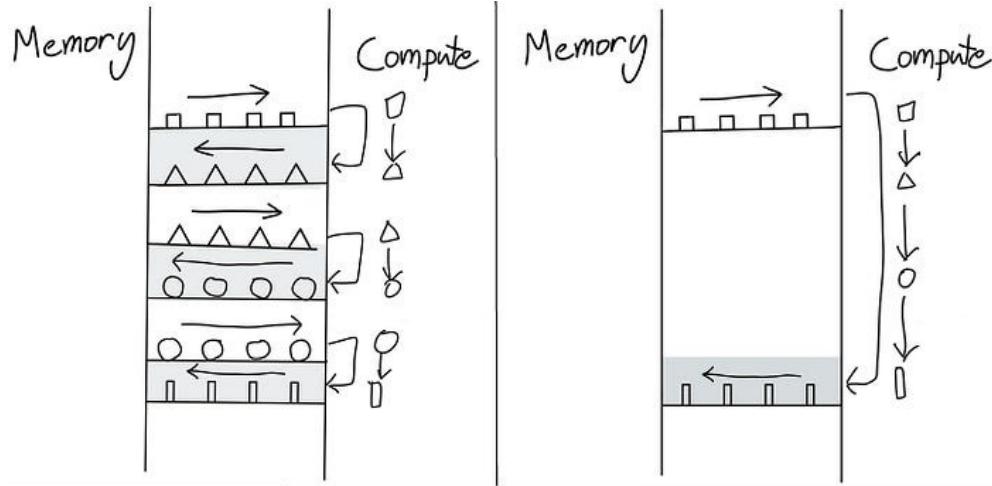
Same FLOPs but ~3x reduction in IO w/ block size 3.
Flash Attention A100 uses 8x8 blocks.

Key contributions

Implement a CUDA kernel to fuse all the attention operations (matmul, mask, softmax, etc) into a *single* GPU kernel.

Compute the softmax operation with neither computing nor storing the $N \times N$ attention matrix, where N is the number of tokens.

Kernel Fusion



Operator Fusion Simplified

https://horace.io/brrr_intro.html

- A **kernel** is a fancy way of saying “a GPU operation.”
- Fusion is combining multiple operations.
- You load from the HBM only once, execute the fused op, and then write the results back.
- This reduces the communication overhead.

1st Contribution: Fused CUDA Kernel

- By default, every tensor operation is implemented as follows:
 - 1) Read operation (read-op),
 - 2) compute-op, and
 - 3) write-op.
- The R/W operations can bottleneck the compute operations

Vanilla Attention	Flash Attention
1. Matmul_op (Q,K) a. Read Q,K to SRAM b. Compute matmul $A=Q \times K$ c. Write A to HBM 2. Mask_op a. Read A to SRAM b. Mask A into A' c. Write A' to HBM 3. Softmax_op a. Read A' to SRAM b. Softmax A' into A'' c. Write A'' to HBM	1. Read Q,K to SRAM 2. Compute $A = Q \times K$ 3. Mask A into A' 4. Softmax A' into A'' 5. Write A'' to HBM

Fused kernel:
Single kernel
that combines
all the 5 ops
into

Image credit: [Ahmed Taha](#)

2nd Contribution: Computing the softmax without realizing $N \times N$ Attention matrix A

$$s(x_i) = \frac{e^{x_i}}{\sum_{j=1}^N e^{x_j}}$$

Do we need to store the entire attention matrix A to compute the softmax denominator?



Image credit: [Ahmed Taha](#)

How to Reduce HBM Read/Writes: Compute by Blocks

- Main Challenges:
 1. Compute software reduction without access to full input
 2. Backward without the large matrix from the forward

The Key Ideas – Using old tricks

Flash attention boils down to 2 main ideas:

- **Tiling** (used during both in the forward & backward passes) — chunking the $N \times N$ softmax/scores matrix into blocks.
- **Recomputation** (used in the backward pass only) — Don't store the attention matrix from the forward pass; recompute it in the backward pass.

Implementation:

Fused CUDA Kernel for fine-grained control of memory accesses

Tiling Idea

Think of attention as a nested loop:
for each block of Q: ← **outer loop**
 for each block of K,V: ← **inner loop**
 compute partial attention

● Inner Loop (Blue in the slide)

→ Sweeps over K and V blocks for one tile of Q

Fix **one tile of Q** (e.g., 128 queries)

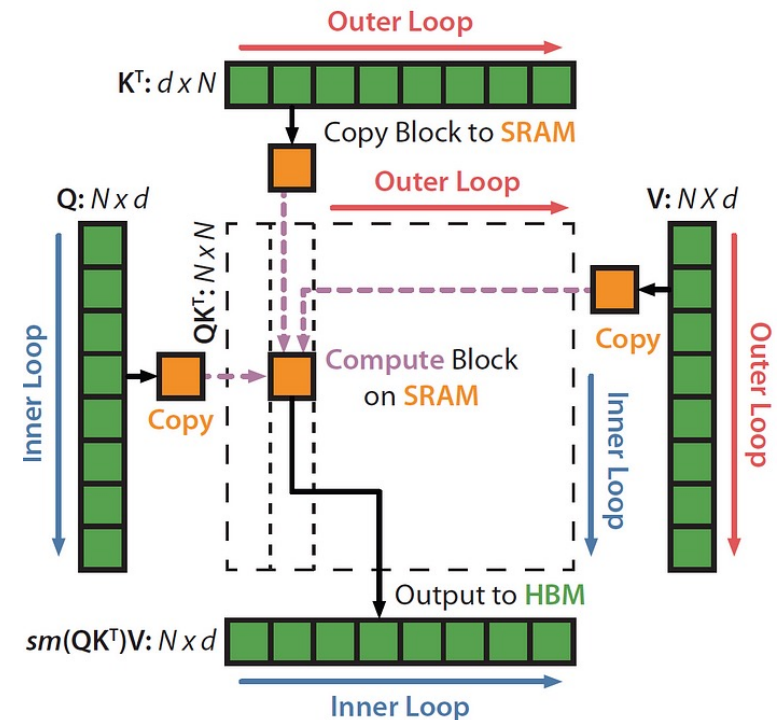
Load a **small tile of K and V** from HBM → SRAM

Compute $Q \times K^T$ for that small tile

Update partial softmax

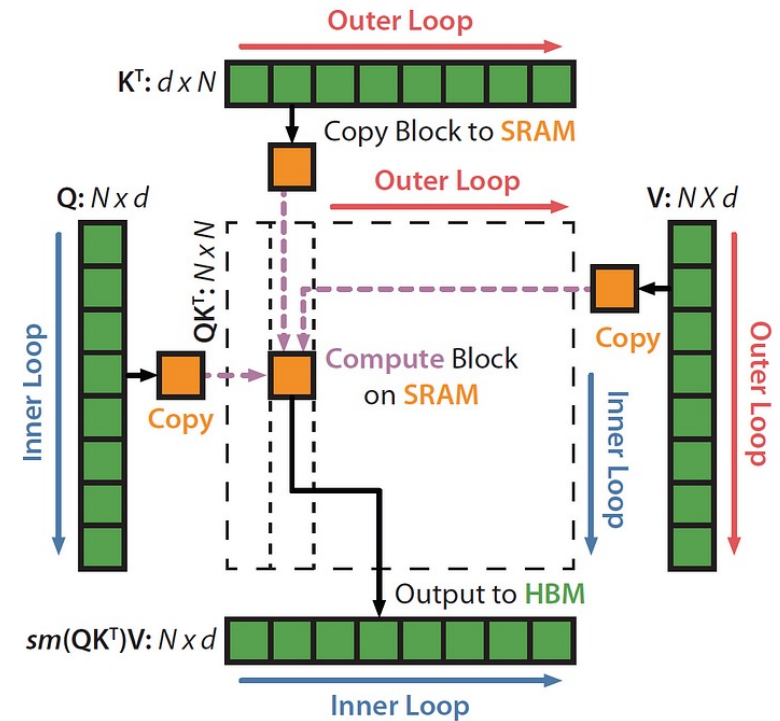
Move to next K/V tile

👉 Inner loop = “scan all K/V tiles for this particular Q tile.”



Tiling Idea

- Decomposing large softmax into smaller ones
- Steps:
 1. Load inputs block by block from HBM to SRAM
 2. On chip, compute attention output w.r.t to that block
 3. Update output in HBM by scaling



Tiling Idea

Think of attention as a nested loop:
for each block of Q: ← **outer loop**
 for each block of K,V: ← **inner loop**
 compute partial attention

● Inner Loop (Blue in the slide)

→ Sweeps over K and V blocks for one tile of Q

Fix **one tile of Q** (e.g., 128 queries)

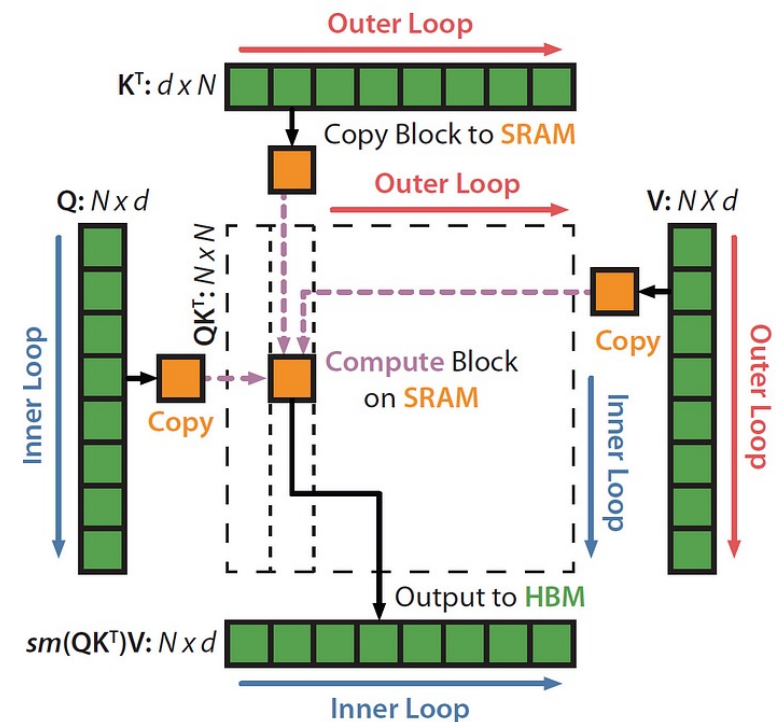
Load a **small tile of K and V** from HBM → SRAM

Compute $Q \times K^T$ for that small tile

Update partial softmax

Move to next K/V tile

👉 Inner loop = “scan all K/V tiles for this particular Q tile.”



Tiling Idea

Think of attention as a nested loop:
for each block of Q: ← **outer loop**
 for each block of K,V: ← **inner loop**
 compute partial attention

● Inner Loop (Blue in the slide)

→ **Sweeps over K and V blocks for one tile of Q**

Fix **one tile of Q** (e.g., 128 queries)

Load a **small tile of K and V** from HBM → SRAM

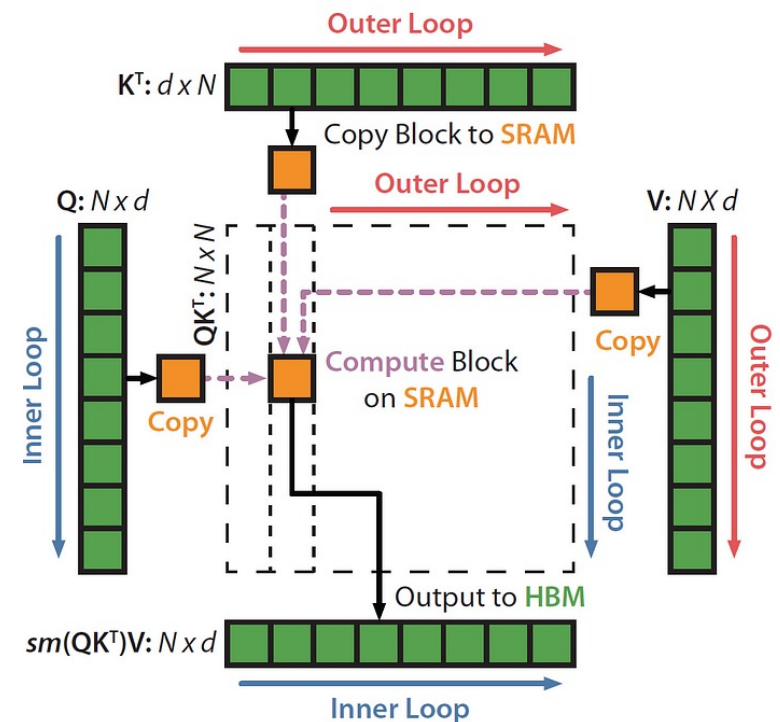
Compute $Q \times K^T$ for that small tile

Update partial softmax

Move to next K/V tile

Inner loop = “scan all K/V tiles for this particular Q tile.”

HPML



Tiling Idea

Think of attention as a nested loop:
for each block of Q: $\leftarrow$ outer loop
for each block of K,V: $\leftarrow$ inner loop
compute partial attention

● Outer Loop (Red in the slide)

→ Sweeps over tiles of Q

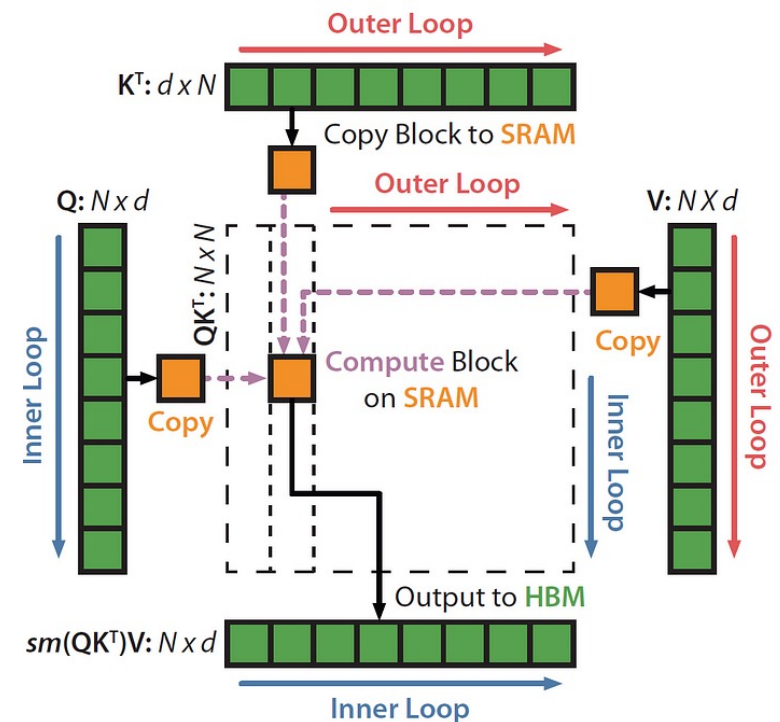
After finishing all K/V tiles for Q-block 1:

Move to the next tile of Q

Repeat inner loop

Produce next chunk of the output

Outer loop = “move to the next group of queries.”



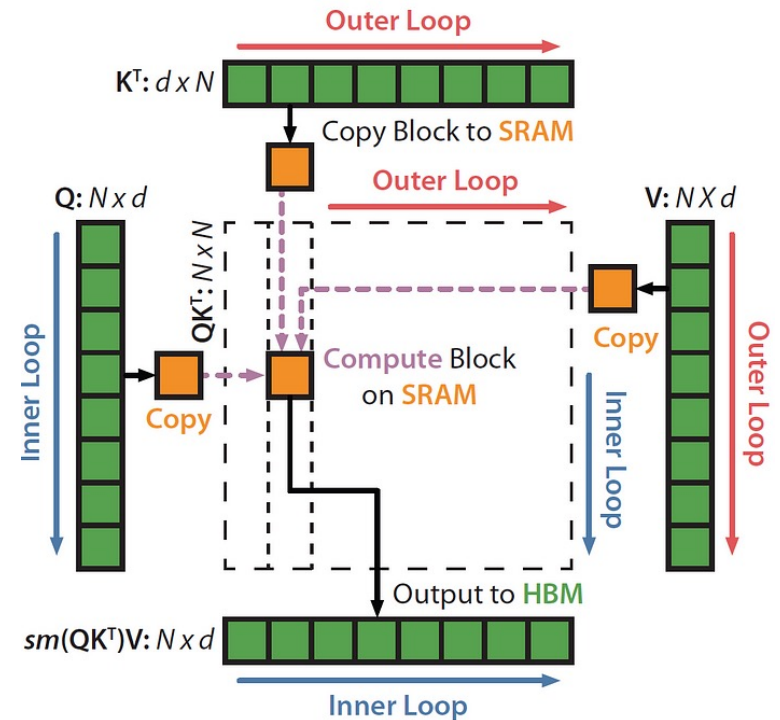
Tiling Idea

Think of attention as a nested loop:
 for each block of Q: $\leftarrow$ outer loop
 for each block of K,V: $\leftarrow$ inner loop
 compute partial attention

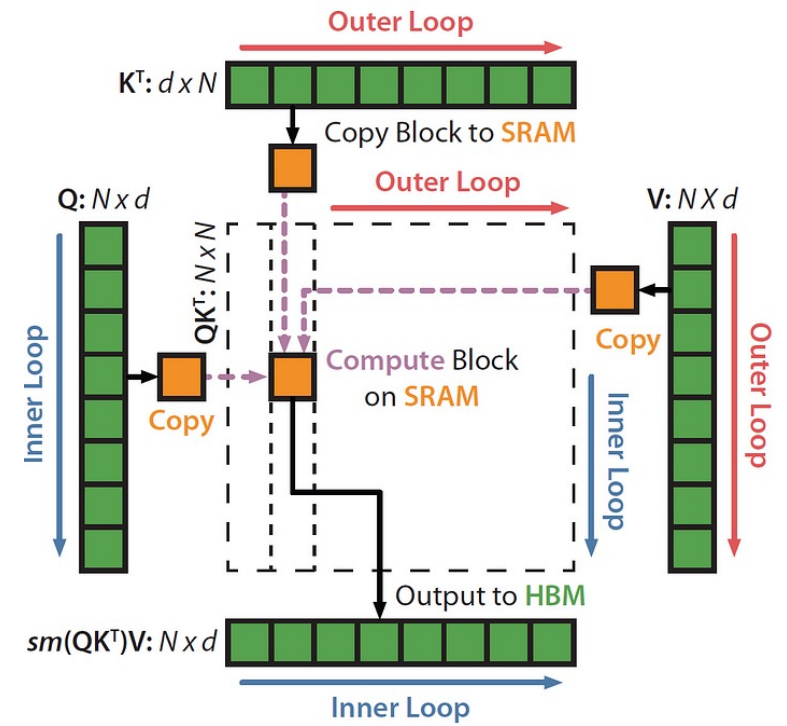
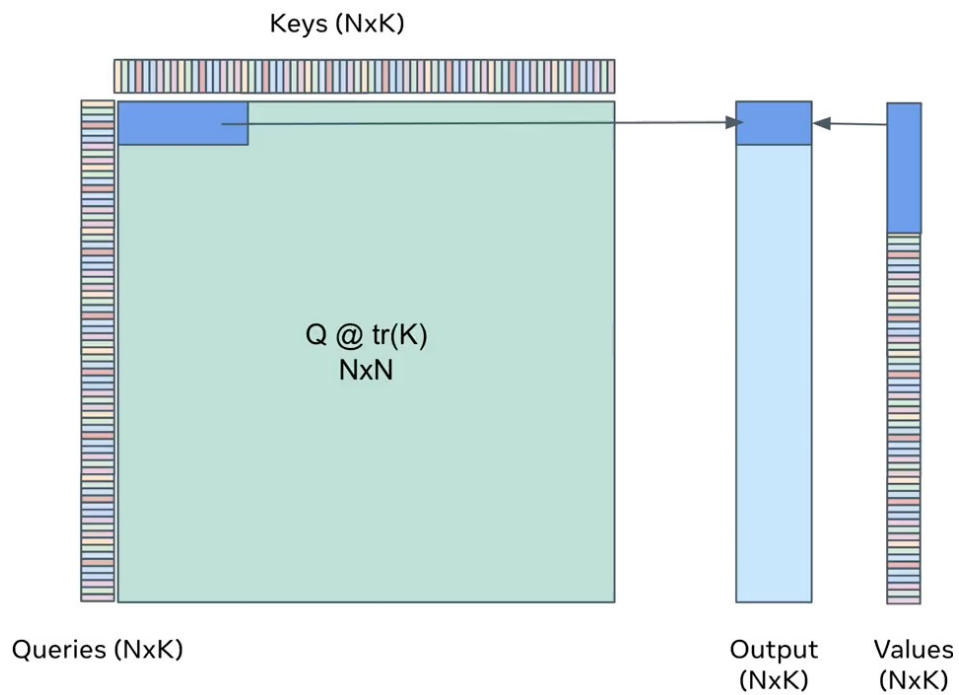
- Decomposing large softmax into smaller ones:

$$\text{softmax}([A_1, A_2]) = [\alpha \text{softmax}(A_1), \beta \text{softmax}(A_2)] .$$

$$\text{softmax}([A_1, A_2]) \begin{bmatrix} V_1 \\ V_2 \end{bmatrix} = \alpha \text{softmax}(A_1) V_1 + \beta \text{softmax}(A_1) V_2 .$$



Illustration



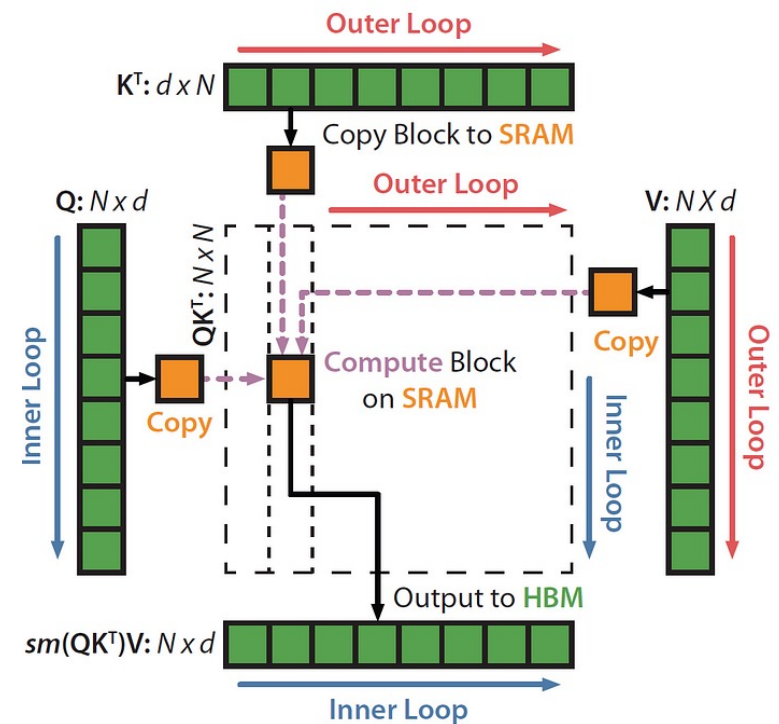
Recomputation Technique

- By storing softmax normalization factors from forward pass, we can quickly recompute attention in the backward from inputs in stored in SRAM

Attention	Standard	FLASHATTENTION
GFLOPs	66.6	75.2
HBM R/W (GB)	40.3	4.4
Runtime (ms)	41.7	7.3

Forward and backward runtime of standard attention vs FlashAttention for GPT-2 medium (seq. Len 1024, head dim. 64. batch size 64 on A100 GPU)

HPML



<https://arxiv.org/pdf/2205.14135.pdf>

39

Attention Softmax Bottleneck

- The main hurdle in getting the tiling approach to work is Softmax
- Softmax couples all of the score columns together.
- To compute the i^{th} output of a Softmax:
 - z_i is the i^{th} score (key-query dot product), and the output is the i^{th} token's probability that we later use to weight the value vector.
- The denominator is the issue
 - To compute how much a particular i^{th} token from the input sequence pays attention to other tokens in the sequence, you'd need to have all of those scores readily available (denoted here by z_j) in SRAM.
- SRAM is severely limited in its capacity. You can't just load the whole thing.
 - N (sequence length) can be 1000 or even 100k tokens. So, N^2 explodes fairly quickly.

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

How to deal with Softmax Challenge (1/2)

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

- Chop the Softmax computation into smaller blocks while producing precisely the same result.
- We can take the first B scores (x_1 through x_B) and compute their softmax.
- **Partial computation of softmax (will iteratively get to the correct softmax numbers).**

$$m(x) := \max_i x_i, \quad f(x) := [e^{x_1 - m(x)} \quad \dots \quad e^{x_B - m(x)}], \quad \ell(x) := \sum_i f(x)_i, \quad \text{softmax}(x) := \frac{f(x)}{\ell(x)}.$$

How to deal with Softmax Challenge (2/2)

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

- We can combine those per-block partial softmax numbers smartly so that the final result is correct.
- To compute the softmax for the scores belonging to the first 2 blocks (of size B), you have to keep track of 2 statistics for each block: $\mathbf{m}(\mathbf{x})$ (maximum score) and $\ell(\mathbf{x})$ (sum of exp scores).
- Continue recursively all the way up to the last, (N/B) -th, block, at which point you have the N -dimensional correct softmax output.

For vectors $x^{(1)}, x^{(2)} \in \mathbb{R}^B$, we can decompose the softmax of the concatenated $x = [x^{(1)} \ x^{(2)}] \in \mathbb{R}^{2B}$ as:

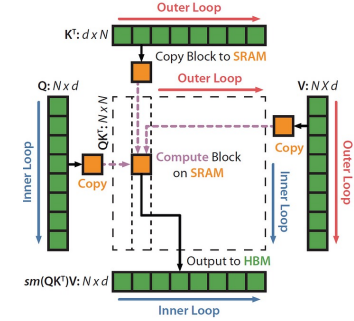
$$\begin{aligned} m(x) &= m([x^{(1)} \ x^{(2)}]) = \max(m(x^{(1)}), m(x^{(2)})), & f(x) &= \begin{bmatrix} e^{m(x^{(1)})-m(x)} f(x^{(1)}) & e^{m(x^{(2)})-m(x)} f(x^{(2)}) \end{bmatrix}, \\ \ell(x) &= \ell([x^{(1)} \ x^{(2)}]) = e^{m(x^{(1)})-m(x)} \ell(x^{(1)}) + e^{m(x^{(2)})-m(x)} \ell(x^{(2)}), & \text{softmax}(x) &= \frac{f(x)}{\ell(x)}. \end{aligned}$$

FlashAttention Algorithm

Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
 - 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
 - 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
 - 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
 - 5: **for** $1 \leq j \leq T_c$ **do**
 - 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
 - 7: **for** $1 \leq i \leq T_r$ **do**
 - 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
 - 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
 - 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
 - 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
 - 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
 - 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
 - 14: **end for**
 - 15: **end for**
 - 16: Return $\mathbf{O}$.
-



- *The algorithm assumes we have a batch of size 1 (i.e., single sequence) and a single attention head;*
- *It can be easily scaled by simply parallelizing across GPU's streaming multiprocessors.*
- *We ignore dropout & masking for the time being.*

(row block size), B_c (column block size)

FlashAttention Algorithm

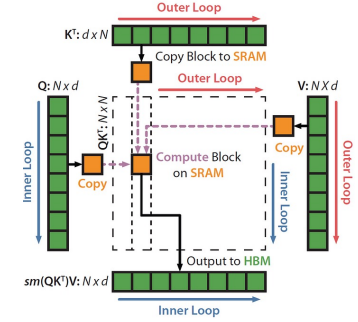
Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lfloor \frac{M}{4d} \rfloor$, $B_r = \min(\lfloor \frac{M}{4d} \rfloor, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

Notation: d — attention head dimension, B_r (row block size), B_c (column block size)

HPML



Step 0: HBM's capacity is measured in GBs (e.g., RTX 3090 has 24 GBs of HBM, A100 has 40–80 GB, etc.) so allocating $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ is not an issue.

FlashAttention Algorithm

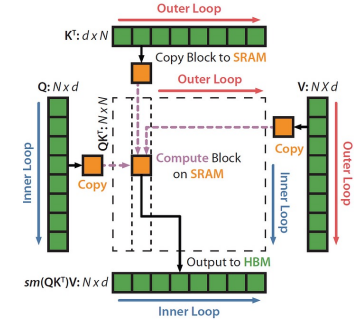
Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

Notation: d — attention head dimension, B_r (row block size), B_c (column block size)

HPML



Step 1: Compute the row/column block sizes.

Why $\text{ceil}(M/4d)$? Because query, key, and value vectors are d -dimensional, we must combine them into the output d -dimensional vector (o). So, this size allows us to max out SRAM capacity with q , k , v , and o vectors.

FlashAttention Algorithm

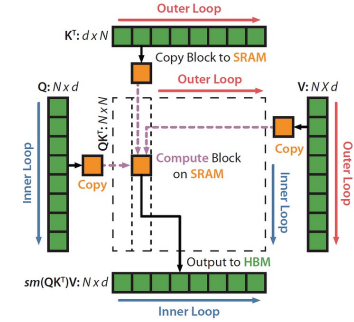
Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

Notation: d — attention head dimension, B_r (**row** block size), B_c (**column** block size)

HPML



Step 2:

- initialize the output matrix $\mathbf{O}$ with all 0s as it acts as an accumulator
- Initialize the ℓ to 0. Its purpose is to hold the cumulative denominator for the softmax (sum of exp of the scores)
- Initialize m to inf because it holds the row-wise max scores.

FlashAttention Algorithm

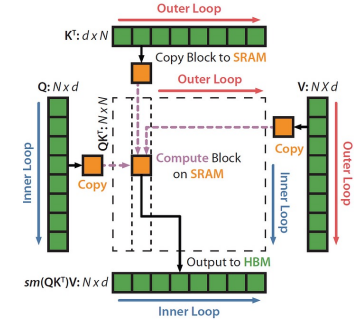
Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

Notation: d — attention head dimension, B_r (row block size), B_c (column block size)

HPML



Step 3:

- split the $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ into blocks using the block sizes from Step 1.

FlashAttention Algorithm

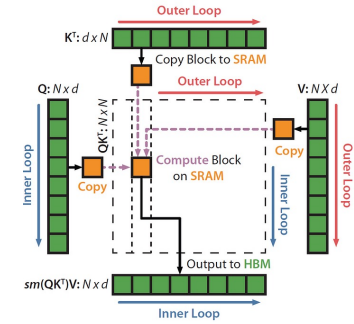
Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$ of size $B_r \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

Notation: d — attention head dimension, B_r (**row** block size), B_c (**column** block size)

HPML



Step 4:

- split $\mathbf{O}$, ℓ , m into blocks (same block size as $\mathbf{Q}$).

FlashAttention Algorithm

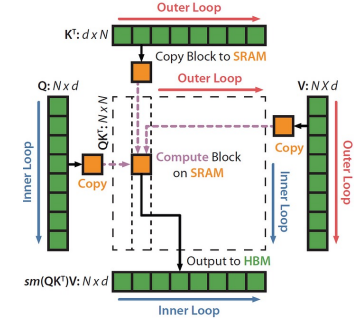
Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

Notation: d — attention head dimension, B_r (row block size), B_c (column block size)

HPML



Step 5:

- Loop across the columns (key/value vectors) as shown in the outer loop in the diagram

FlashAttention Algorithm

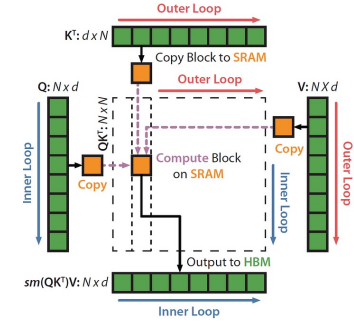
Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

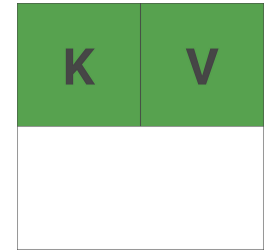
Notation: d — attention head dimension, B_r (**row** block size), B_c (**column** block size)

HPML



Step 6:

- Load the $\mathbf{K}_j$ and $\mathbf{V}_j$ blocks from HBM to SRAM.
- Because of the way we constructed the block sizes, 50% of the SRAM is still unoccupied at this point (dedicated to $\mathbf{Q}$ and $\mathbf{O}$).



FlashAttention Algorithm

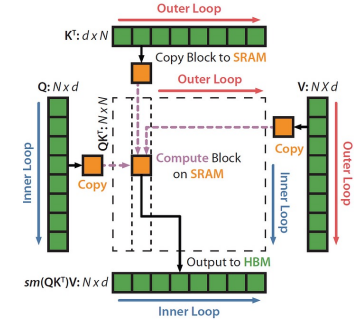
Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

Notation: d — attention head dimension, B_r (**row** block size), B_c (**column** block size)

HPML



Step 7:

- Start the inner loop across the rows i.e., across query vectors

FlashAttention Algorithm

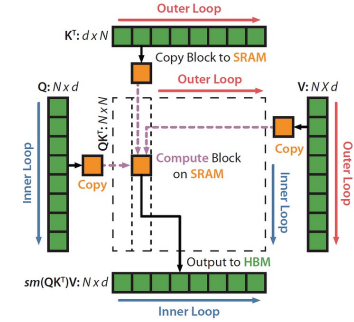
Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

Notation: d — attention head dimension, B_r (**row** block size), B_c (**column** block size)

HPML



Step 8:

- Load $\mathbf{Q}_i$ ($B_r \times d$) and $\mathbf{O}_i$ ($B_r \times d$) blocks, as well as ℓ_i (B_r) & m_i (B_r) into SRAM.

FlashAttention Algorithm

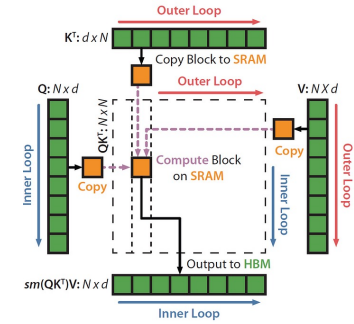
Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

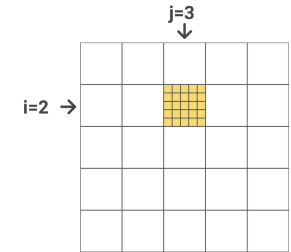
Notation: d — attention head dimension, B_r (row block size), B_c (column block size)

HPML



Step 9:

- Compute the dot product between $\mathbf{Q}_i$ ($B_r \times d$) and $\mathbf{K}_j$ transposed ($d \times B_c$) to get the scores ($B_r \times B_c$). As you can see we don't have the whole $N \times N$ S(scores) matrix "materialized". Only a fraction of it ($\mathbf{S}_{ij}$).



FlashAttention Algorithm

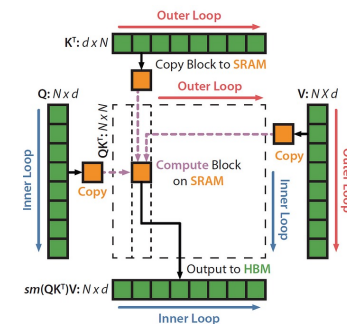
Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

Notation: d — attention head dimension, B_r (row)

HPML



Step 10:

- Compute $\tilde{m}_{ij}$, $\tilde{\ell}_{ij}$, and $\tilde{\mathbf{P}}_{ij}$ using the scores computed in the previous step.
- $\tilde{m}_{ij}$ is computed row-wise, find the max element for each of the above rows.
- We get $\tilde{\mathbf{P}}_{ij}$ by applying elementwise ops:
 - Normalization — take the row max and subtract it from row scores
 - Exp
 - $\tilde{\ell}_{ij}$ is simply a row-wise sum of the matrix $\tilde{\mathbf{P}}$.

FlashAttention Algorithm

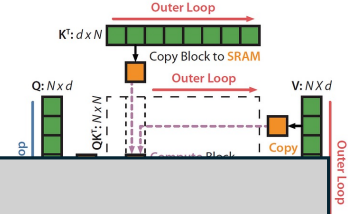
Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

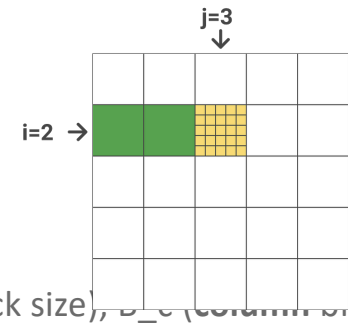
Notation: d — attention head dimension, B_r (row block size), B_c (column block size)

HPML



Step 11:

- Compute m_{new_i} and ℓ_{new_i}
- m_i contains row-wise maximums for all of the blocks that came before ($j=1$ & $j=2$, colored in green).
- $m_{\sim i_j}$ contains the row-wise maximums for the current block (colored in yellow).
- To get the m_{new_i} we just have to apply a max between $m_{\sim i_j}$ & m_i .



FlashAttention Algorithm

Algorithm

Require

- 1: Set
- 2: Init
- 3: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 4: Divide $\mathbf{K}$ into T_c blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$ of size $B_c \times d$ each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}, m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

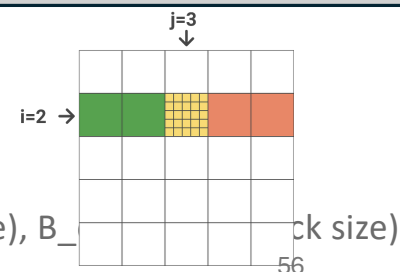
$$\text{Write } \mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$$

$$f(x) = \begin{bmatrix} e^{m(x^{(1)}) - m(x)} f(x^{(1)}) & e^{m(x^{(2)}) - m(x)} f(x^{(2)}) \end{bmatrix}$$

blocks

Step 12:

- Implement the partial softmax sum
- **The first part**, underlined in green, updates the current softmax estimate for the blocks before the current block in the same row of blocks.
- **The second part**, underlined in yellow, multiplies the $\tilde{\mathbf{P}}_{ij}$ matrix with the block of $\mathbf{V}$ vectors ($\mathbf{V}_j$).
- The e^x terms are there to modify the matrix $\tilde{\mathbf{P}}_{ij}$ & $\mathbf{O}_i$ by canceling out the m from the previous iteration and instead updating it with the latest estimate (m_{new_i}) that contains the row-wise max so far.



Notation: d — attention head dimension, B_r (row block size), B_c (column block size)

HPML

56

FlashAttention Algorithm

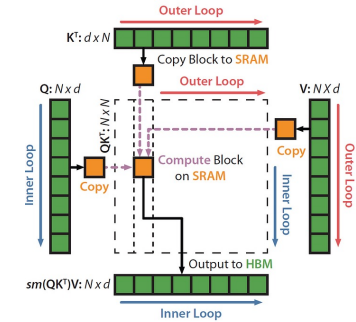
Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

Notation: d — attention head dimension, B_r (**row** block size), B_c (**column** block size)

HPML



Step 13:

- Write the newest cumulative statistics (ℓ_i & m_i) back to HBM. Notice these are of dimension B_r .

FlashAttention Algorithm

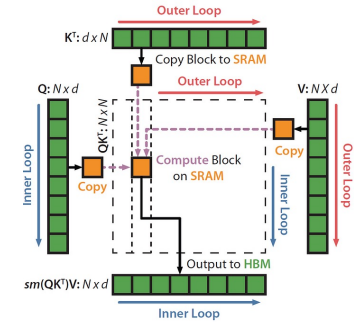
Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

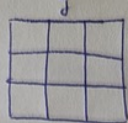
Notation: d — attention head dimension, B_r (row block size), B_c (column block size)

HPML



Step 14, 15, and 16:

- Once the nested for loop is over, $\mathbf{O}$ ($N \times d$) will contain the final result: **attention-weighted value vectors for each of the input tokens**

Assume i  ← Let's focus on this row ($i=1$) $(3 \times 3 \text{ blocks})$

1st iteration: $i=1, j=1$

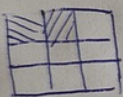
$$O_1 = \text{diag}(l_1^{\text{new}})^{-1} \left(\text{diag}(l_1) \cdot e^{\tilde{m}_{11}-m_1^{\text{new}}} O_1 + e^{\tilde{m}_{11}-m_1^{\text{new}}} \tilde{P}_{11} V_1 \right)$$



$$\Rightarrow O_1 = \text{diag}(l_1^{\text{new}})^{-1} \left(e^{\tilde{m}_{11}-m_1^{\text{new}}} \tilde{P}_{11} V_1 \right)$$

these 2 are the same

2nd iteration: $i=1, j=2$



$$O_1 = \text{diag}(l_1^{\text{new}})^{-1} \left(\text{diag}(l_1) \cdot e^{\tilde{m}_{11}-m_1^{\text{new}}} \left[\frac{e^{\tilde{m}_{12}-m_1^{\text{new}}} \tilde{P}_{11} V_1}{\text{diag}(l_1^{\text{new}})} \right] + e^{\tilde{m}_{12}-m_1^{\text{new}}} \tilde{P}_{12} V_2 \right)$$

$\tilde{P}_{11}$ in this iteration
is equal to
 $\tilde{P}_{11}^{\text{new}}$ from
previous iteration

$$O_1 = \text{diag}(l_1^{\text{new}})^{-1} \left(e^{\tilde{m}_{11}-m_1^{\text{new}}} \tilde{P}_{11} V_1 + e^{\tilde{m}_{12}-m_1^{\text{new}}} \tilde{P}_{12} V_2 \right)$$

$$\tilde{P}_{11} = \begin{bmatrix} e^{x_{11}-\tilde{m}_{11}} & e^{x_{12}-\tilde{m}_{11}} \\ e^{x_{21}-\tilde{m}_{11}} & e^{x_{22}-\tilde{m}_{11}} \end{bmatrix} \quad (\text{assuming blocks have only 2 vectors})$$

$$\textcircled{1} \begin{bmatrix} e^{\tilde{m}_{11}-m_1^{\text{new}}} & 0 \\ 0 & e^{\tilde{m}_{11}-m_1^{\text{new}}} \end{bmatrix} \cdot \begin{bmatrix} e^{x_{11}-\tilde{m}_{11}} & e^{x_{12}-\tilde{m}_{11}} \\ e^{x_{21}-\tilde{m}_{11}} & e^{x_{22}-\tilde{m}_{11}} \end{bmatrix} \Rightarrow \begin{bmatrix} e^{x_{11}-m_1^{\text{new}}} & e^{x_{12}-m_1^{\text{new}}} \\ e^{x_{21}-m_1^{\text{new}}} & e^{x_{22}-m_1^{\text{new}}} \end{bmatrix}$$

Similar thing happens to $e^{\tilde{m}_{12}-m_1^{\text{new}}} \tilde{P}_{12}$

we end up with

$$\begin{bmatrix} e^{x_{13}-m_1^{\text{new}}} & e^{x_{14}-m_1^{\text{new}}} \\ e^{x_{23}-m_1^{\text{new}}} & e^{x_{24}-m_1^{\text{new}}} \end{bmatrix}$$

Finally iteration 2:

$$O_1 = \text{diag}(l_1^{\text{new}})^{-1} \left(\begin{bmatrix} e^{x_{11}-m_1^{\text{new}}} & e^{x_{12}-m_1^{\text{new}}} \\ e^{x_{21}-m_1^{\text{new}}} & e^{x_{22}-m_1^{\text{new}}} \end{bmatrix} V_1 + \text{O} \cdot V_2 \right)$$

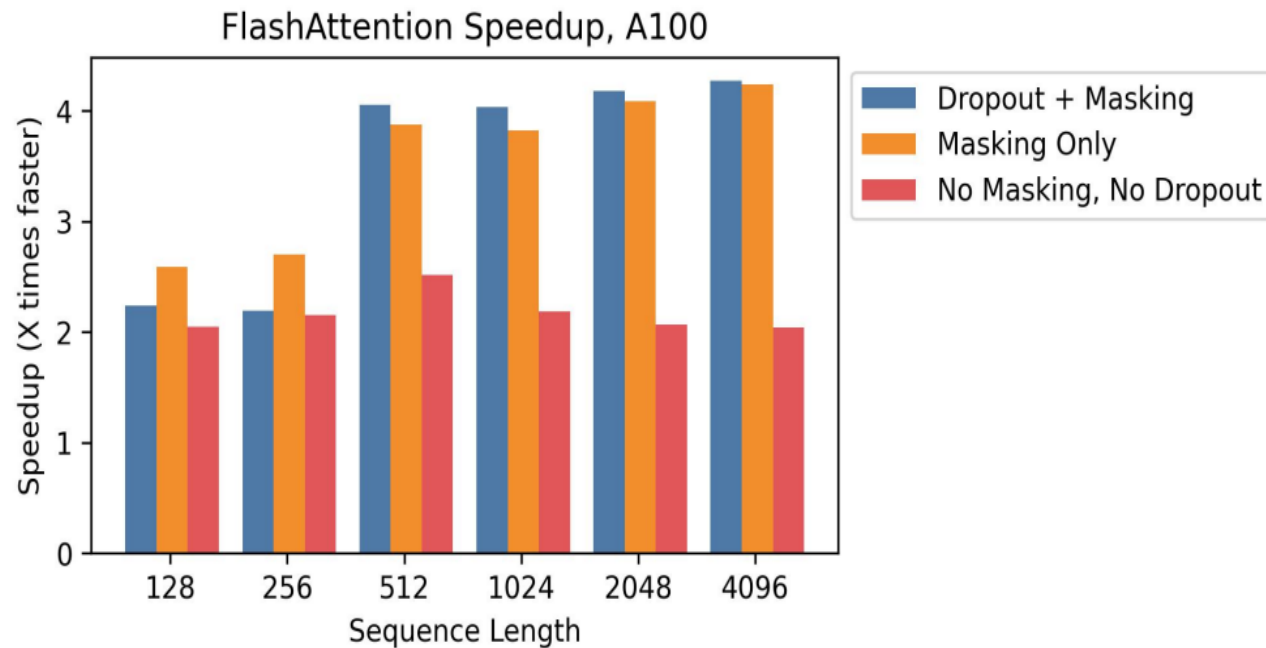
all normalized with the
Latest estimate of m_1^{new} !!!

Content Credit: [Aleksa Gordić](https://gordicaleksa.medium.com/eli5-flash-attention-5c44017022ad)

<https://gordicaleksa.medium.com/eli5-flash-attention-5c44017022ad>

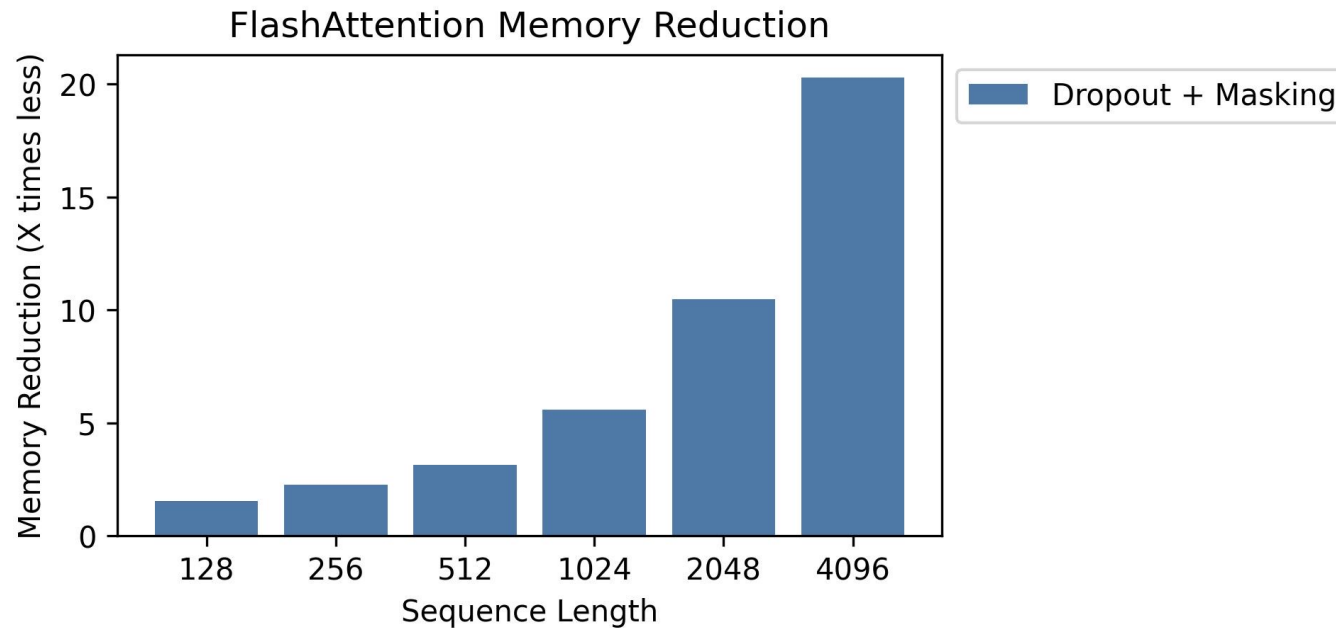
FlashAttention Performance – Speedup

- 2-4x speedup with no approximation



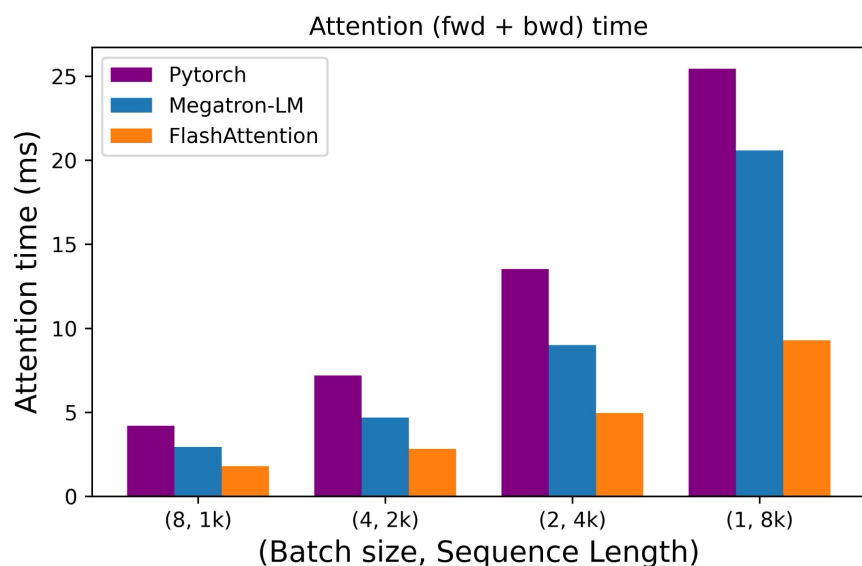
FlashAttention Performance – Memory Reduction

- 10-20x memory reduction – memory linear in sequence length



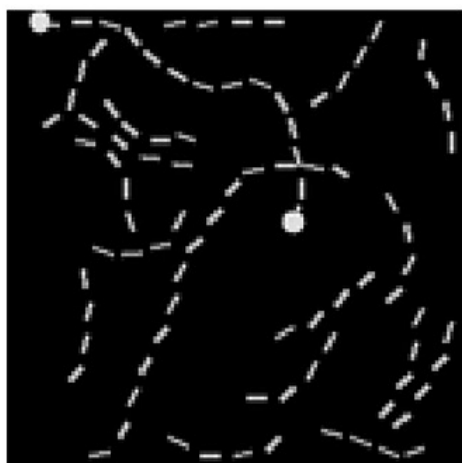
End-to-end Training Benchmark

- Training Transformers of size up to 2.7B on sequences of length 8K
- 2.2-2.7x faster attention for long sequence lengths (8k)

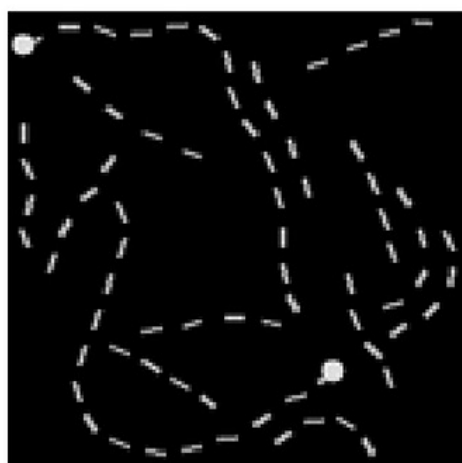


BERT Implementation	Training time (minutes)
Nvidia MLPerf 1.1 [58]	20.0 ± 1.5
FLASHATTENTION (ours)	17.4 ± 1.4

Solving Path-X and Path-256 benchmarks



Positive Example



Negative Example

This is a challenging benchmark where the task is to classify whether two points in a black and white 128×128 (or 256×256) image have a path connecting them.

Path-X and Path-256 require sequence Lengths of 16k/64k

Model	Path-X	Path-256
Transformer	✗	✗
Linformer [84]	✗	✗
Linear Attention [50]	✗	✗
Performer [12]	✗	✗
Local Attention [80]	✗	✗
Reformer [51]	✗	✗
SMYRF [19]	✗	✗
FLASHATTENTION	61.4	✗
Block-sparse FLASHATTENTION	56.0	63.1

FlashAttention is the first transformer model that achieves non-random performance on Path-X and Path-256 benchmarks.

Massive Adoption

<https://github.com/Dao-AILab/flash-attention?tab=readme-ov-file>

Wide adoption and various implementations:

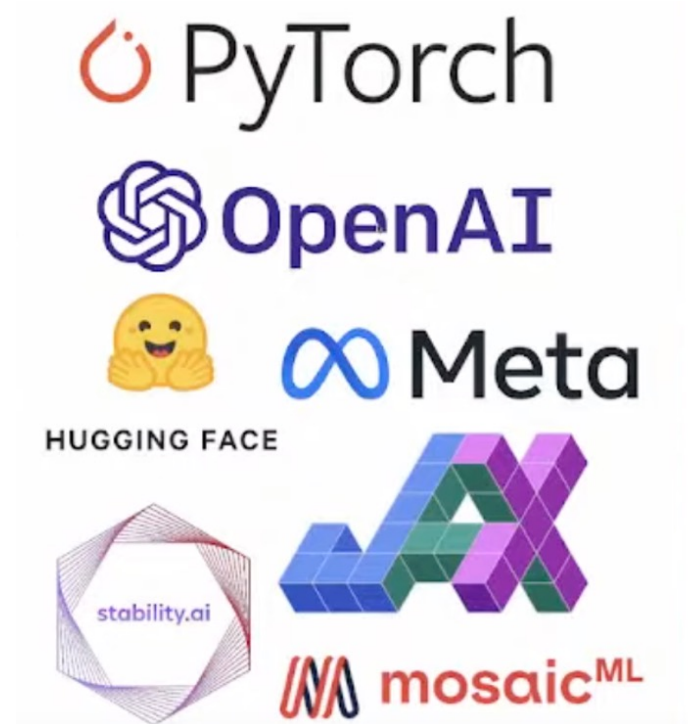
<https://github.com/Dao-AILab/flash-attention/blob/main/usage.md>

FlashAttention adoption

We've been very happy to see FlashAttention being adopted by many organizations and research labs to speed up their training / inference (within 6 months after FlashAttention's release, at the time of writing). This page contains a partial list of places where FlashAttention is being used. If you'd like to add links to your organization / product / codebase, please open a PR or email us. We'd very much like to hear from you!

Integrated into machine learning frameworks

- Pytorch: [integrated](#) into core Pytorch in nn.Transformer.
- Huggingface's [transformers](#) library. [On-going](#), blogpost coming soon.
- Microsoft's [DeepSpeed](#): FlashAttention is [integrated](#) into DeepSpeed's inference engine.
- Nvidia's [Megatron-LM](#). This library is a popular framework on training large transformer language models at scale.
- MosaicML [Composer](#) library. Composer is a library for efficient neural network training.
- EleutherAI's [GPT-NeoX](#). This is a research library for training large language transformer models at scale based on NVIDIA's Megatron-LM and Microsoft's DeepSpeed.
- PaddlePaddle: integrated into the framework with [API](#) `paddle.nn.functional.flash_attention`.



FlashAttention Key Takeaways

- **Fast**
 - BERT-large (512 seq. Length) is 15% faster than the training speed record in MLPerf 1.1
 - GPT2 (seq. length 1K) is 3x faster than baseline implementations from HuggingFace
 - Megatron-LM (seq. length 1K-4K) is 2.4x faster than the baseline.
- **Memory-efficient**
 - Vanilla attention is quadratic in sequence length $O(N^2)$
 - FlashAttention is sub-quadratic/linear in N ($O(N)$).
- **Exact**
 - Not an approximation of the attention mechanism (like, e.g., sparse or low-rank matrix approximation methods)
 - Its outputs are the same as those of the “vanilla” attention mechanism.
- **IO-aware**
 - It leverages the knowledge of the memory hierarchy of the underlying hardware (e.g., GPUs)

FlashAttention Family vs Others

Variant	Key Innovation	Best Use Case	Difference
FA1	Tiled attention + online softmax	Training long sequences	First to eliminate $O(n^2)$ memory
FA2	Better tiling + kernel split	Fastest dense attention	Higher GPU utilization
FA3	Decode optimization	Inference	Specialized for KV-cache
Triton FA	Flexible kernels	Research	Slower but customizable
xFormers	Modular sparsity patterns	Longformer/BigBird	Multi-kernel framework
FlexAttention	User-defined patterns	PyTorch-native customization	Max flexibility
PagedAttention	KV-cache paging	LLM inference at scale	Complements FA
Ulysses / Ring Attn	Multi-GPU scaling	1M+ context	Distributed algorithms

FlashAttention: Where It Helps in Training vs Inference

FA contributes to both phases — but for different reasons and with different variants

TRAINING

use FA1 / FA2

What FA enables

- Long-context training (32K, 128K, even 1M tokens)
- Larger batch sizes — saved memory becomes batch
- Lower activation memory in forward pass
- Recomputation in backward (extra FLOPs, less HBM)

Measured impact

- Memory: $O(N^2) \rightarrow O(N)$ | 10–20× reduction
- Attention kernel: 2–4× faster
- End-to-end training: 2–3× faster overall

Why it matters here

Training is dominated by long forward + backward passes over many tokens. Memory savings unlock longer context; speed savings shorten the training loop.

INFERENCE

use FA2 + FA3 + Paged

What FA enables

- Fast prefill on long input prompts
- Decode-phase optimization (FA3 split-K)
- KV-cache management via PagedAttention
- Multi-query / grouped-query attention support

Measured impact

- Prefill phase: 2× faster (matches training gain)
- Decode phase: 1.5–2× faster on H100 (FA3)
- Serving throughput: 2–5× more requests/sec

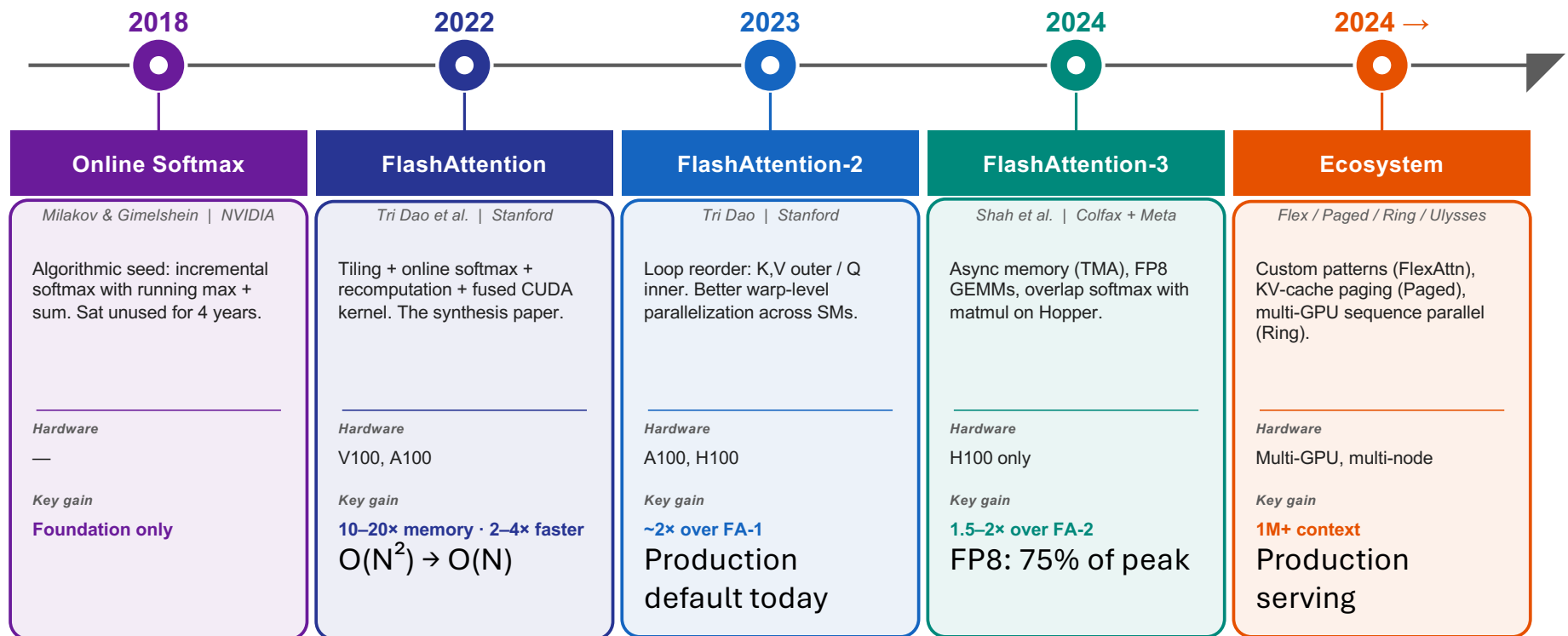
Why it matters here

Inference is two regimes: prefill (compute-bound, like a small training step) and decode (memory-bound, KV-cache dominated). FA3 was redesigned specifically for decode.

Both regimes apply the same IO-aware principle: minimize HBM traffic, do all the work in SRAM

FlashAttention Evolution: Hardware-Driven Innovation

Each version is a response to new GPU capabilities — not new attention math

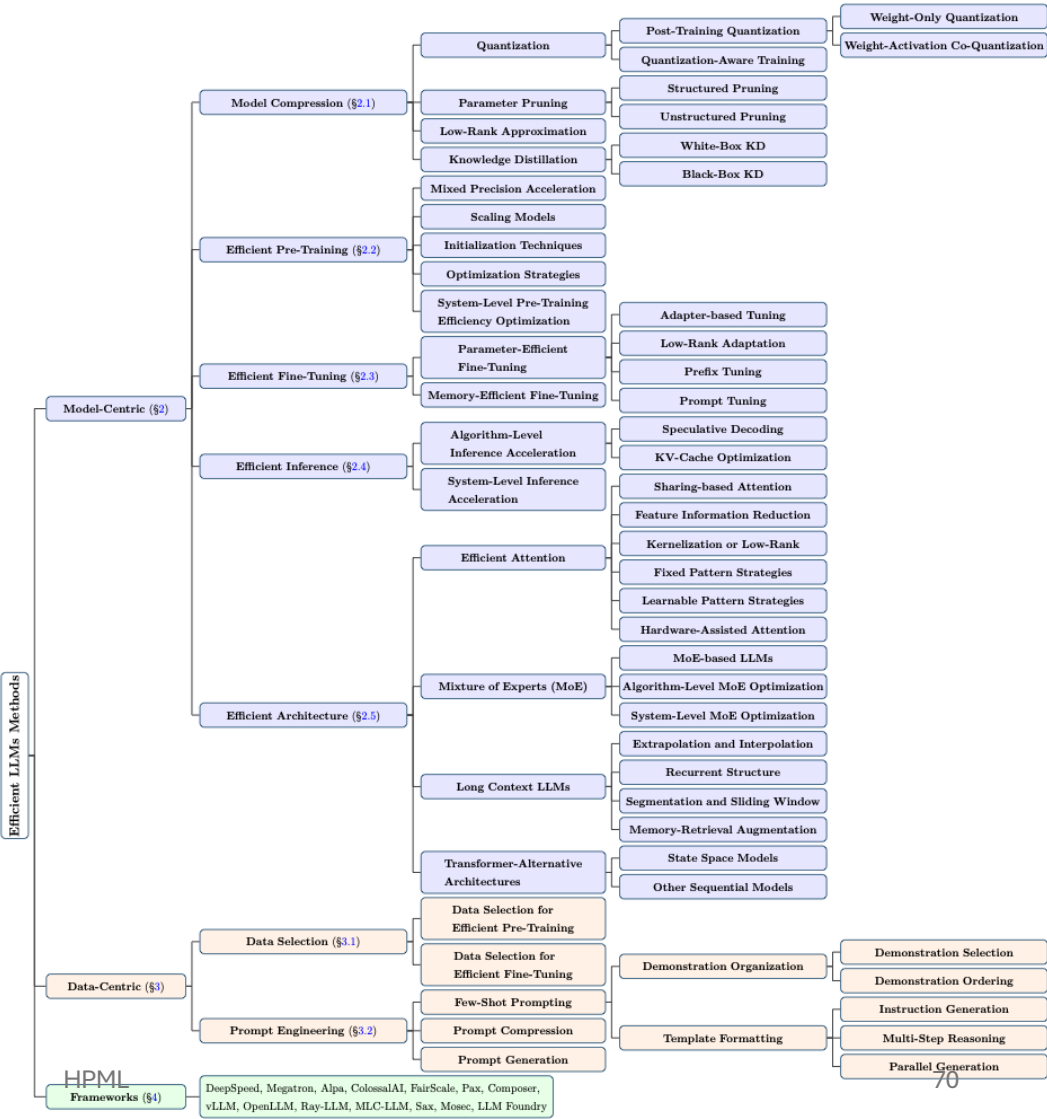


The pattern: each version unlocks the previous generation's idle hardware features (warps → async memory → low precision → multi-chip)

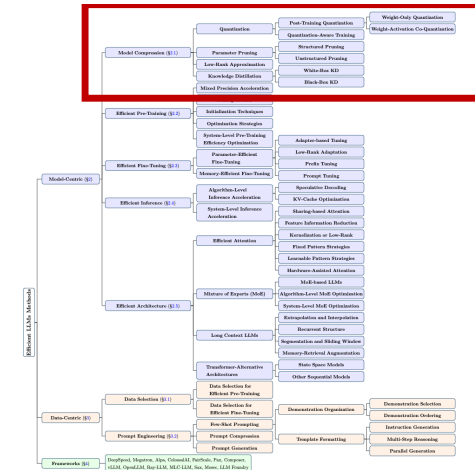
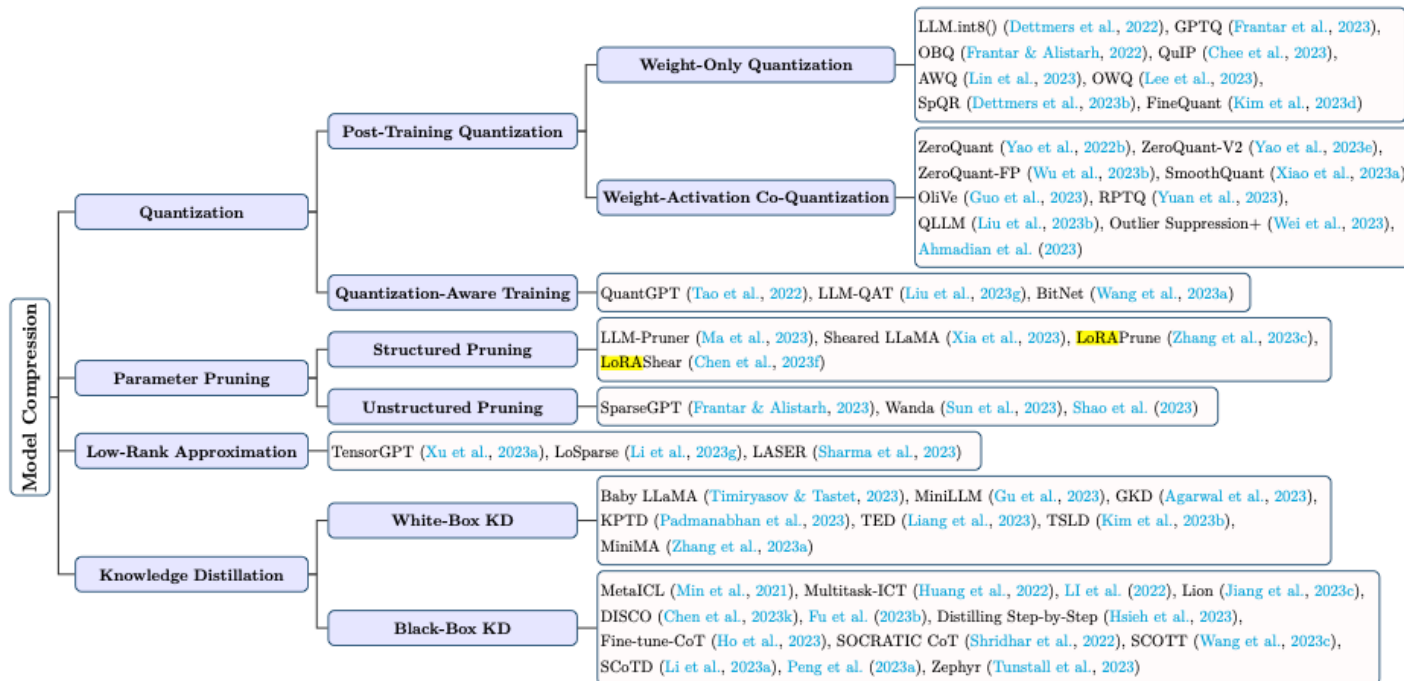
Backup Slides/Reference Materials

Efficient LLM Methods

Efficient Large Language Models: A Survey
<https://arxiv.org/pdf/2312.03863.pdf>



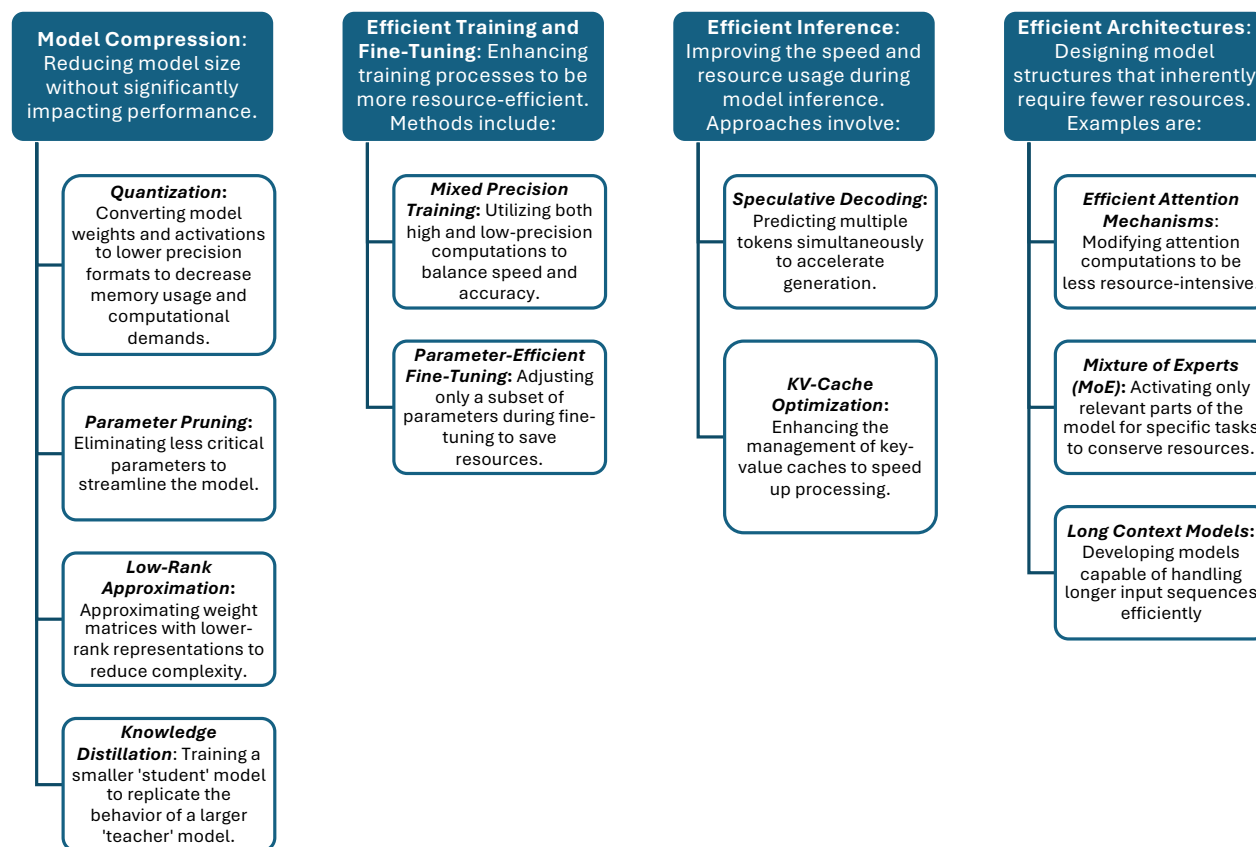
Model Compression Techniques for LLMs



Efficient Large Language Models: A Survey
<https://arxiv.org/pdf/2312.03863.pdf>

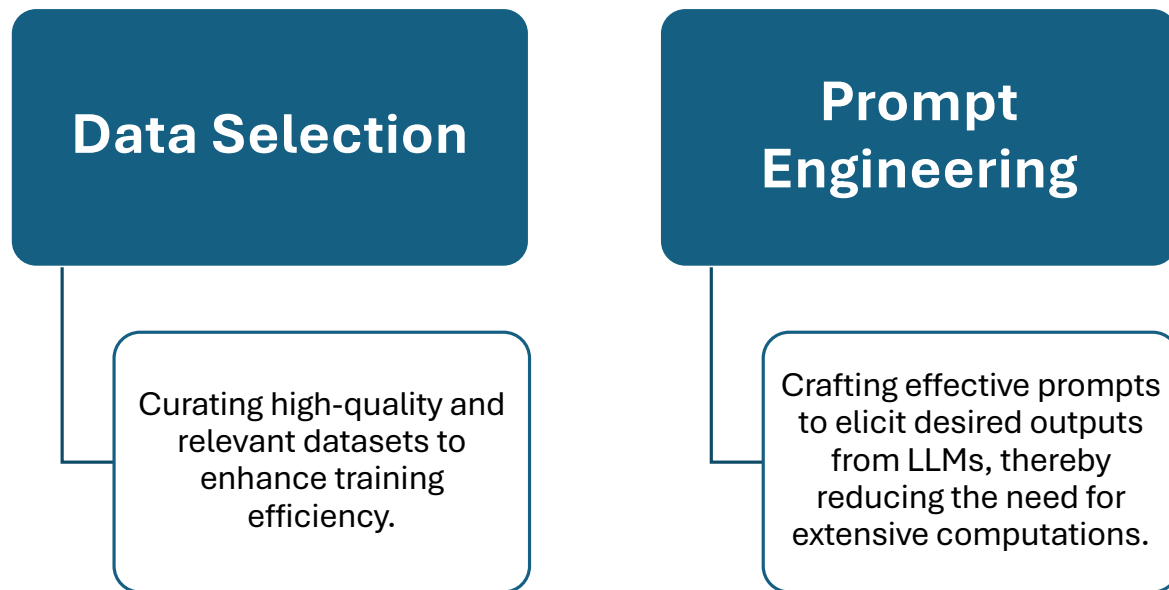
Model Centric Methods

These focus on optimizing the model's architecture and operations to improve efficiency.



Data-Centric Method

These strategies aim to optimize the data used for training and inference to improve efficiency. Key techniques include:



Frameworks

Various software frameworks are designed to support efficient training, fine-tuning, and deployment of LLMs.

DeepSpeed

Developed by Microsoft, DeepSpeed is an integrated framework for training and deploying LLMs. It has been used to train large models like Megatron-Turing NLG 530B and BLOOM.

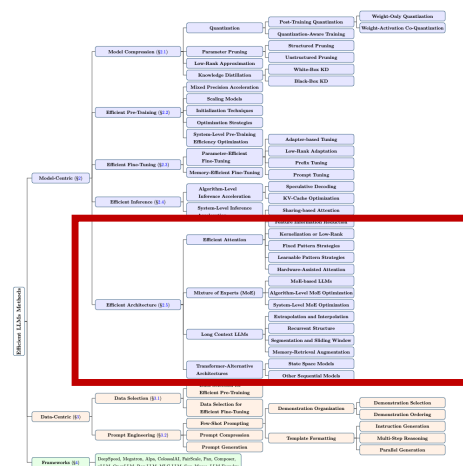
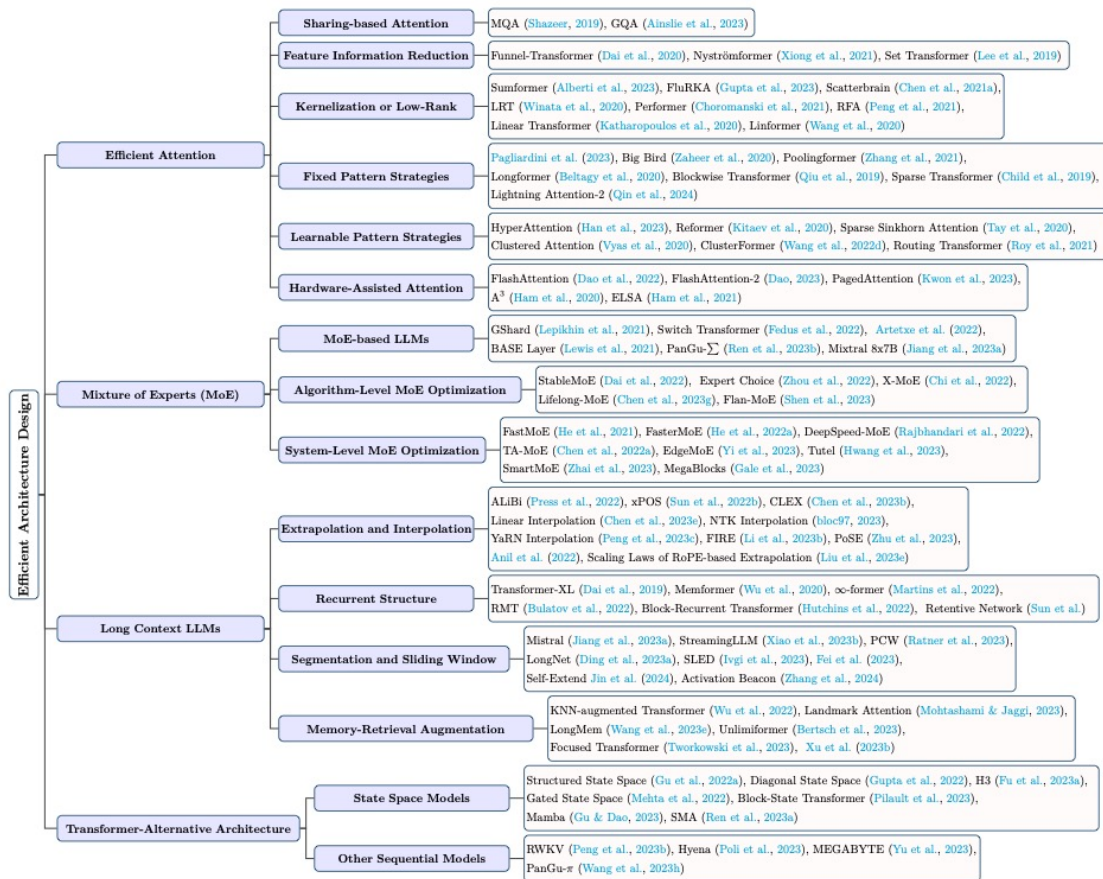
Megatron

A framework by NVIDIA tailored for training large transformer models.

Colossal-AI

An open-source system that provides a suite of tools for efficient-training of large-scale models.

Efficient Architecture Designs for LLMs

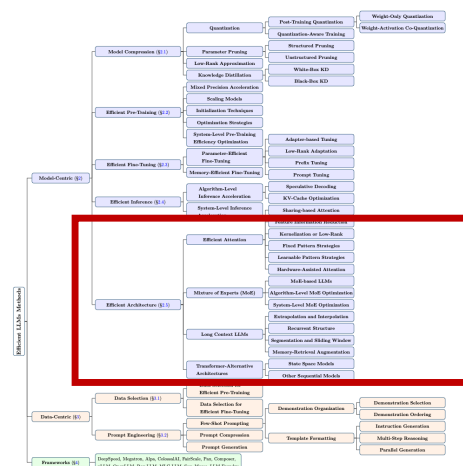
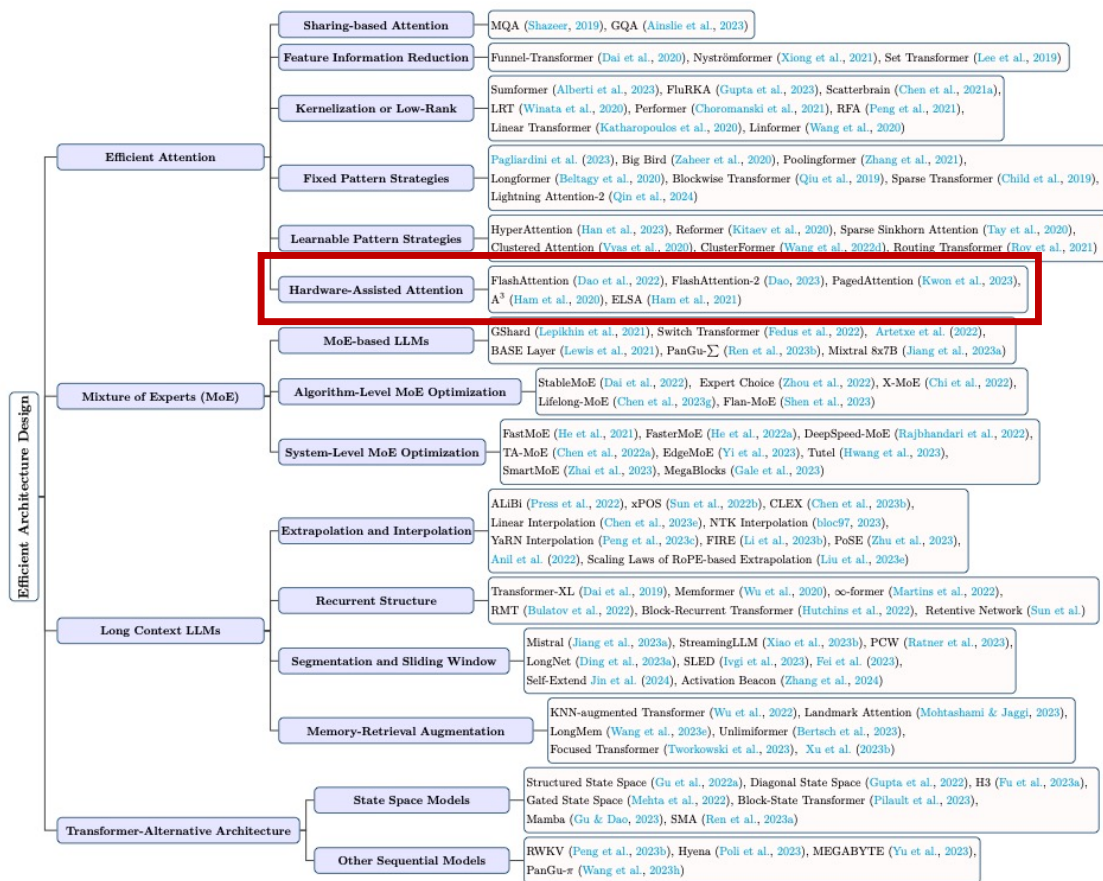


Efficient Large Language Models: A Survey
<https://arxiv.org/pdf/2312.03863.pdf>

HPML

75

Efficient Architecture Designs for LLMs

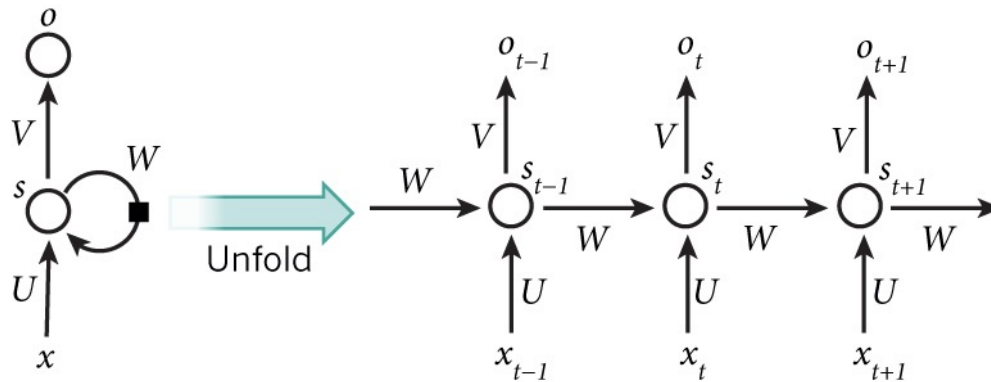


Efficient Large Language Models: A Survey
<https://arxiv.org/pdf/2312.03863.pdf>

Overview of the Transformer Architecture

Reference for the Slides: <https://jalammar.github.io/illustrated-transformer/>

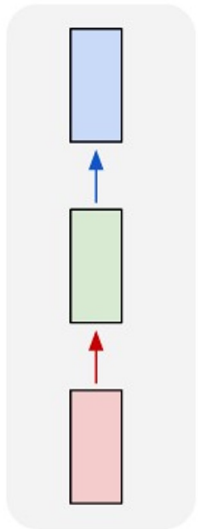
Recurrent Neural Networks: RNN



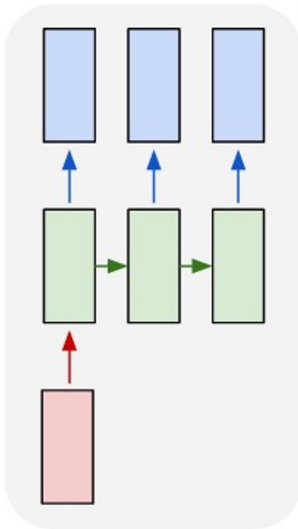
Parameters to be learned:
 U, V, W

Sequence Modeling with RNNs

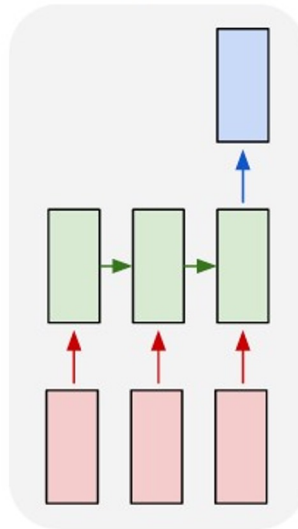
one to one



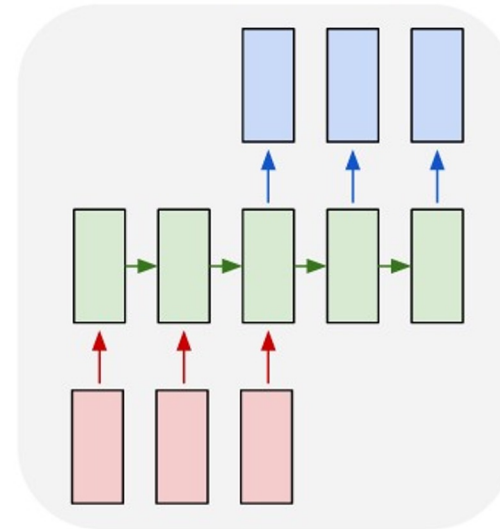
one to many



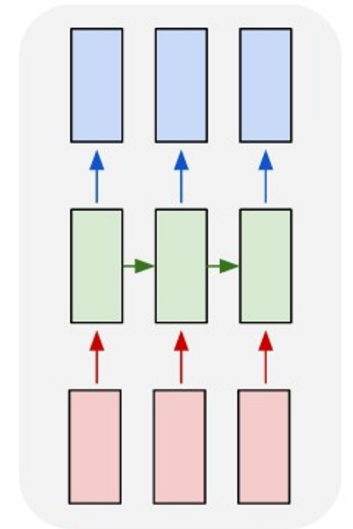
many to one



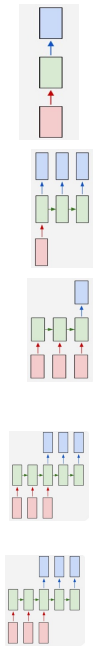
many to many



many to many



Example Applications



Configuration	Example Application	Input	Output
One-to-One	Image Classification	Single image	Single class label
One-to-Many	Music Generation	Starting note	Sequence of notes
Many-to-One	Sentiment Analysis	Sentence (sequence of words)	Sentiment label
Many-to-Many (Same Length)	POS Tagging	Sequence of words	Sequence of POS tags
Many-to-Many (Different Lengths)	Machine Translation	Sentence in one language	Sentence in another language

Machine Translation

we are eating bread



estamos comiendo pan

Machine Translation



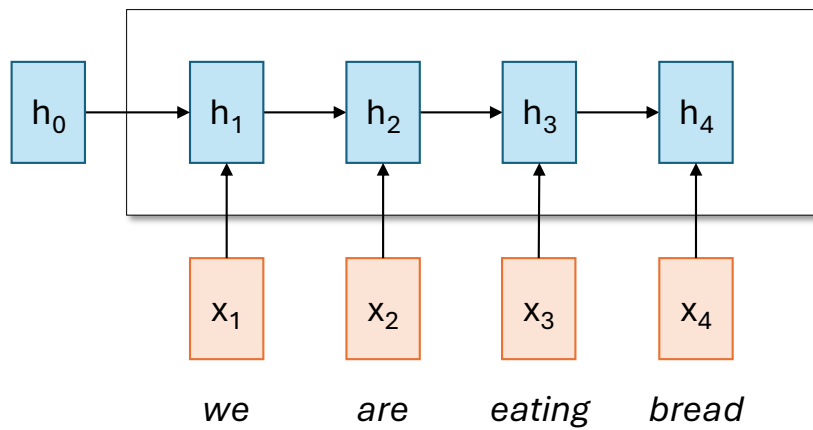
we are eating bread



estamos comiendo pan

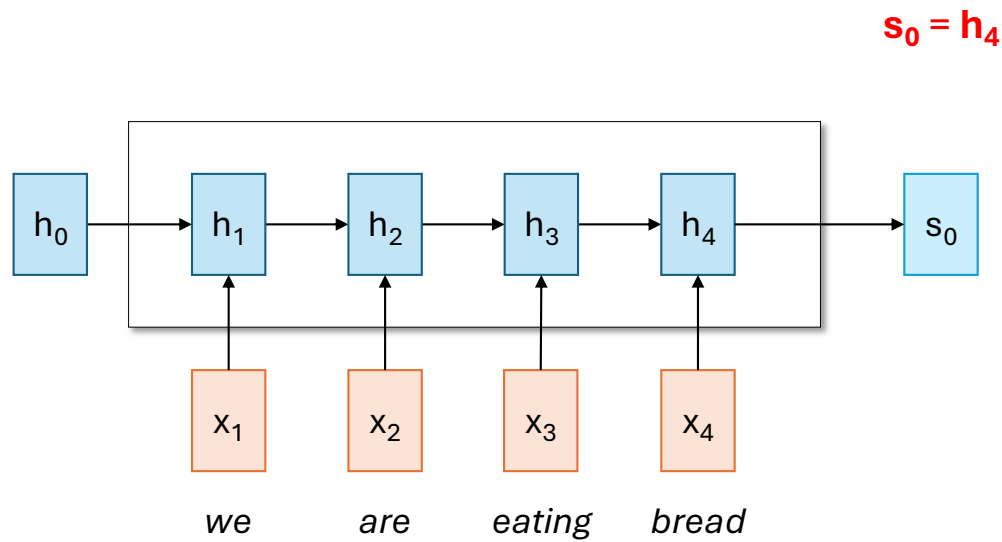
Machine Translation with RNNs

Encoder: $h_t = f_W(x_t, h_{t-1})$



Machine Translation with RNNs

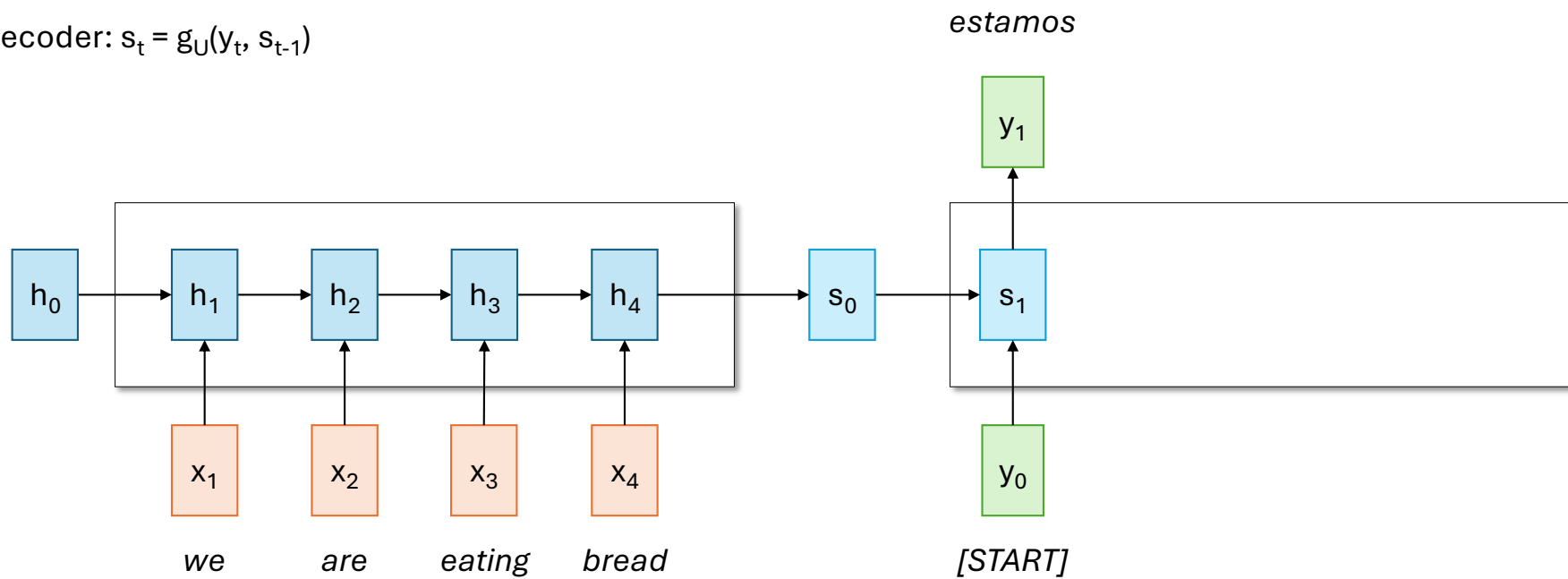
Encoder: $h_t = f_W(x_t, h_{t-1})$



Machine Translation with RNNs

Encoder: $h_t = f_W(x_t, h_{t-1})$

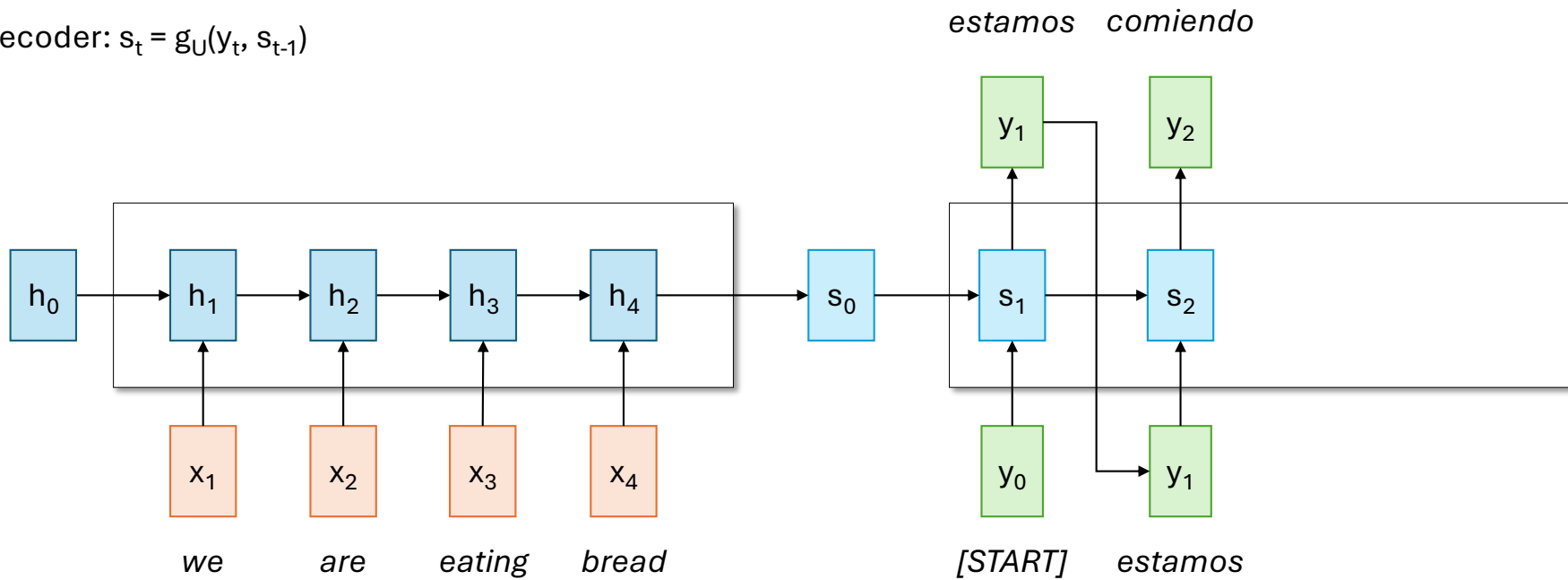
Decoder: $s_t = g_U(y_t, s_{t-1})$



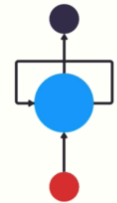
Machine Translation with RNNs

Encoder: $h_t = f_W(x_t, h_{t-1})$

Decoder: $s_t = g_U(y_t, s_{t-1})$

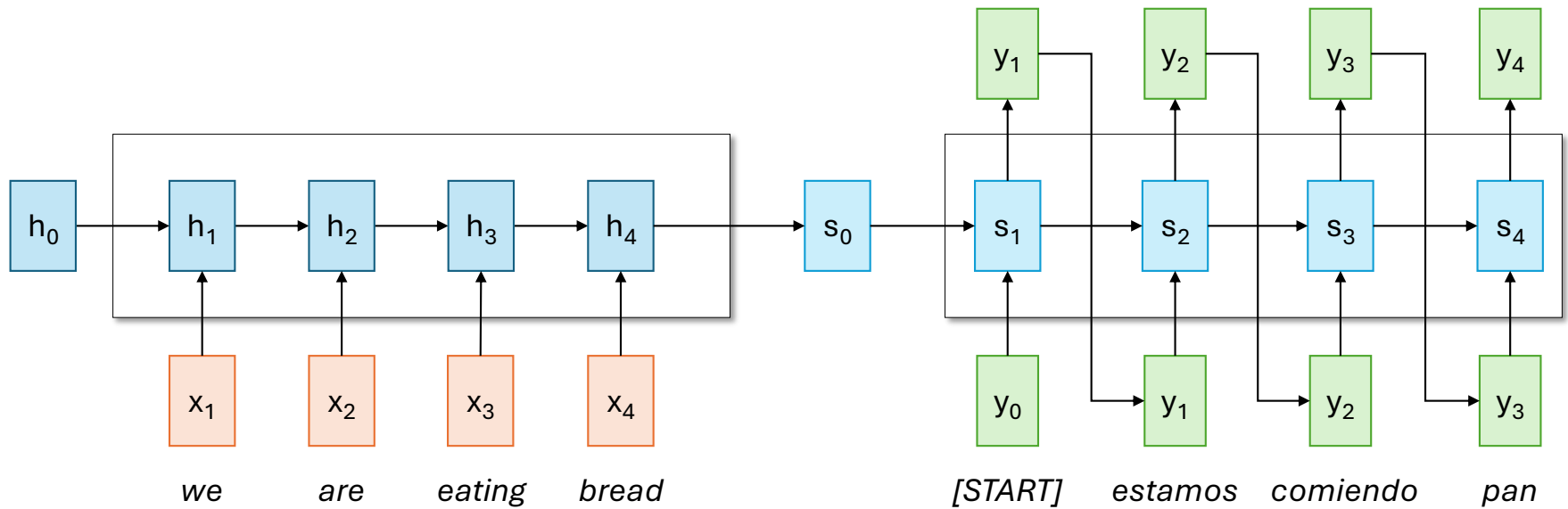


Machine Translation with RNNs

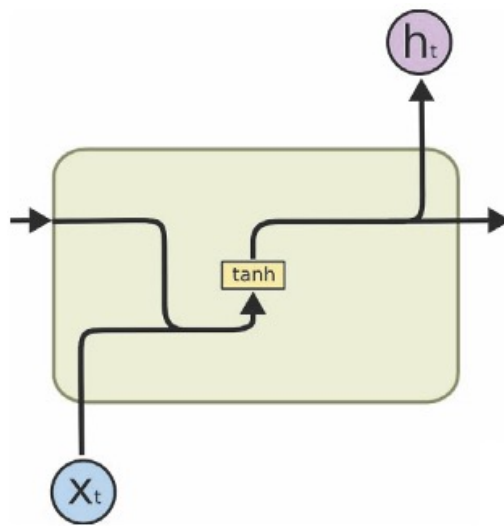


Encoder: $h_t = f_W(x_t, h_{t-1})$

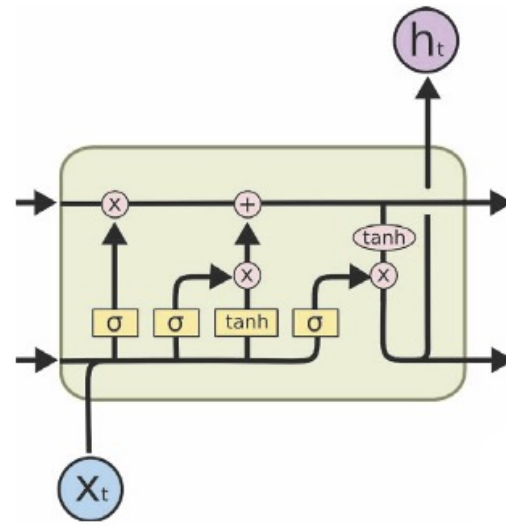
Decoder: $s_t = g_U(y_t, s_{t-1})$



Simple RNN vs LSTM



(a) RNN

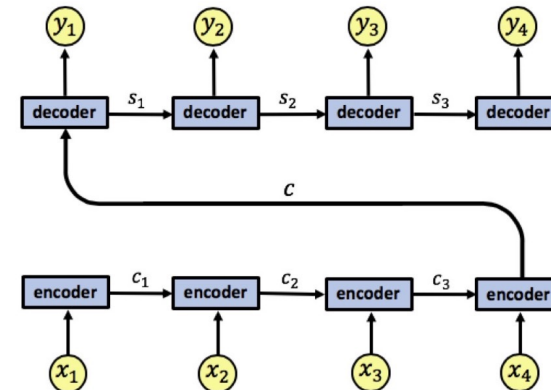


(b) LSTM

Long **Short-Term Memory** (LSTM) helps for the “long-term dependency” requirement (or solves short-term memory issue) as the default behavior is to remember long term information.

More RNN/LSTM Limitations

- The final state of RNN (better to say, encoder) must hold information for the entire input sequence.
- **A major drawback with this architecture:**
- The encoding step needs to represent the entire input sequence x_1, x_2, x_3, x_4 as a single vector c
- This *can cause information loss* as all information needs to be *compressed into c* .
- The decoder needs to decipher the passed information from this single vector only, *a highly complex task in itself indeed*.

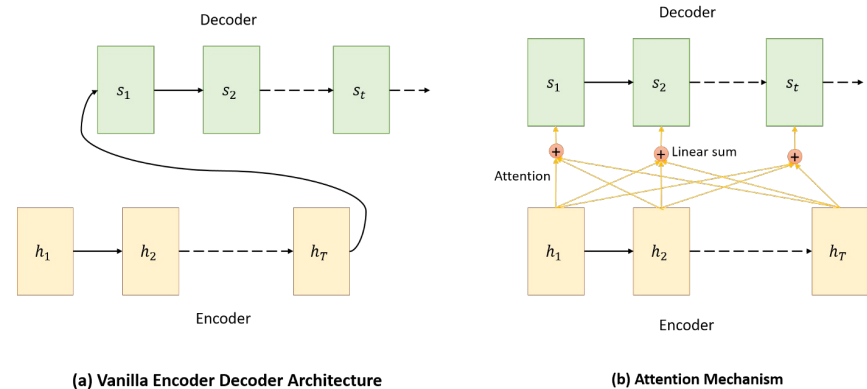


Simple representation of traditional Encoder-Decoder architecture using RNN/LSTM

<https://medium.com/swlh/a-simple-overview-of-rnn-lstm-and-attention-mechanism-9e844763d07b>

Attention Mechanism to the Rescue

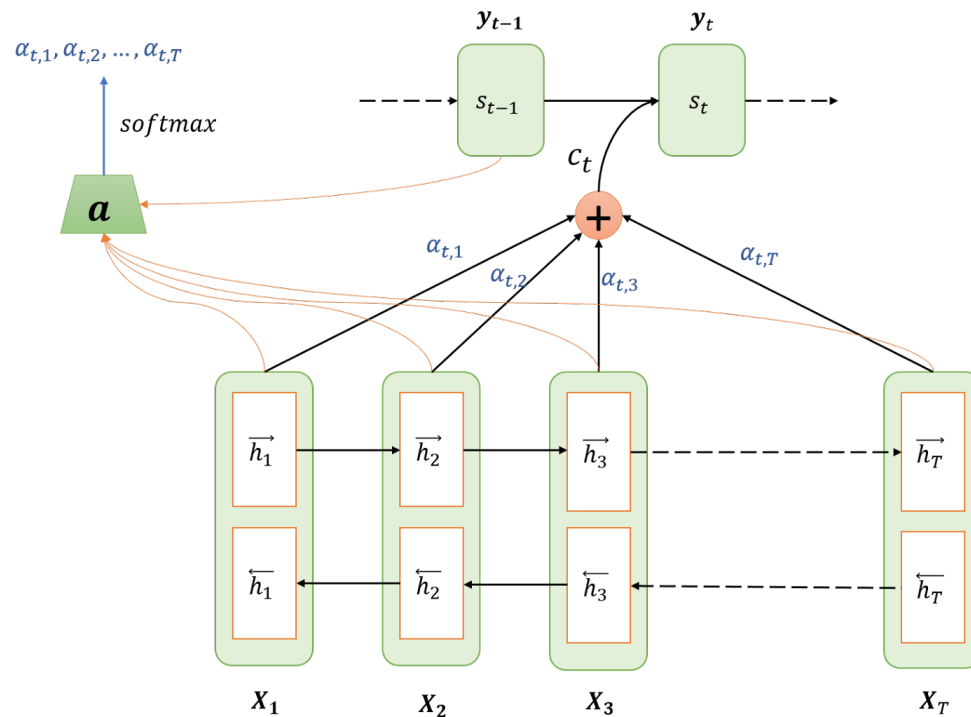
- *Attention Mechanism for sequence modelling was first used in the paper: [Neural Machine Translation by jointly learning to align and translate, Bengio et. al. ICLR 2015](#). Although the notion of 'attention' wasn't as famous back then and the word has been sparingly used within the paper itself.*
- **The attention mechanism helps to look at all hidden states from the encoder sequence for making predictions, unlike the vanilla Encoder-Decoder approach.**



Simple representation of traditional Encoder-Decoder architecture using RNN/LSTM

How to learn which hidden states to attend to

Solution: Use a Neural Network to “learn” which hidden encoder states to “attend” to and by how much.

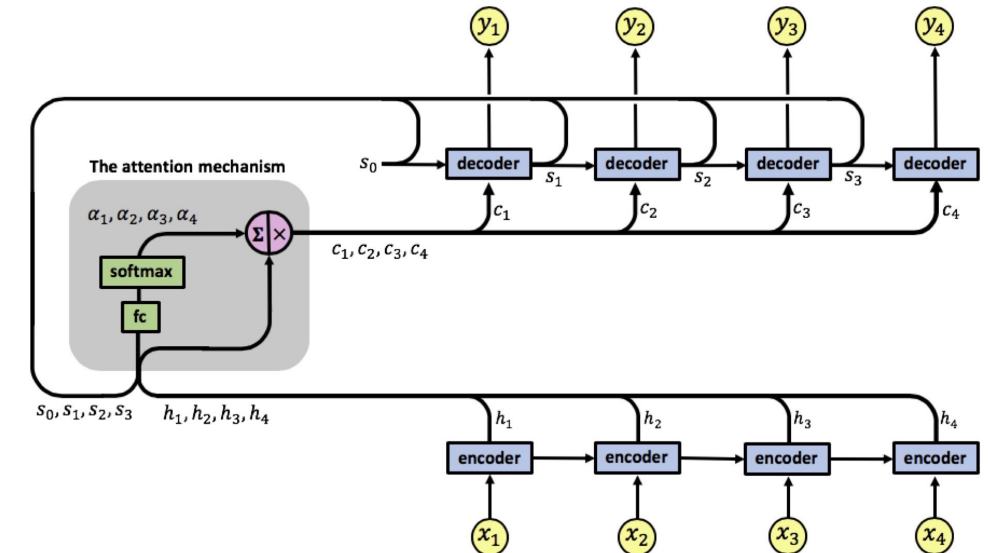
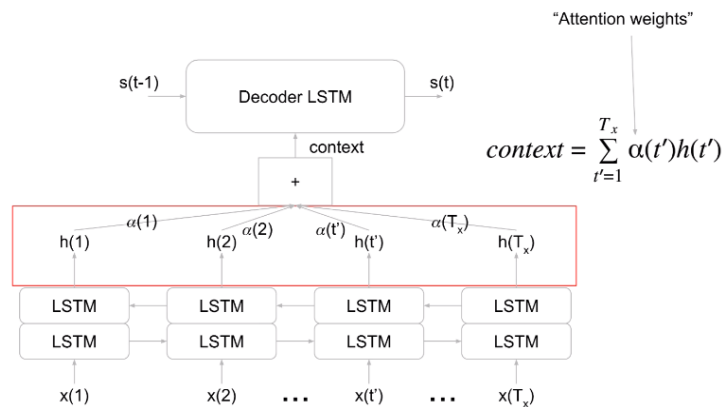


<https://medium.com/swlh/a-simple-overview-of-rnn-lstm-and-attention-mechanism-9e844763d07b>

Calculating the Attention Weights

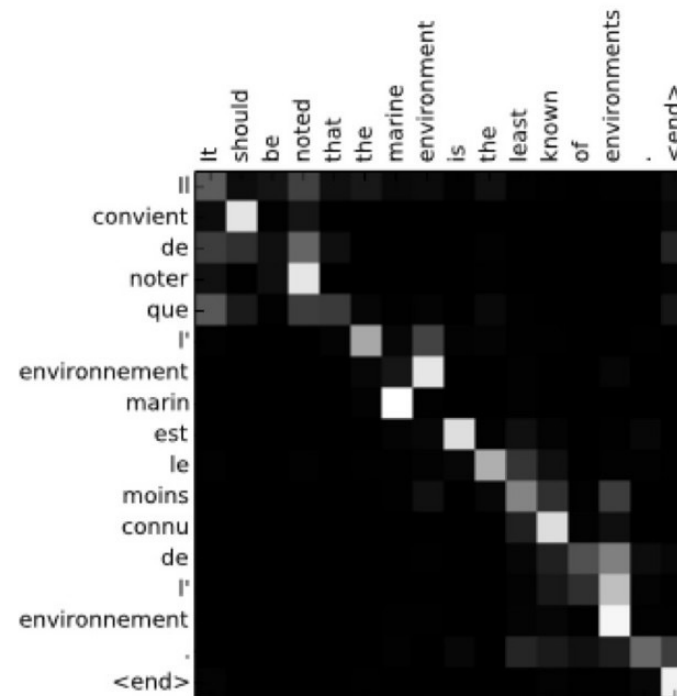
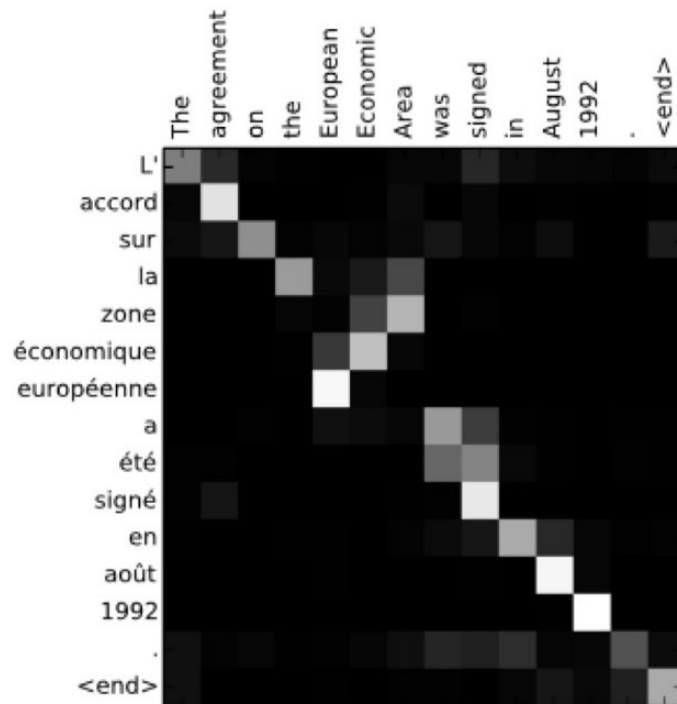
$$\alpha_{t'} = \text{NeuralNet}([s_{t-1}, h_{t'}]), \quad t' = 1 \dots T_x$$

$$\text{context} = \sum_{t'=1}^{T_x} \alpha(t') h(t')$$



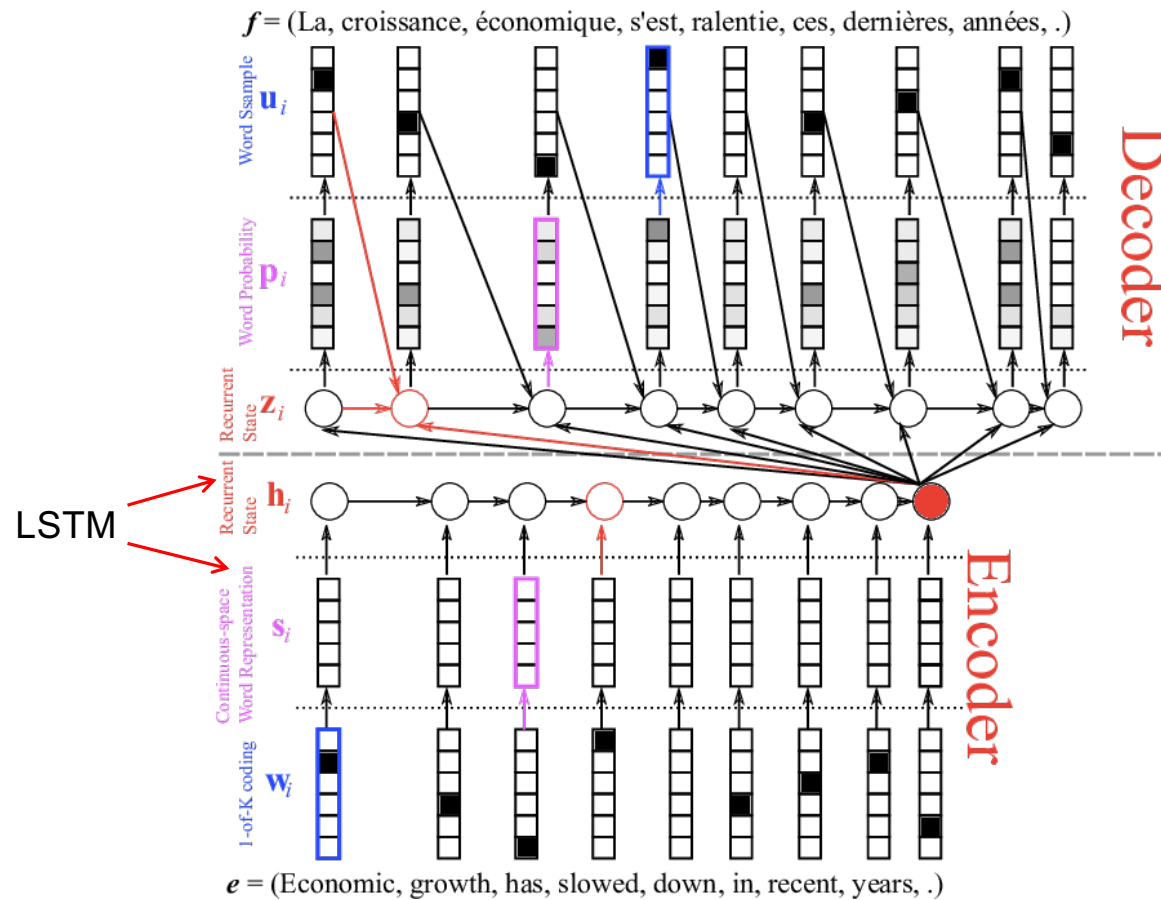
<https://medium.com/swlh/a-simple-overview-of-rnn-lstm-and-attention-mechanism-9e844763d07b>

Visualizing Attention

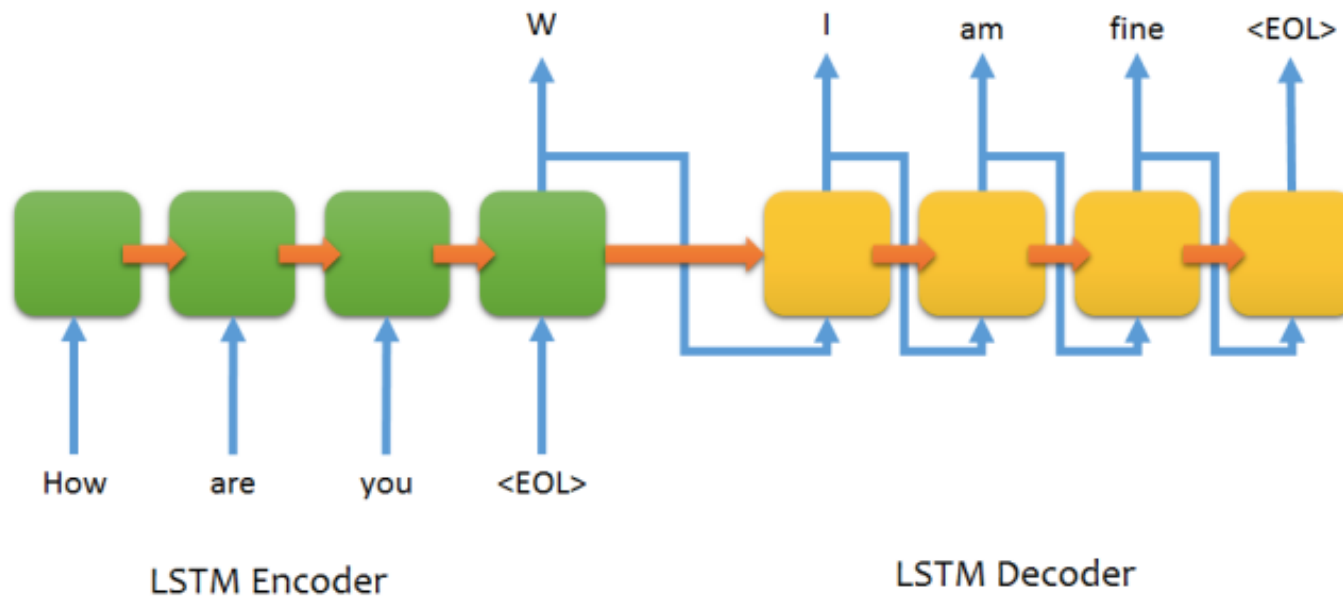


<https://medium.com/swlh/a-simple-overview-of-rnn-lstm-and-attention-mechanism-9e844763d07b>

Encoder-Decoder machine translation

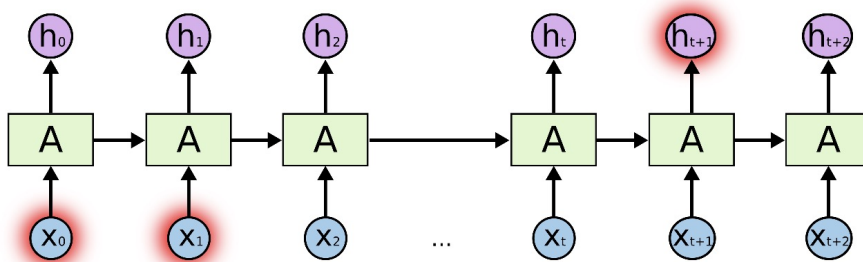


Encoder-Decoder LSTM structure for chatting

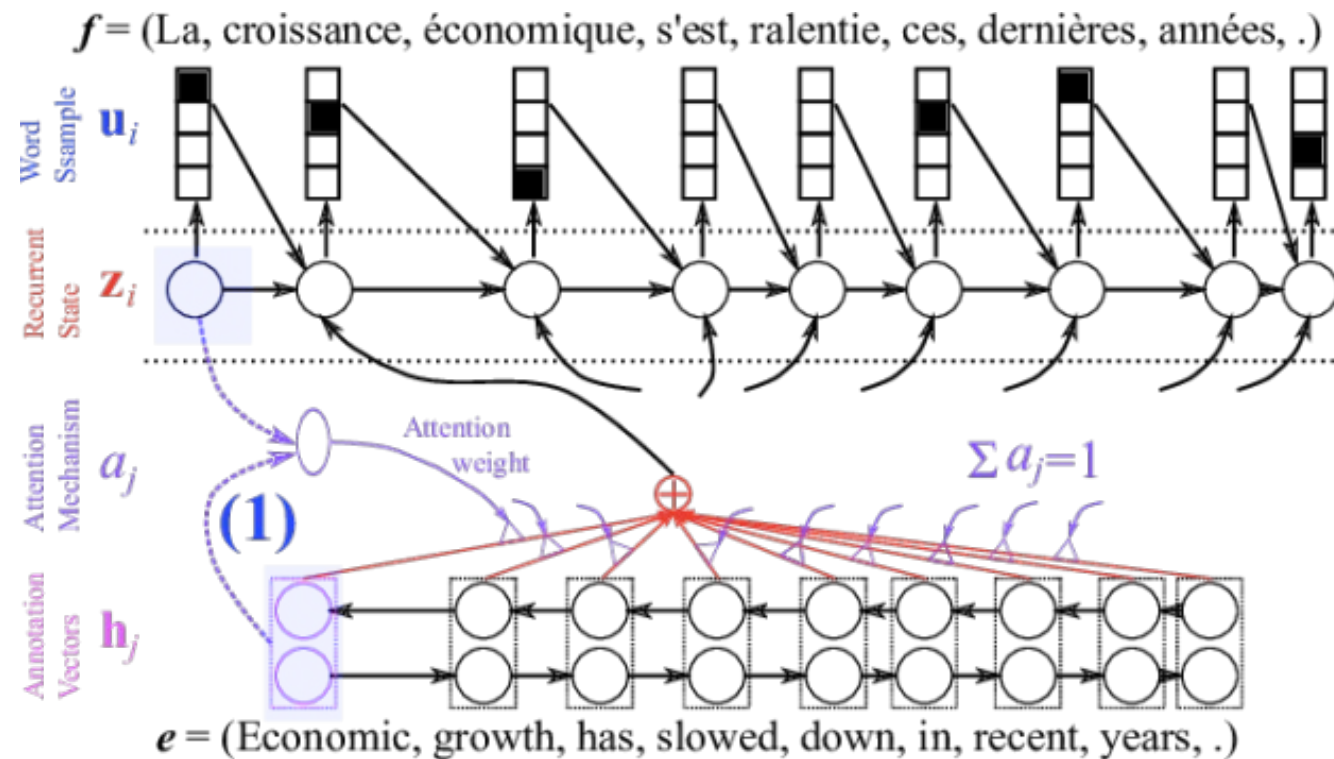


Challenges with RNN

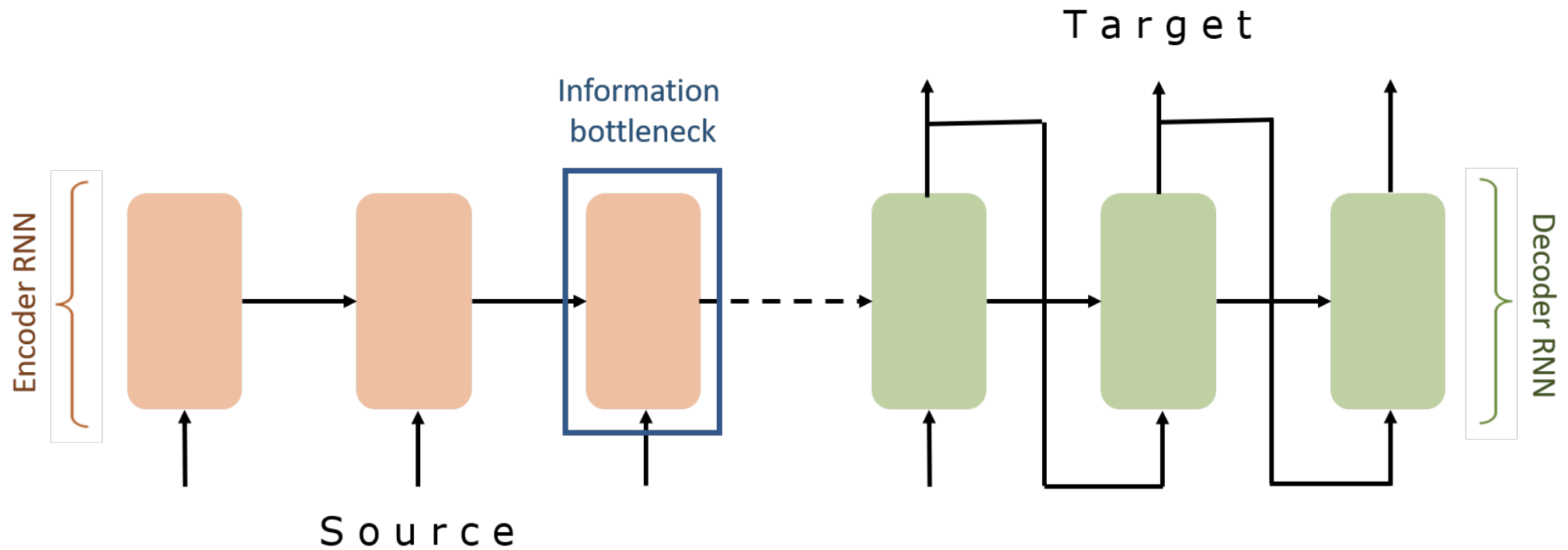
- Vanishing gradient problem in NNs during backpropagation across layers
- This causes RNN to **NOT** learn the long-range dependencies well across time steps.



RNN with Attention

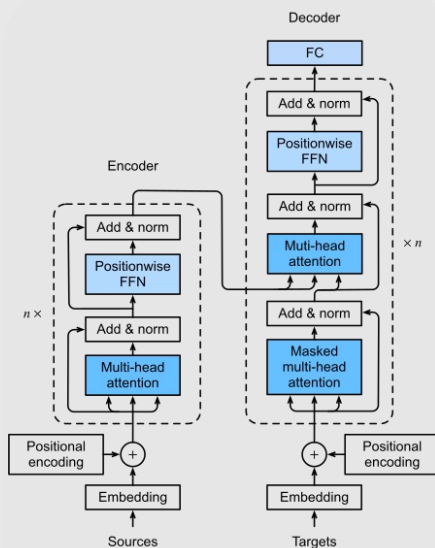


Avoiding Information bottleneck

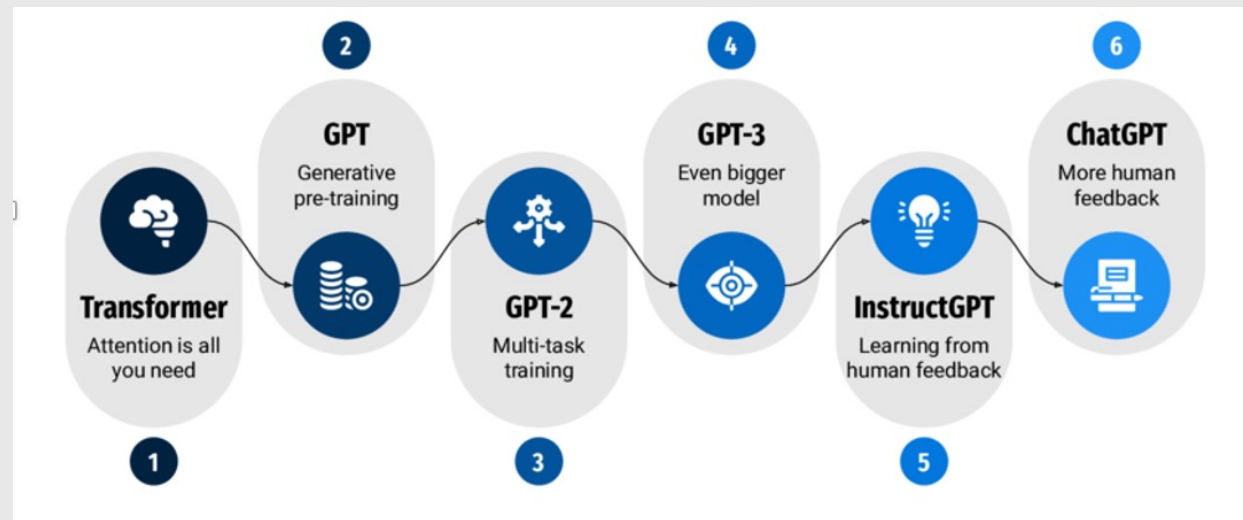


Why Attention was all you need?
What was not needed?

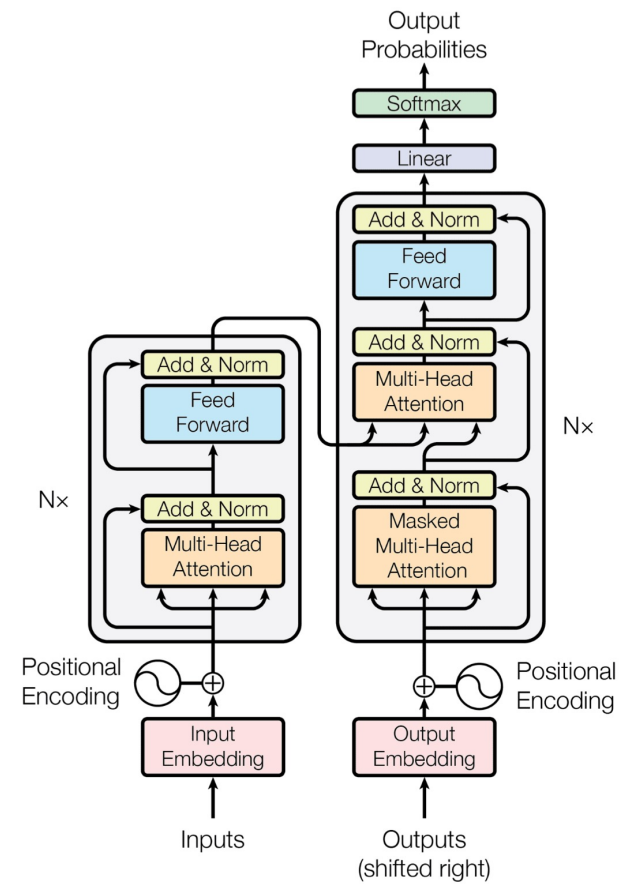
Evolution from Transformer Architecture to ChatGPT



Transformer neural network architecture introduced by Google in 2017

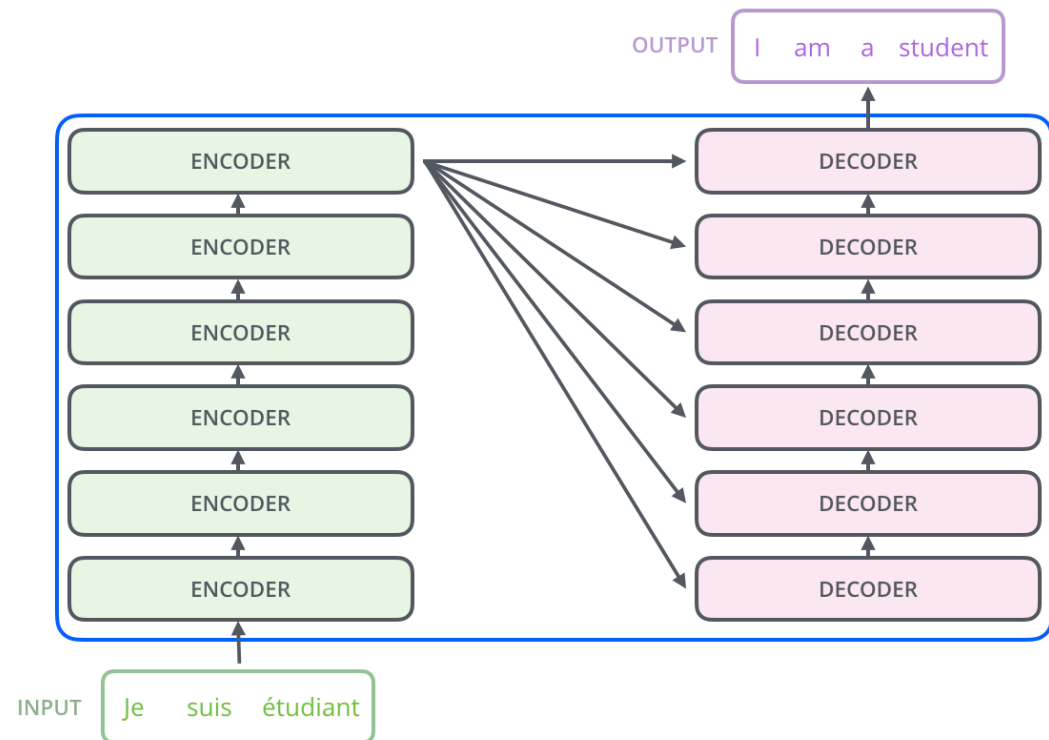


Attention Architecture



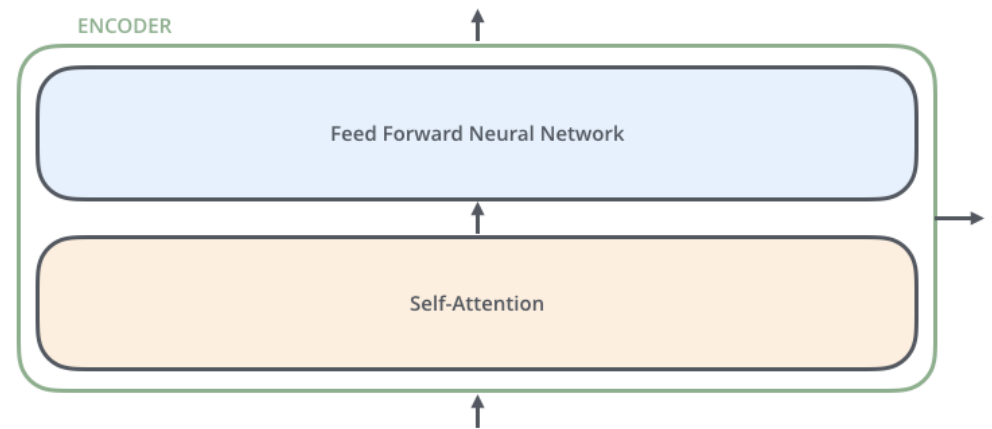
Transformer

- The encoding component is a stack of encoders (the paper stacks six of them on top of each other).
- We can experiment with different arrangements.
- The decoding component is a stack of decoders of the same number.



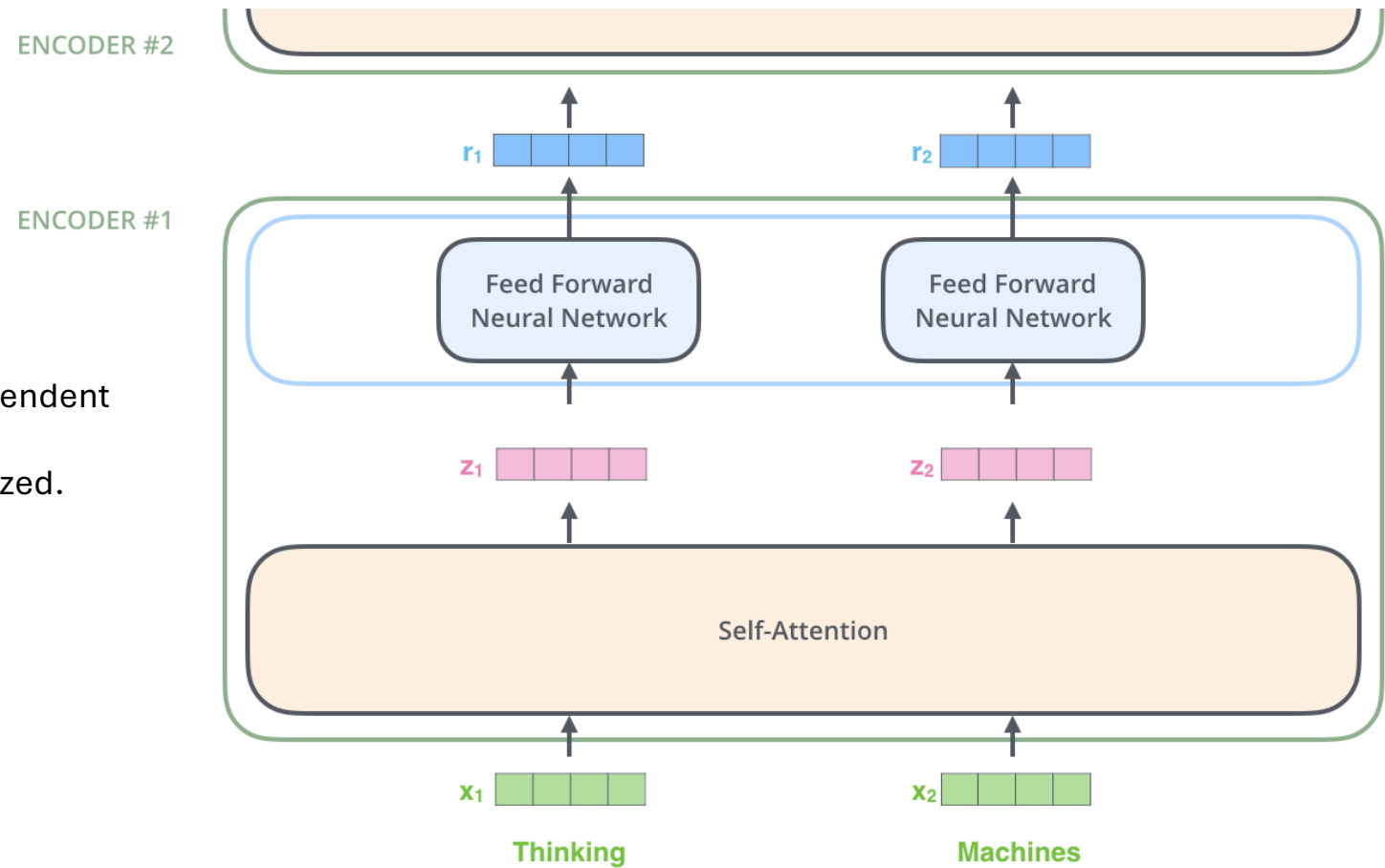
An Encoder Block: same structure, different parameters

- The encoder's inputs first flow through a self-attention layer.
- The outputs of the self-attention layer are fed to a feed-forward neural network.
- The exact same feed-forward network is independently applied to each position.



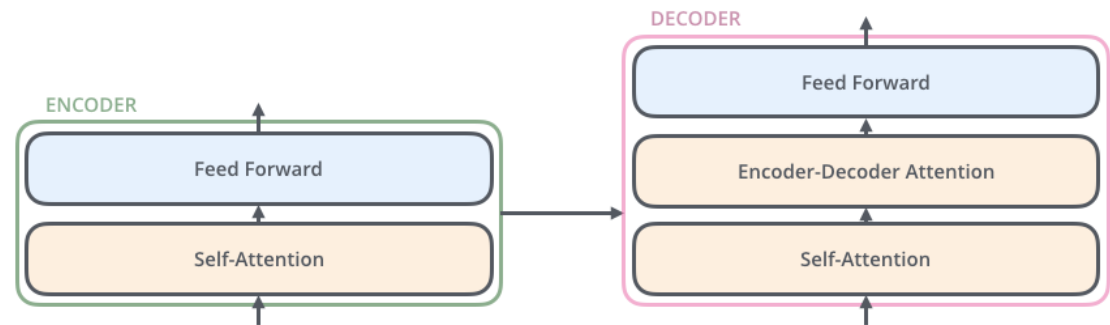
Encoder

Note: The ffnn is independent for each word.
Hence can be parallelized.



An Encoder Block: same structure, different parameters

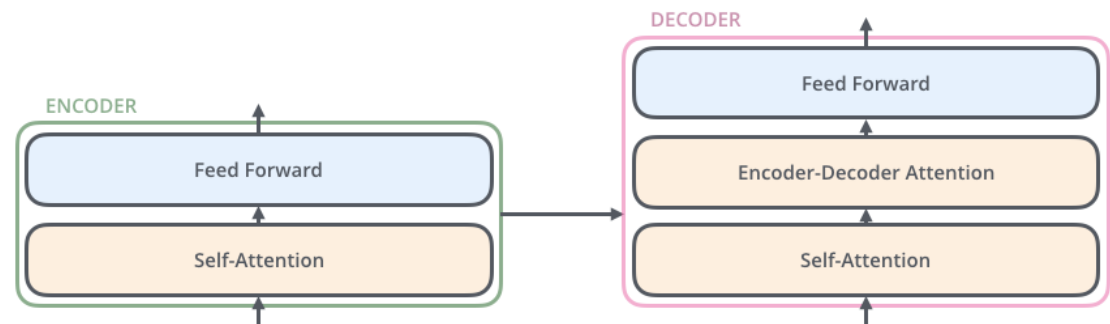
- The decoder has both those layers, but between them is an attention layer that helps the decoder focus on relevant parts of the input sentence
 - similar to what attention does in [seq2seq models](#).



Feature	Encoder	Decoder
Input	Full source sentence	Previously generated tokens
Self-Attention	Bi-directional	Masked (causal, to prevent peeking ahead)
Extra Attention	✗	✓ Encoder-Decoder Attention
Purpose	Encode input into latent representation	Decode output using context from encoder and prior outputs

An Encoder Block: same structure, different parameters

- The decoder has both those layers, but between them is an attention layer that helps the decoder focus on relevant parts of the input sentence
 - similar to what attention does in [seq2seq models](#).



Feature	Encoder	Decoder
Input	Full source sentence	Previously generated tokens
Self-Attention	Bi-directional	Masked (causal, to prevent peeking ahead)
Extra Attention	✗	✓ Encoder-Decoder Attention
Purpose	Encode input into latent representation	Decode output using context from encoder and prior outputs

What does Q, K, and V Mean?

1. Query (Q):

- **What it Represents:**
 - The Query is a vector that represents the current token (or input element) that is trying to determine how much attention it should pay to other tokens in the sequence.
 - It acts as a "question" about what the model should focus on.
- **Role in Computation:**
 - It is compared with all the Keys in the sequence to calculate attention scores, which represent the relevance of other tokens to the current token.
- **Dimension:**
 - $Q \in \mathbb{R}^{d_k}$, where d_k is the dimensionality of the Query vector.

2. Key (K):

- **What it Represents:**
 - The Key is a vector that encodes information about a specific token in the sequence. It acts as an "index" or "identifier" that other tokens can refer to when computing attention.
- **Role in Computation:**
 - Queries are compared against Keys (via dot product) to calculate similarity scores, which determine how much focus a Query should place on a particular Key.
- **Dimension:**
 - $K \in \mathbb{R}^{d_k}$, where d_k is the dimensionality of the Key vector.

3. Value (V):

- **What it Represents:**
 - The Value is a vector containing the actual information or content associated with a token. It is what the model ultimately "pays attention to" and aggregates to produce the output.
- **Role in Computation:**
 - After computing attention scores between Queries and Keys, the scores are used to weight the Values. The weighted sum of Values is the output of the attention mechanism.
- **Dimension:**
 - $V \in \mathbb{R}^{d_v}$, where d_v is the dimensionality of the Value vector.

4. d_k and d_v :

- **What They Represent:**
 - d_k : The dimensionality of the Query and Key vectors. This defines the size of the space in which similarity between Queries and Keys is computed.
 - d_v : The dimensionality of the Value vector. This determines the size of the output produced by the attention mechanism.
 - d : Often used to refer to the model's hidden size, with $d = d_k = d_v$ in some cases (e.g., in Transformer implementations).
- **Why Scaling Matters:**
 - In the attention computation, the dot product of Query (Q) and Key (K) can result in large values if d_k is large. To stabilize the softmax computation, the dot product is scaled by dividing by $\sqrt{d_k}$:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- This ensures that the attention scores remain balanced and do not lead to extremely small or large gradients during training.

Analogy with a Database: Query, Key & Value

Database

{Key, Value store}

{Query: "Order details of order_104"}

OR

{Query: "Order details of order_106"}

```
{
  "order_100": {
    "items": "a1",
    "delivery_date": "a2",
    ...
  },
  "order_101": {
    "items": "b1",
    "delivery_date": "b2",
    ...
  },
  "order_102": {
    "items": "c1",
    "delivery_date": "c2",
    ...
  },
  "order_103": {
    "items": "d1",
    "delivery_date": "d2",
    ...
  },
  "order_104": {
    "items": "e1",
    "delivery_date": "e2",
    ...
  },
  "order_105": {
    "items": "f1",
    "delivery_date": "f2",
    ...
  },
  "order_106": {
    "items": "g1",
    "delivery_date": "g2",
    ...
  },
  "order_107": {
    "items": "h1",
    "delivery_date": "h2",
    ...
  },
  "order_108": {
    "items": "i1",
    "delivery_date": "i2",
    ...
  },
  "order_109": {
    "items": "j1",
    "delivery_date": "j2",
    ...
  },
  "order_110": {
    "items": "k1",
    "delivery_date": "k2",
    ...
  }
}
```


Analogy with a Database: Query, Key & Value

{Query: "Order details of order_104"}

OR

{Query: "Order details of order_106"}

{Key, Value store}

```
{ "order_100": { "items": "a1", "delivery_date": "a2", ... },  
  "order_101": { "items": "b1", "delivery_date": "b2", ... },  
  "order_102": { "items": "c1", "delivery_date": "c2", ... },  
  "order_103": { "items": "d1", "delivery_date": "d2", ... },  
  "order_104": { "items": "e1", "delivery_date": "e2", ... },  
  "order_105": { "items": "f1", "delivery_date": "f2", ... },  
  "order_106": { "items": "g1", "delivery_date": "g2", ... },  
  "order_107": { "items": "h1", "delivery_date": "h2", ... },  
  "order_108": { "items": "i1", "delivery_date": "i2", ... },  
  "order_109": { "items": "j1", "delivery_date": "j2", ... },  
  "order_110": { "items": "k1", "delivery_date": "k2", ... }
```

Analogy with a Database: Query, Key & Value

{Query: "Order details of order_104"}

OR

{Query: "Order details of order_106"}

{Key, Value store}

```
{ "order_100": { "items": "a1", "delivery_date": "a2", ... },  
  "order_101": { "items": "b1", "delivery_date": "b2", ... },  
  "order_102": { "items": "c1", "delivery_date": "c2", ... },  
  "order_103": { "items": "d1", "delivery_date": "d2", ... },  
  "order_104": { "items": "e1", "delivery_date": "e2", ... },  
  "order_105": { "items": "f1", "delivery_date": "f2", ... },  
  "order_106": { "items": "g1", "delivery_date": "g2", ... },  
  "order_107": { "items": "h1", "delivery_date": "h2", ... },  
  "order_108": { "items": "i1", "delivery_date": "i2", ... },  
  "order_109": { "items": "j1", "delivery_date": "j2", ... },  
  "order_110": { "items": "k1", "delivery_date": "k2", ... }
```

Analogy with a Database: Query, Key & Value

{Query: "Order details of order_104"}

OR

{Query: "Order details of order_106"}

{Key, Value store}

```
{
  "order_100": {
    "items": "a1",
    "delivery_date": "a2",
    ...
  },
  "order_101": {
    "items": "b1",
    "delivery_date": "b2",
    ...
  },
  "order_102": {
    "items": "c1",
    "delivery_date": "c2",
    ...
  },
  "order_103": {
    "items": "d1",
    "delivery_date": "d2",
    ...
  },
  "order_104": {
    "items": "e1",
    "delivery_date": "e2",
    ...
  },
  "order_105": {
    "items": "f1",
    "delivery_date": "f2",
    ...
  },
  "order_106": {
    "items": "g1",
    "delivery_date": "g2",
    ...
  },
  "order_107": {
    "items": "h1",
    "delivery_date": "h2",
    ...
  },
  "order_108": {
    "items": "i1",
    "delivery_date": "i2",
    ...
  },
  "order_109": {
    "items": "j1",
    "delivery_date": "j2",
    ...
  },
  "order_110": {
    "items": "k1",
    "delivery_date": "k2",
    ...
  }
}
```

Analogy with a Database: Query, Key & Value

Done at the same time !!

{Query: "Order details of order_104"}

OR

{Query: "Order details of order_106"}

{Key, Value store}

```
{
  "order_100": {
    "items": "a1",
    "delivery_date": "a2",
    ...
  },
  "order_101": {
    "items": "b1",
    "delivery_date": "b2",
    ...
  },
  "order_102": {
    "items": "c1",
    "delivery_date": "c2",
    ...
  },
  "order_103": {
    "items": "d1",
    "delivery_date": "d2",
    ...
  },
  "order_104": {
    "items": "e1",
    "delivery_date": "e2",
    ...
  },
  "order_105": {
    "items": "f1",
    "delivery_date": "f2",
    ...
  },
  "order_106": {
    "items": "g1",
    "delivery_date": "g2",
    ...
  },
  "order_107": {
    "items": "h1",
    "delivery_date": "h2",
    ...
  },
  "order_108": {
    "items": "i1",
    "delivery_date": "i2",
    ...
  },
  "order_109": {
    "items": "j1",
    "delivery_date": "j2",
    ...
  },
  "order_110": {
    "items": "k1",
    "delivery_date": "k2",
    ...
  }
}
```

Analogy with a Database: Query, Key & Value

{Query: "Order details of order_104"}

OR

{Query: "Order details of order_106"}

```
{ "order_100": { "items": "a1", "delivery_date": "a2", ... },  
  "order_101": { "items": "b1", "delivery_date": "b2", ... },  
  "order_102": { "items": "c1", "delivery_date": "c2", ... },  
  "order_103": { "items": "d1", "delivery_date": "d2", ... },  
  "order_104": { "items": "e1", "delivery_date": "e2", ... },  
  "order_105": { "items": "f1", "delivery_date": "f2", ... },  
  "order_106": { "items": "g1", "delivery_date": "g2", ... },  
  "order_107": { "items": "h1", "delivery_date": "h2", ... },  
  "order_108": { "items": "i1", "delivery_date": "i2", ... },  
  "order_109": { "items": "j1", "delivery_date": "j2", ... },  
  "order_110": { "items": "k1", "delivery_date": "k2", ... }
```

Query

1. Search for info

Key

1. Interacts directly with Queries
2. Distinguishes one object from another
3. Identify which object is the most relevant and by how much

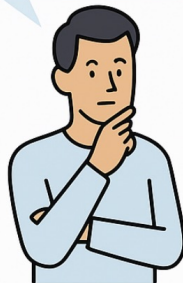
Value

1. Actual details of the object
2. More fine grained

Illustration

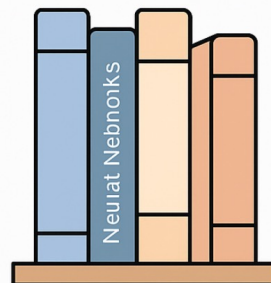
Query (Q)

*Tell me about
neural networks*



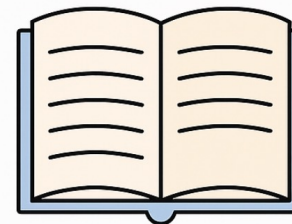
You:
What you're
looking for

Key (K)



Book Title
Tells what
the book is about

Value (V)



Book Content
The actual
information

Self Attention

– Step 2

Now, **we need to calculate a score** to determine how much focus to place on other Parts of the input.

Score each word of the input against the word "Thinking"

Input

Embedding

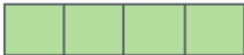
Queries

Keys

Values

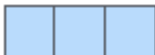
Score

Thinking

x_1 

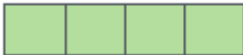
q_1 

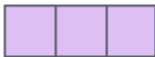
k_1 

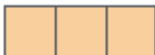
v_1 

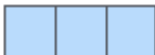
$$q_1 \cdot k_1 = 112$$

Machines

x_2 

q_2 

k_2 

v_2 

$$q_1 \cdot k_2 = 96$$

Self Attention

– Step 5 and 6

Formula

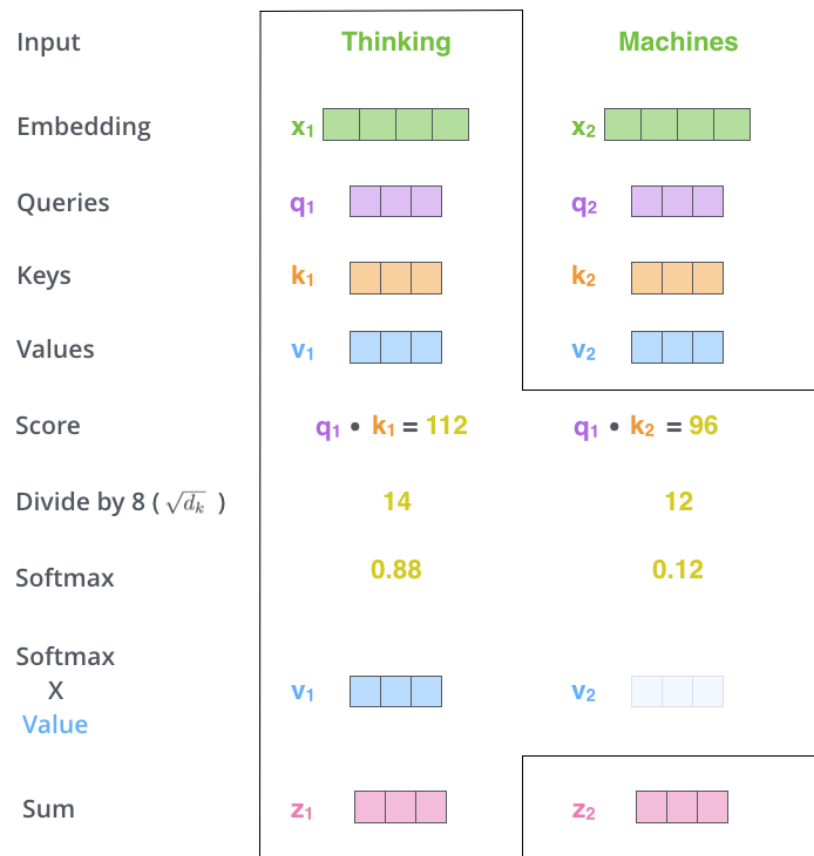
$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V$$

64x64 64x512

Q K^T V

Z

$d_k=64$ is dimension of key vector



$$z_1 = 0.88v_1 + 0.12v_2$$

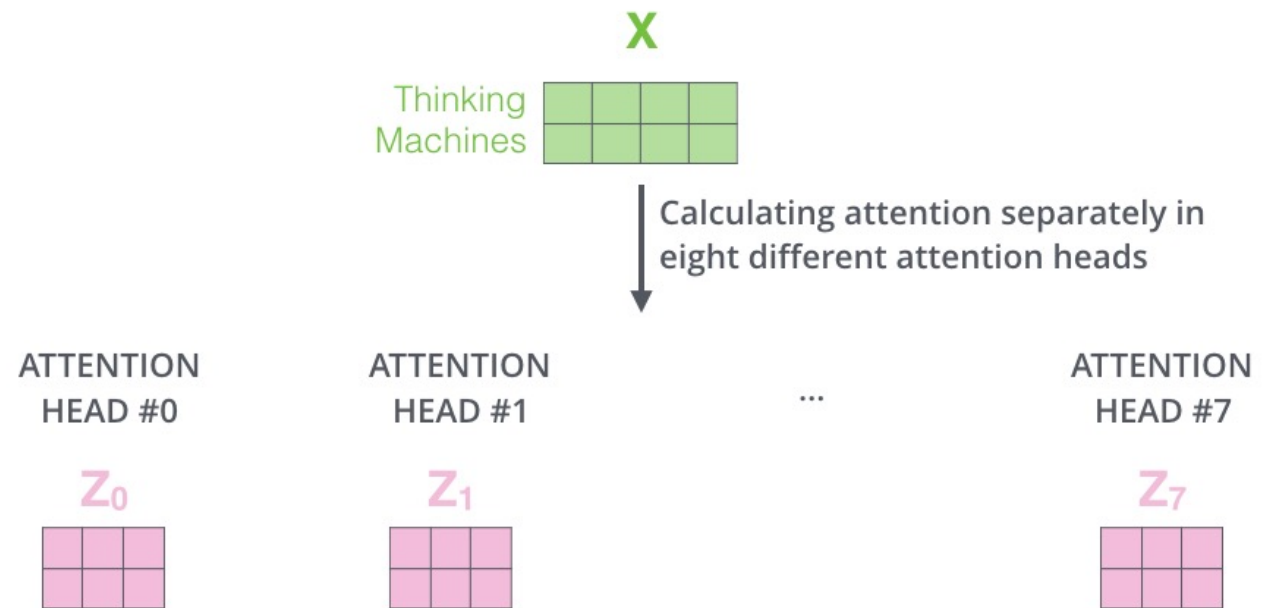
This produces the output of the self-attention layer at this position (for the first word).

Example

- Let's consider a multi-head attention mechanism in a Transformer model used for machine translation
 - Translating a complex sentence from English to French. The sentence is: "The lawyer who won the case thanked the jury."
- In a multi-head attention setup, different heads might focus on different aspects of this sentence:
 1. **Head 1:** Focuses on the subject-verb relationship, linking "lawyer" to "thanked".
 2. **Head 2:** Concentrates on the clause, understanding the relationship between "who" and "won the case".
 3. **Head 3:** Pays attention to the object of the sentence, connecting "thanked" to "jury".
- Each head captures different dependencies and contexts:
 1. **Head 1** understands who is doing the action.
 2. **Head 2** provides context about the lawyer's achievement.
 3. **Head 3** identifies who is being thanked.

Multiple Heads

1. It expands the model's ability to focus on different positions.
2. It gives the attention layer multiple “representation subspaces”

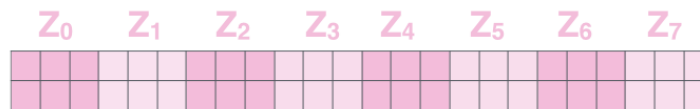


If we do the same self-attention calculation we outlined above, just eight different times with different weight matrices, we end up with eight different Z matrices

Multiple Heads

The output is
expecting
only a 2x4
matrix, hence,

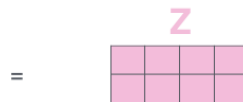
1) Concatenate all the attention heads



2) Multiply with a weight matrix W^O that was trained jointly with the model

x

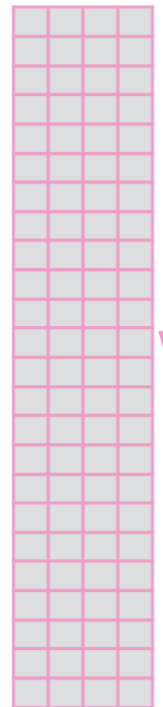
3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



4

24

W^O



If you want some more intuition on attention: watch <https://www.youtube.com/watch?v=-9vVhYEXeyQ>

Putting it all together

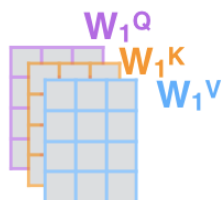
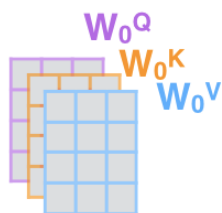
1) This is our input sentence*

Thinking
Machines

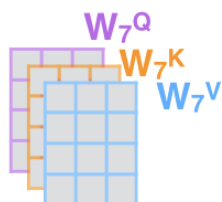
2) We embed each word*



3) Split into 8 heads. We multiply X or R with weight matrices



...



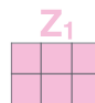
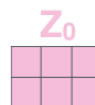
4) Calculate attention using the resulting $Q/K/V$ matrices



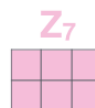
...



5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



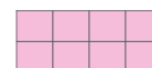
...



W^O



Z

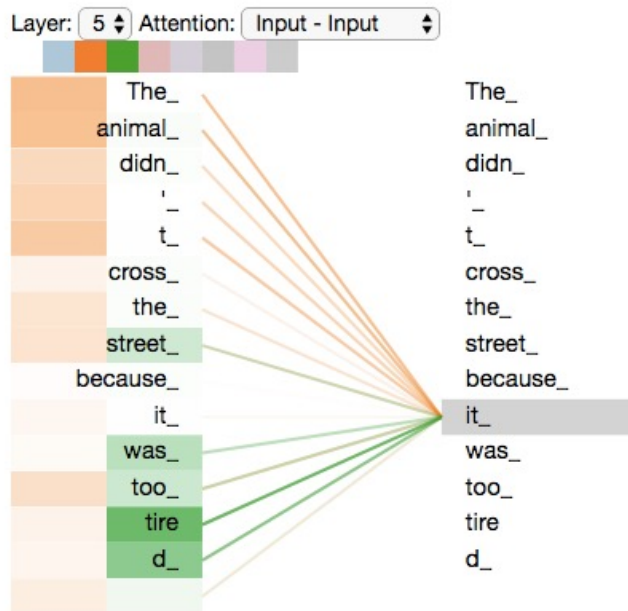


* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



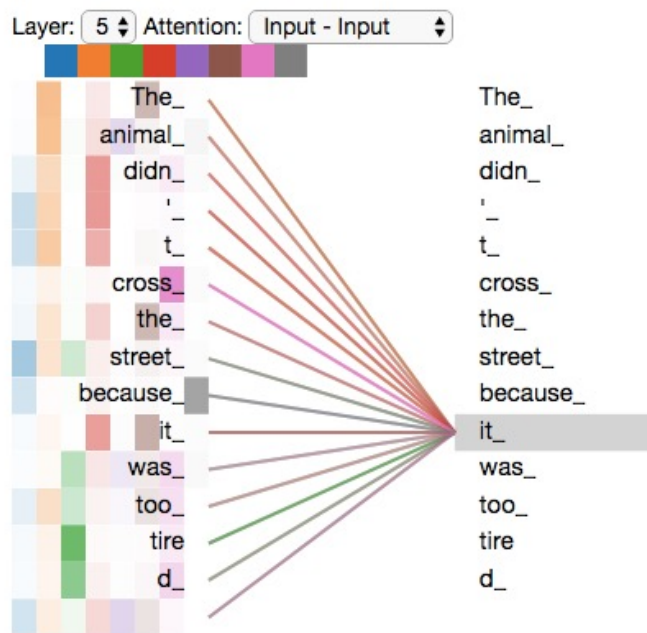
HPML

Attention Head Interpretation



As we encode the word "it", one attention head is focusing most on "the animal", while another is focusing on "tired" - in a sense, the model's representation of the word "it" bakes in some of the representation of both " animal " and " tired ".

Attention Heads



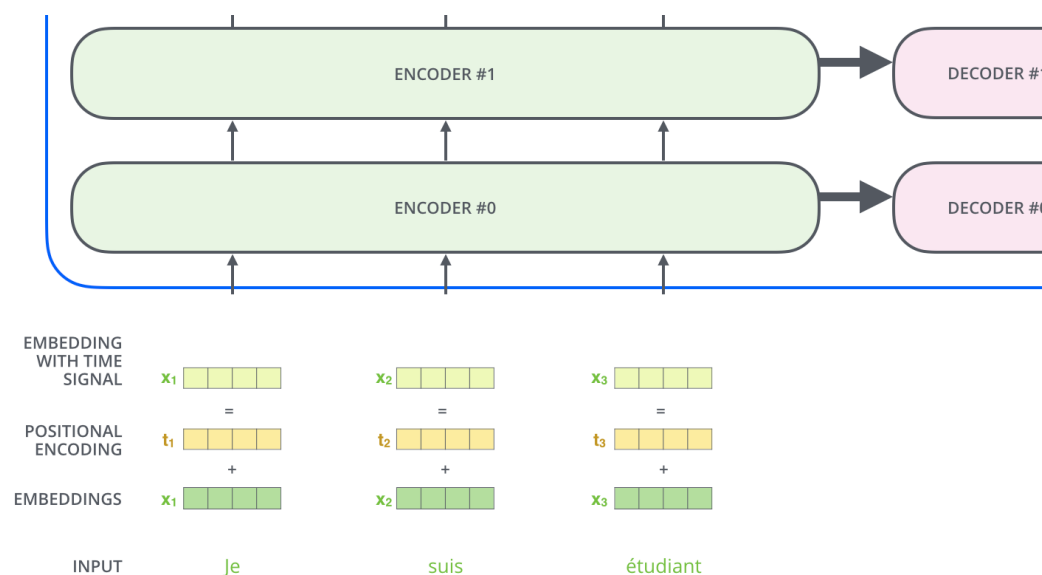
Adding all Attention Heads to the pictures,
It becomes hard to interpret

Representing the input order (positional encoding)

The transformer adds a vector to each input embedding. These vectors follow a specific pattern that the model learns, which helps it determine the position of each word, or the distance between different words in the sequence. The intuition here is that adding these values to the embeddings provides meaningful distances between the embedding vectors once they're projected into Q/K/V vectors and during dot-product attention.

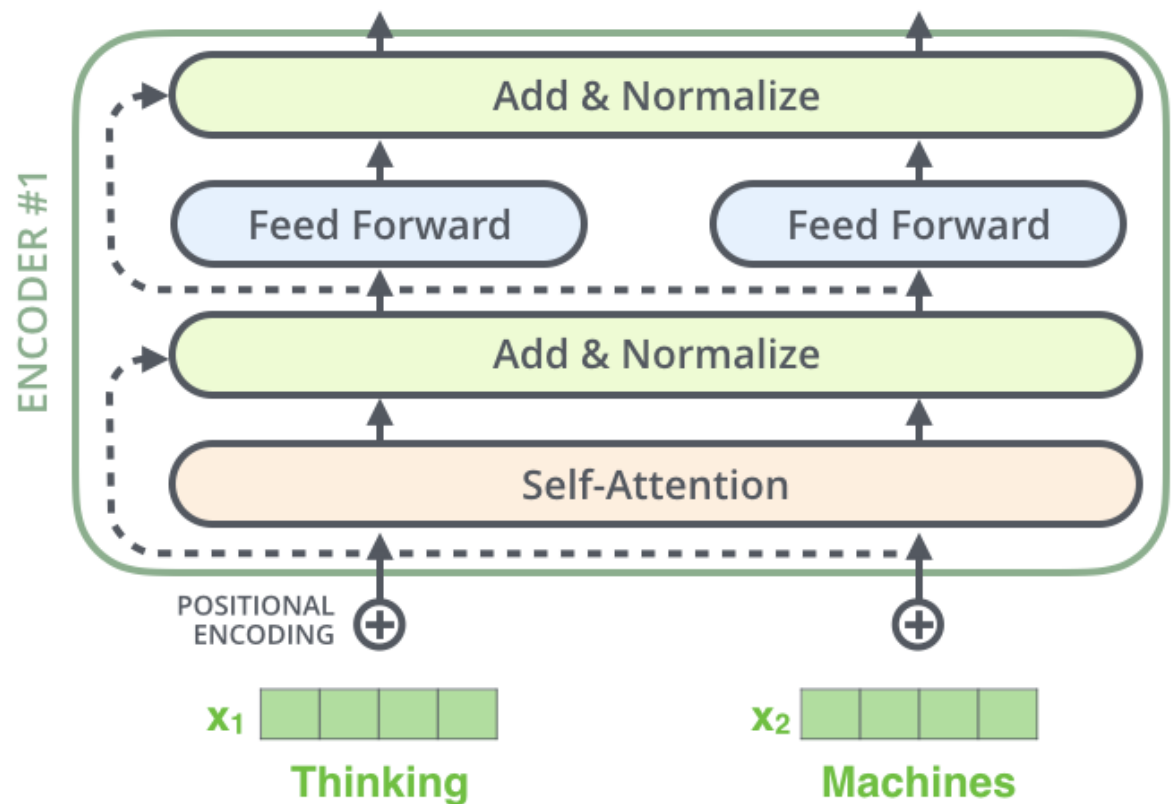
Give the model a sense of the order of the words

More on positional encoding:
https://kazemnejad.com/blog/transformer_architecture_positional_encoding/



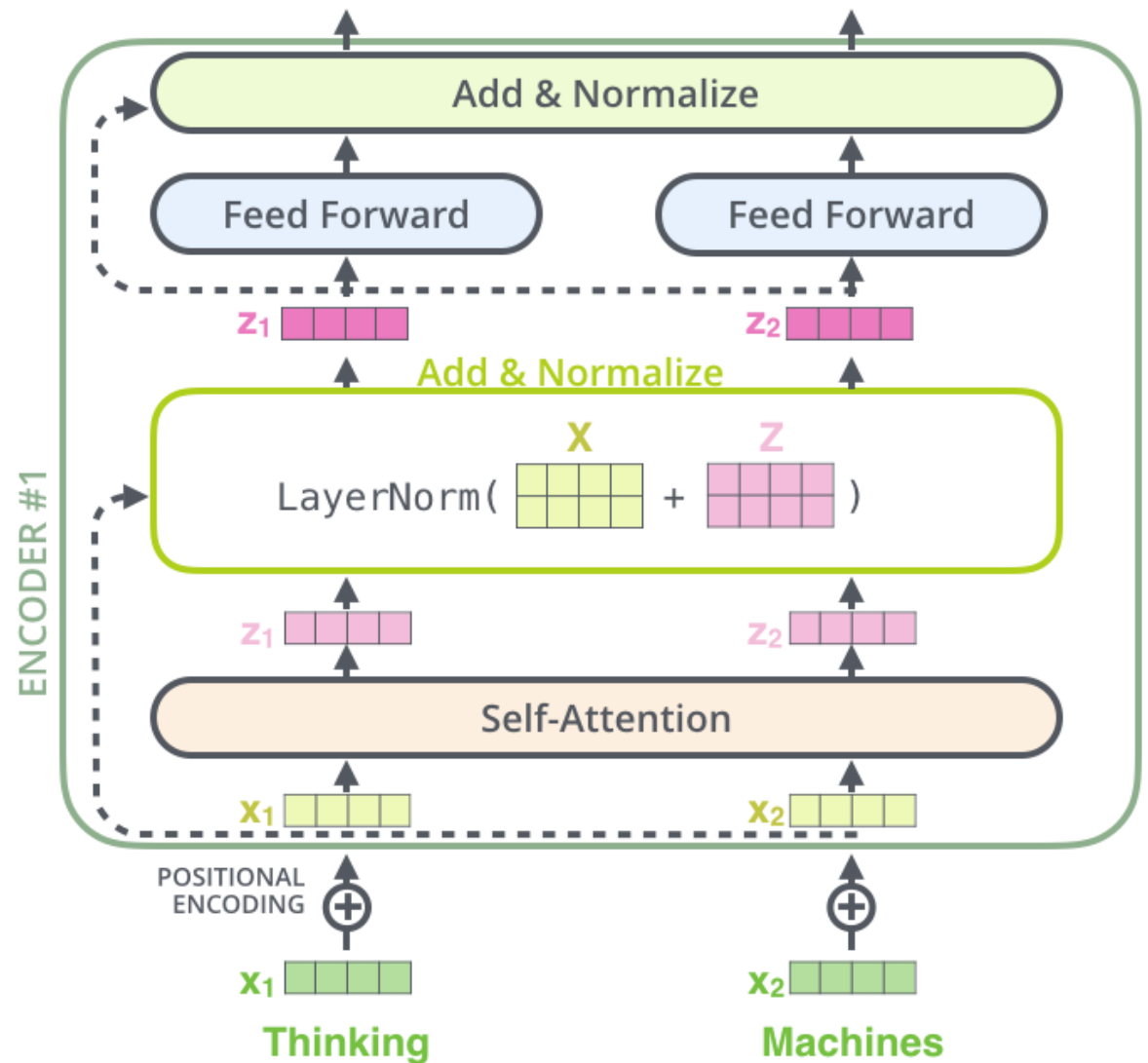
The Residuals

Each sub-layer (self-attention, ffn) in each encoder has a residual connection around it, and is followed by a [layer-normalization](#) step



Add and Normalize

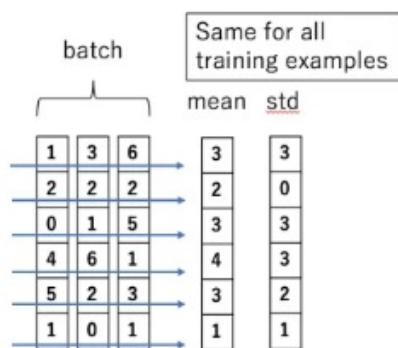
In order to regulate the computation, this is a normalization layer so that each feature (column) have the same average and deviation.



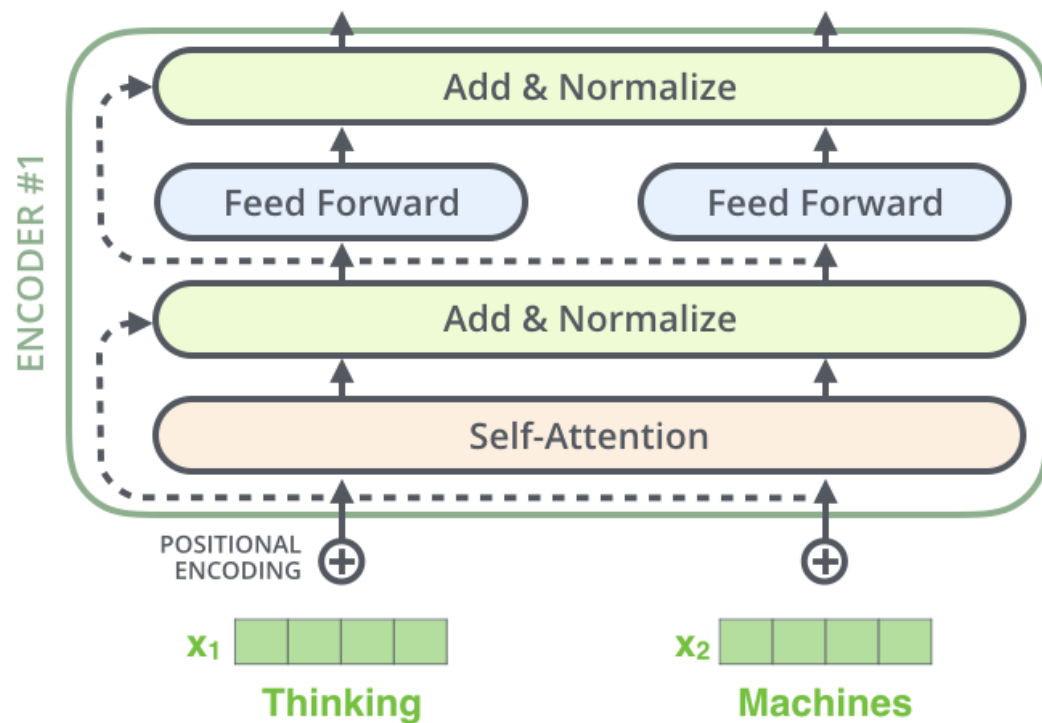
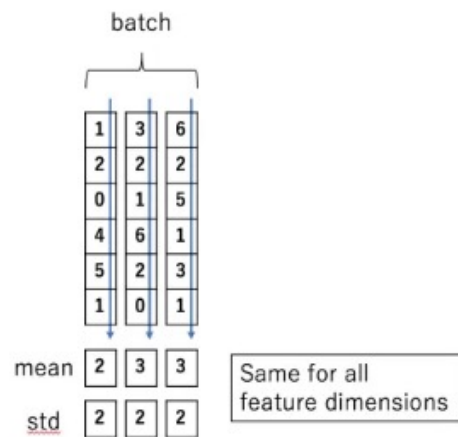
Layer Normalization (Hinton)

Layer normalization normalizes the inputs across the features.

Batch Normalization

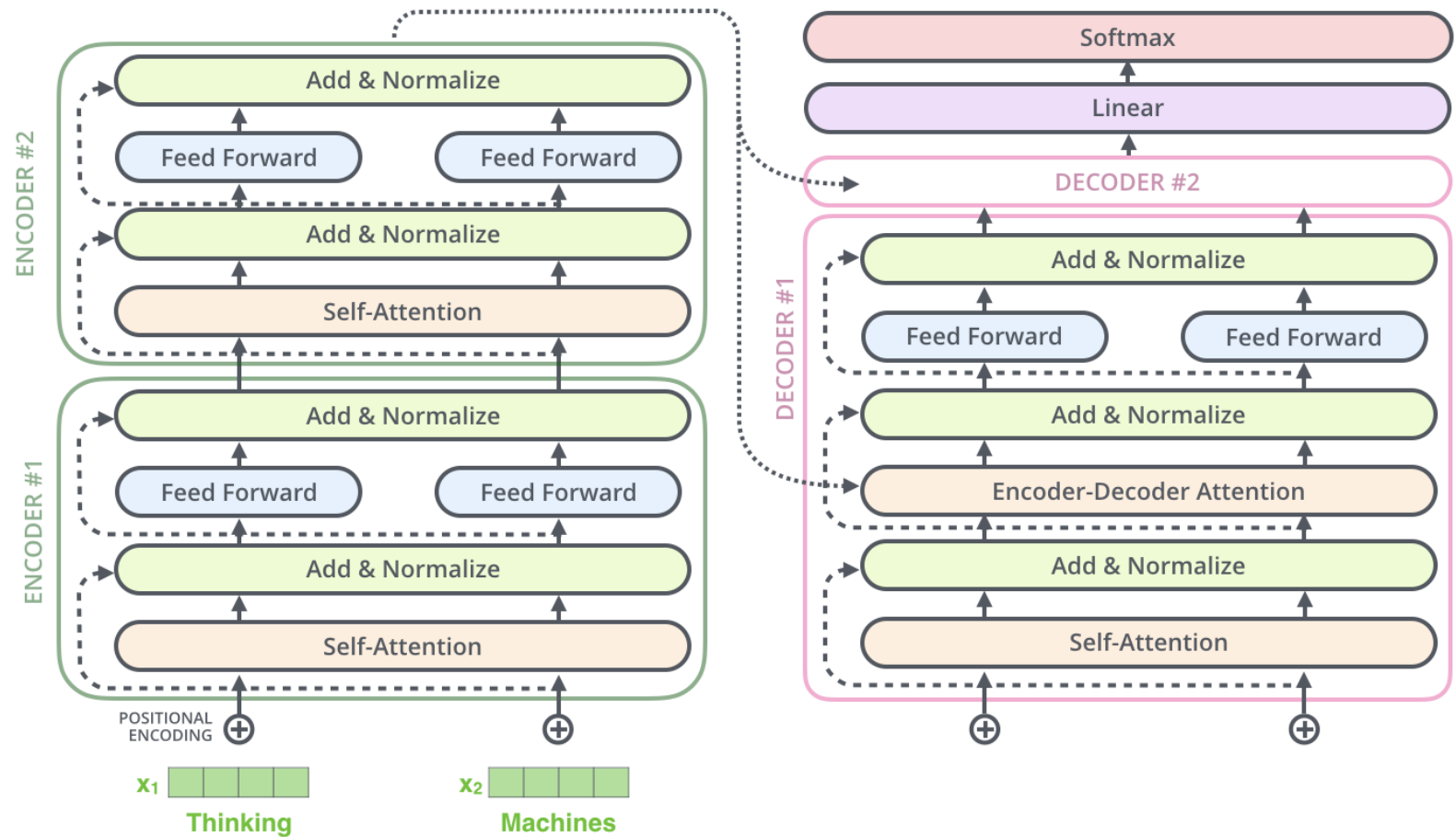


Layer Normalization



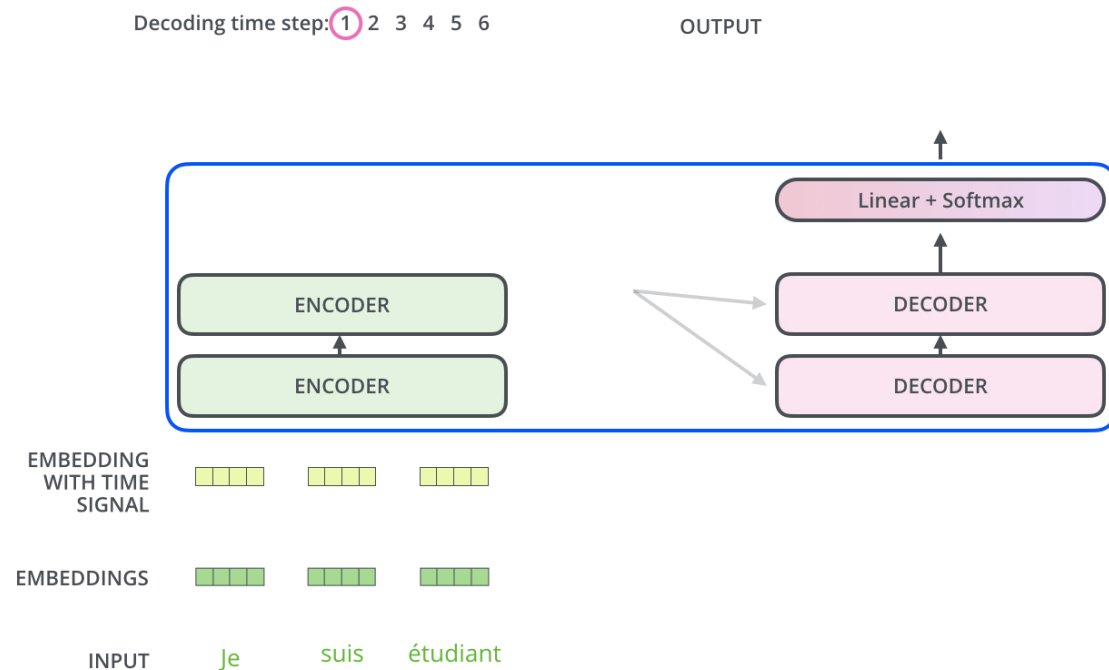
The complete transformer

The encoder-decoder attention is just like self attention, except it uses K, V from the top of encoder output, and its own Q



The decoder

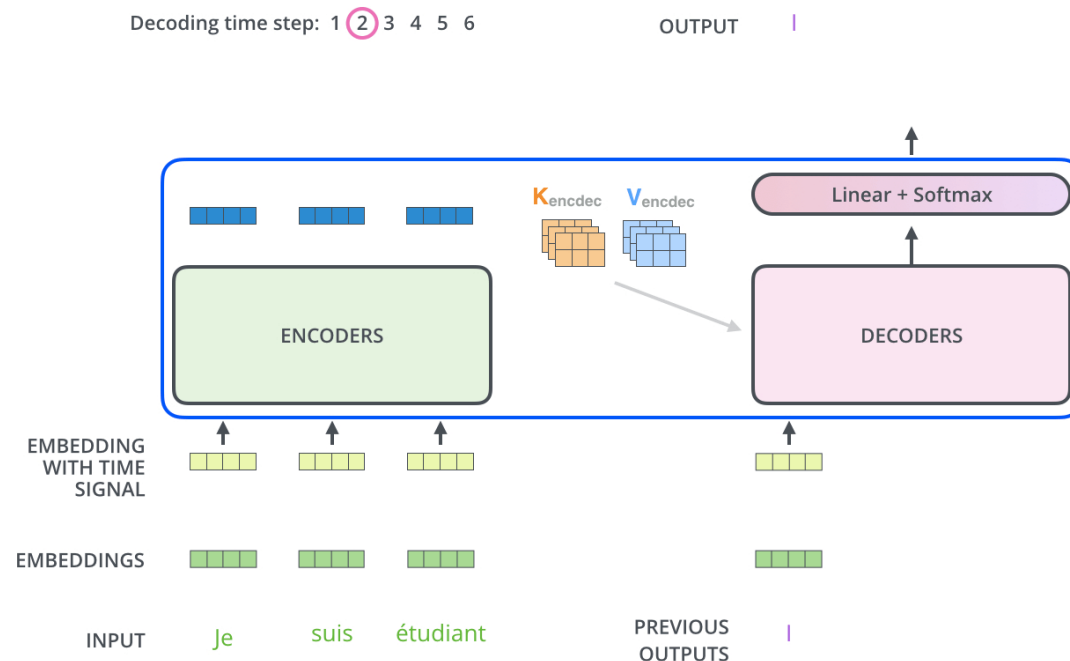
- The encoder starts by processing the input sequence.
- The output of the top encoder is then transformed into a set of attention vectors K and V.
- These are to be used by each decoder in its “encoder-decoder attention” layer which helps the decoder focus on appropriate places in the input sequence



After finishing the encoding phase, we begin the decoding phase. Each step in the decoding phase outputs an element from the output sequence (the English translation sentence in this case).

The decoder

- We repeat the process until a special symbol is reached indicating the transformer decoder has completed its output.
- The output of each step is fed to the bottom decoder in the next time step
- The decoders bubble up their decoding results just like the encoders did.
- We embed and add positional encoding to those decoder inputs to indicate the position of each word.



In the decoder, the self-attention layer can only attend to earlier positions in the output sequence. This is done by masking future positions (setting them to $-\infty$) before the softmax step in the self-attention calculation.

Note: In the decoder, the input is "incomplete" when calculating self-attention.

The solution is to set future unknown values with `"-inf"` .

The Final Linear and Softmax Layer

- The decoder stack outputs a vector of floats.
- How do we turn that into a word?
- That's the job of the final Linear layer which is followed by a Softmax Layer.
- The cell with the highest probability is chosen.

Which word in our vocabulary is associated with this index?

am

Get the index of the cell with the highest value (argmax)

5

log_probs



Softmax

logits



Linear

Decoder stack output

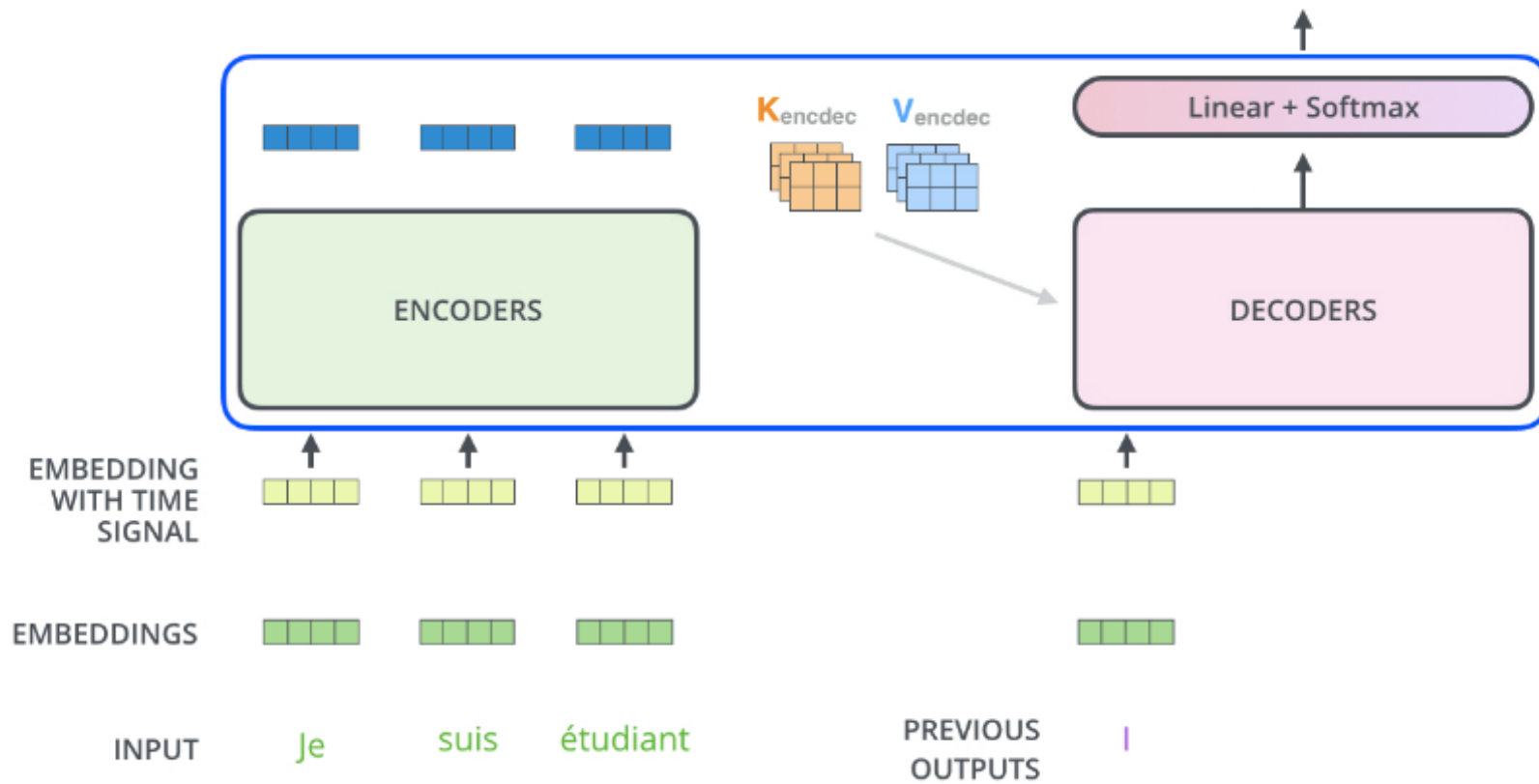


How it works

Decoding time step: 1 2 3 4 5 6

OUTPUT

| am



But what about Self-attention?

Cross Entropy and KL (Kullback-Leibler) divergence

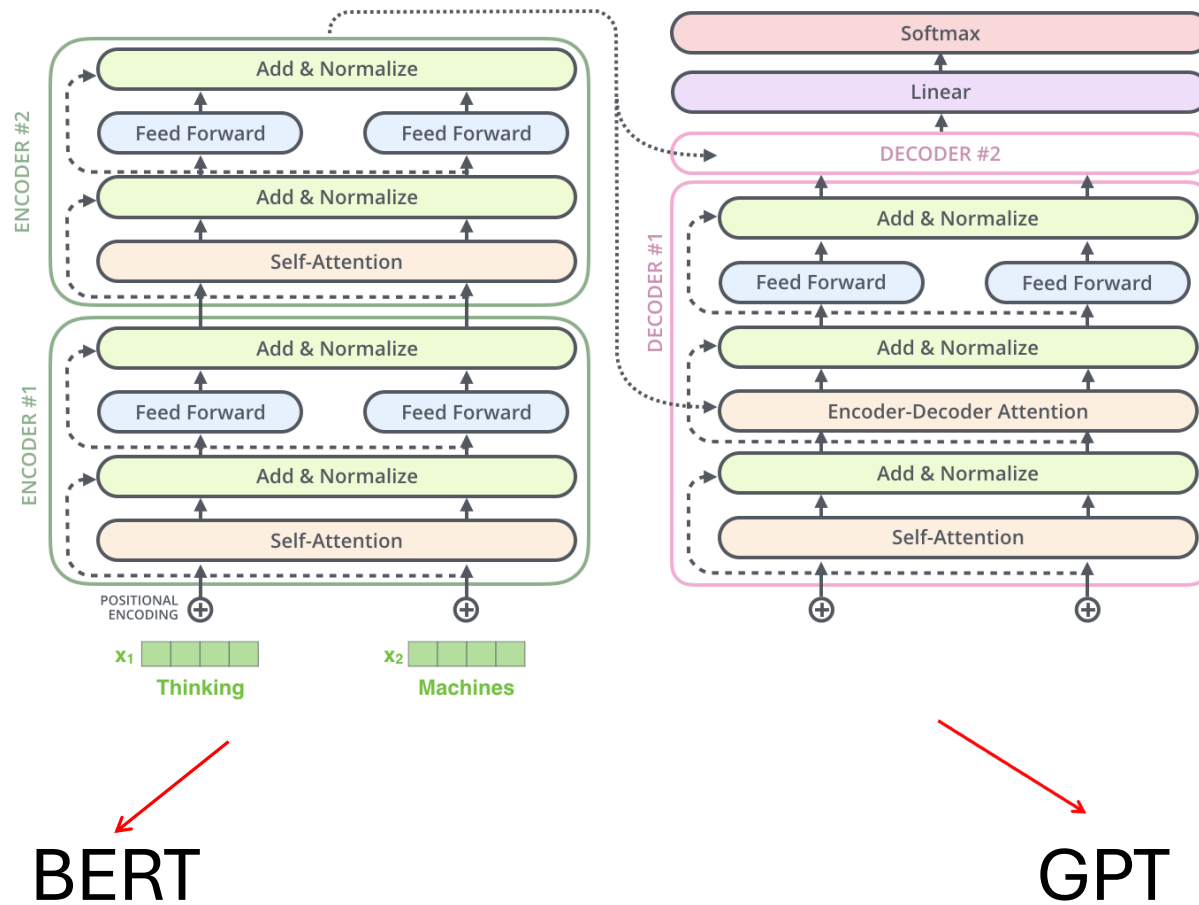
- **Entropy**: $E(P) = - \sum_i P(i) \log P(i)$ - expected prefix-free code length (also optimal)
 - **Cross Entropy**: $C(P) = - \sum_i P(i) \log Q(i)$ – expected coding length using optimal code for Q
 - **KL divergence**:
 $D_{KL}(P \parallel Q) = \sum_i P(i) \log[P(i)/Q(i)] = \sum_i P(i) [\log P(i) - \log Q(i)]$, extra bits to code using Q rather than P
 - **JSD**(P||Q) = $\frac{1}{2} D_{KL}(P||M) + \frac{1}{2} D_{KL}(Q||M)$, $M = \frac{1}{2} (P+Q)$, symmetric KL
- * JSD = Jensen-Shannon Divergency

Transformer Results

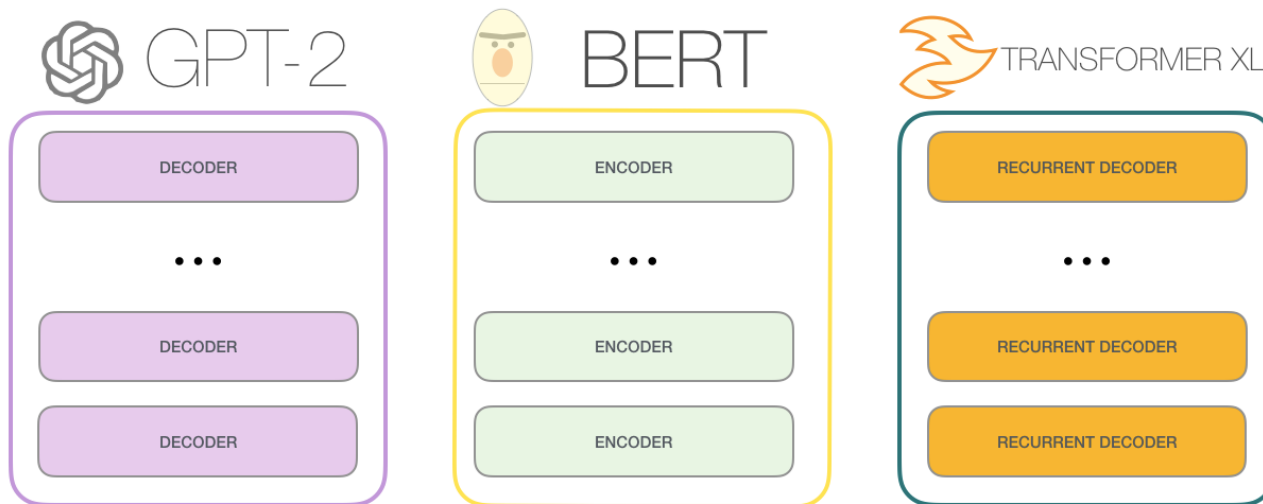
Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

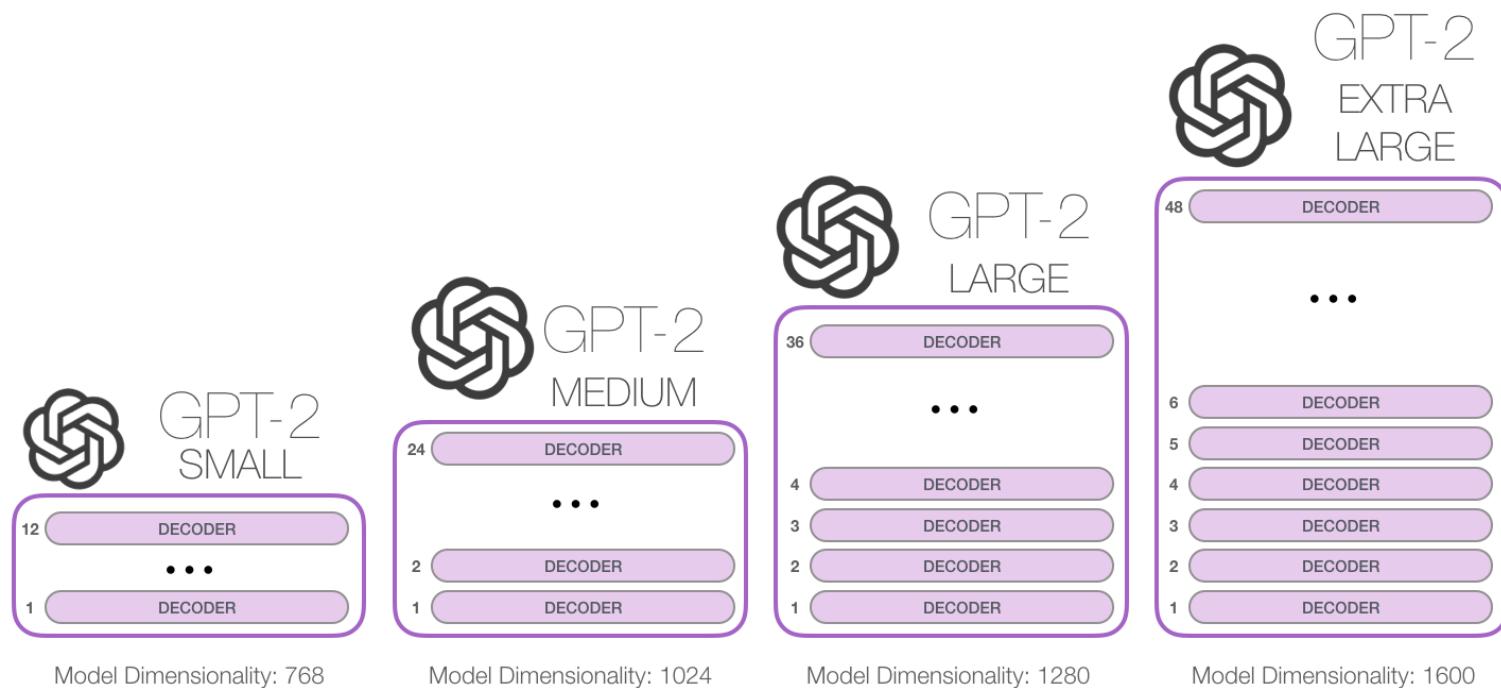
Pretraining



Transformers for Language Modeling

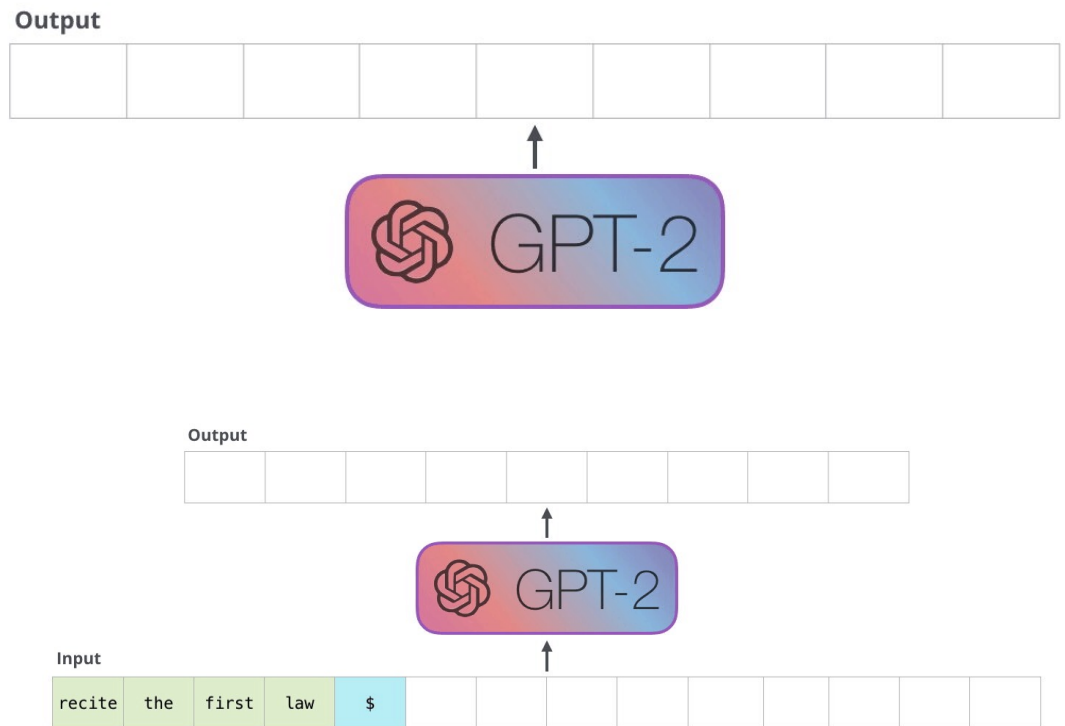


How high can we stack?

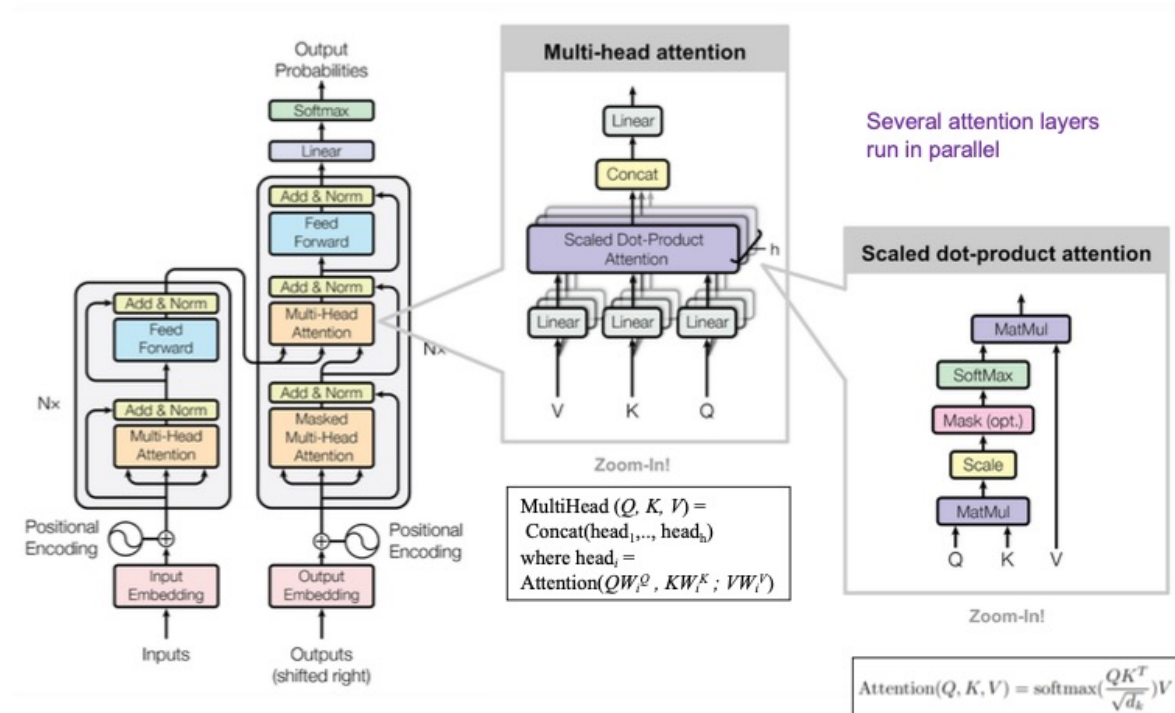


BERT vs. GPT-2

- The GPT-2 is built using transformer decoder blocks.
- BERT uses transformer encoder blocks.
- Autoregression:
 - After each token is produced, that token is added to the sequence of inputs.
 - The new sequence becomes the input to the model in its next step.
 - This is an idea called “auto-regression”.

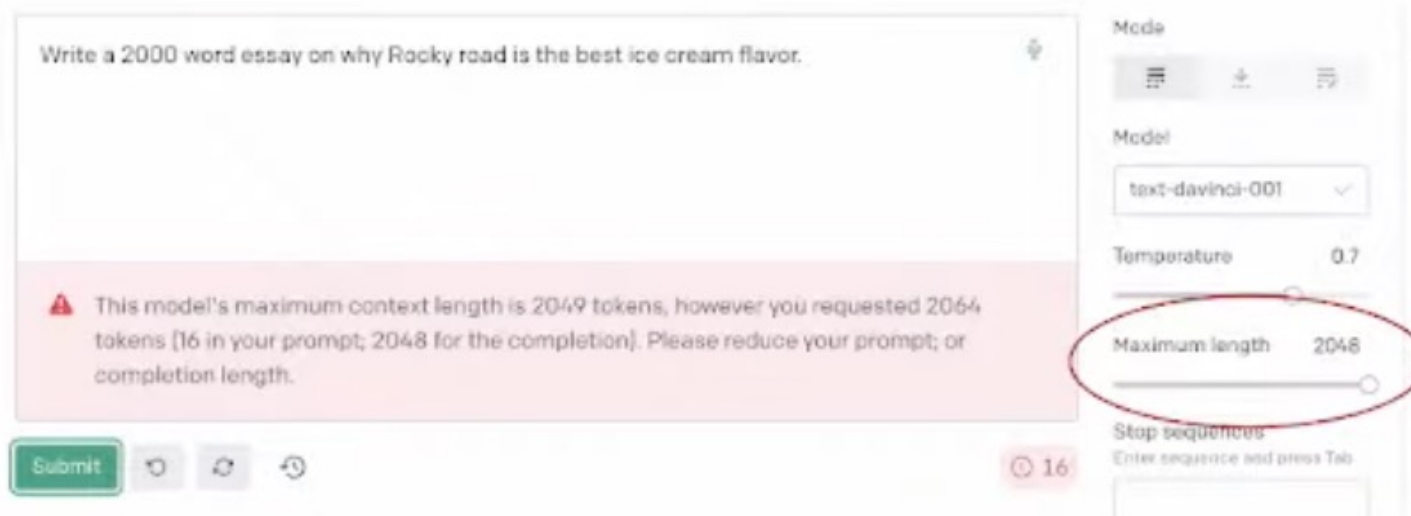


Multi-head Attention



Naïve Transformer's Limitations

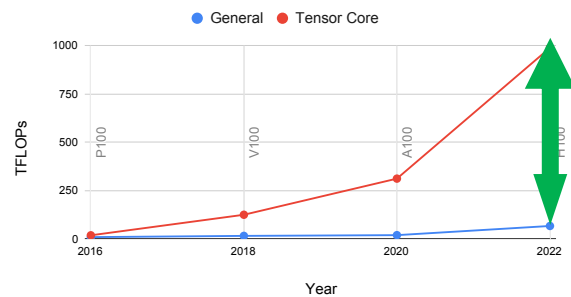
- Example using GPT3:
 - Ask GPT3 to write a 2000-word essay



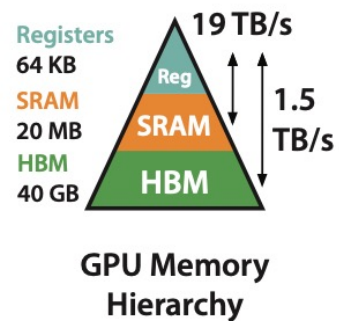
The screenshot shows the OpenAI GPT-3 web interface. The main text area contains the prompt: "Write a 2000 word essay on why Rocky road is the best ice cream flavor." Below the text area, a red error message states: "This model's maximum context length is 2049 tokens, however you requested 2064 tokens [16 in your prompt; 2048 for the completion]. Please reduce your prompt; or completion length." To the right of the text area, there are settings for "Mode", "Model" (set to "text-davinci-001"), "Temperature" (set to 0.7), and "Maximum length" (set to 2048). The "Maximum length" setting is circled in red. At the bottom left, there is a green "Submit" button and three circular icons. At the bottom right, there is a red icon with the number 16.

The Classical Balance

GPU TFLOPs over Time



Processing Speed
(Arithmetic Intensity)

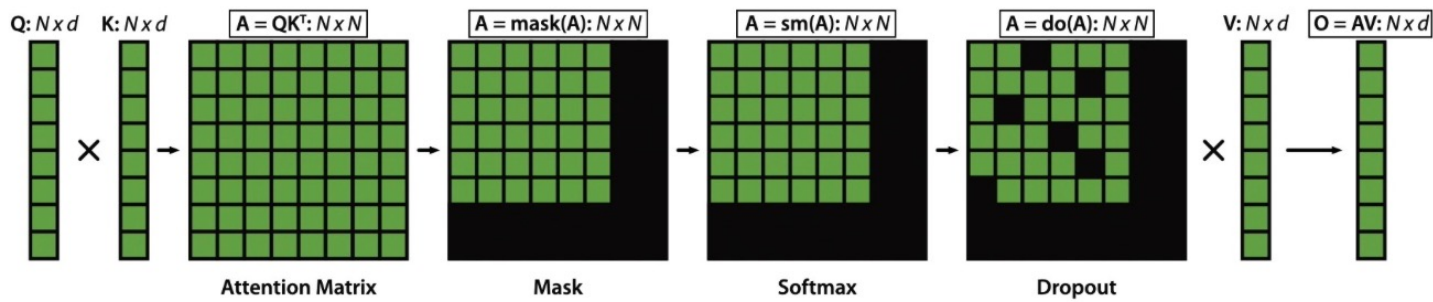


Memory Bottleneck
(Memory Intensity)



As accelerators get faster, memory is more often the bottleneck.

Attention Bottleneck: Memory Reads/Writes



$$O = \text{Dropout}(\text{Softmax}(\text{Mask}(QK^T)))V$$

**Naive implementation requires
repeated R/W from slow GPU HBM**

[FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness](#)

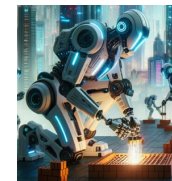
DATA MOVEMENT IS ALL YOU NEED: A CASE STUDY ON OPTIMIZING TRANSFORMERS

Andrei Ivanov^{*1} Nikoli Dryden^{*1} Tal Ben-Nun¹ Shigang Li¹ Torsten Hoefler¹

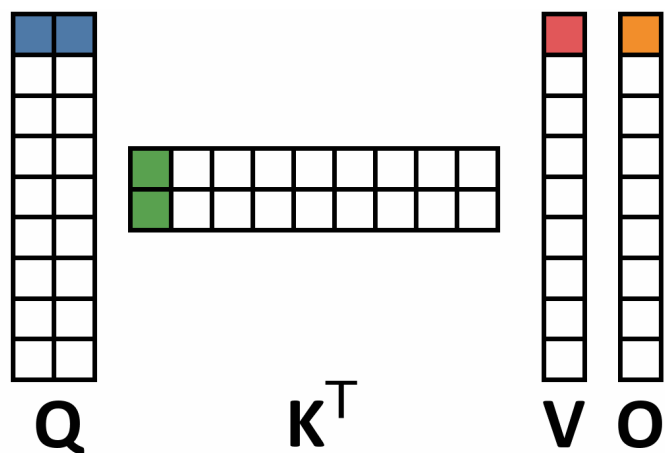
ABSTRACT

Transformers are one of the most important machine learning workloads today. Training one is a very compute-intensive task, often taking days or weeks, and significant attention has been given to optimizing transformers. Despite this, existing implementations do not efficiently utilize GPUs. We find that data movement is the key bottleneck when training. Due to Amdahl's Law and massive improvements in compute performance, training has now become memory-bound. Further, existing frameworks use suboptimal data layouts. Using these insights, we present a recipe for globally optimizing data movement in transformers. We reduce data movement by up to 22.91% and overall achieve a 1.30× performance improvement over state-of-the-art frameworks when training a BERT encoder layer and 1.19× for the entire BERT. Our approach is applicable more broadly to optimizing deep neural networks, and offers insight into how to tackle emerging performance bottlenecks.

- Minimize IO (HBM to SRAM) – not FLOPs
- Aggressive **fusion**: when you pull in data use it.



Minimize IO to HBM in Flash Attention



$r(Q) = 9$ is the rows of Q, $|Q|$ is number of tiles.

$$|Q| + r(Q)(|K| + |V|) + |O| = 18 + 9 \times (18 + 9) + 9 = 270 \text{ IO}$$

How to Reduce HBM Read/Writes: Compute by Blocks

- Main Challenges:
 1. Compute software reduction without access to full input
 2. Backward without the large matrix from the forward

Tiling Idea

Think of attention as a nested loop:
for each block of Q : $\leftarrow$ outer loop
 for each block of K, V : $\leftarrow$ inner loop
 compute partial attention

● Inner Loop (Blue in the slide)

→ Sweeps over K and V blocks for one tile of Q

Fix **one tile of Q** (e.g., 128 queries)

Load a **small tile of K and V** from HBM → SRAM

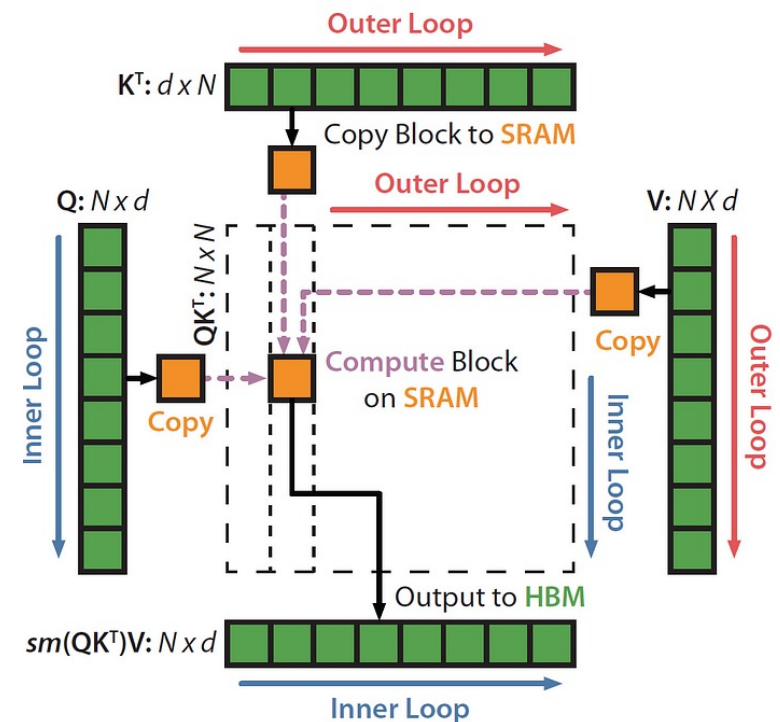
Compute $Q \times K^T$ for that small tile

Update partial softmax

Move to next K/V tile

Inner loop = “scan all K/V tiles for this particular Q tile.”

HPML



Tiling Idea

Think of attention as a nested loop:
for each block of Q: ← **outer loop**
 for each block of K,V: ← **inner loop**
 compute partial attention

● **Outer Loop (Red in the slide)**

→ **Sweeps over tiles of Q**

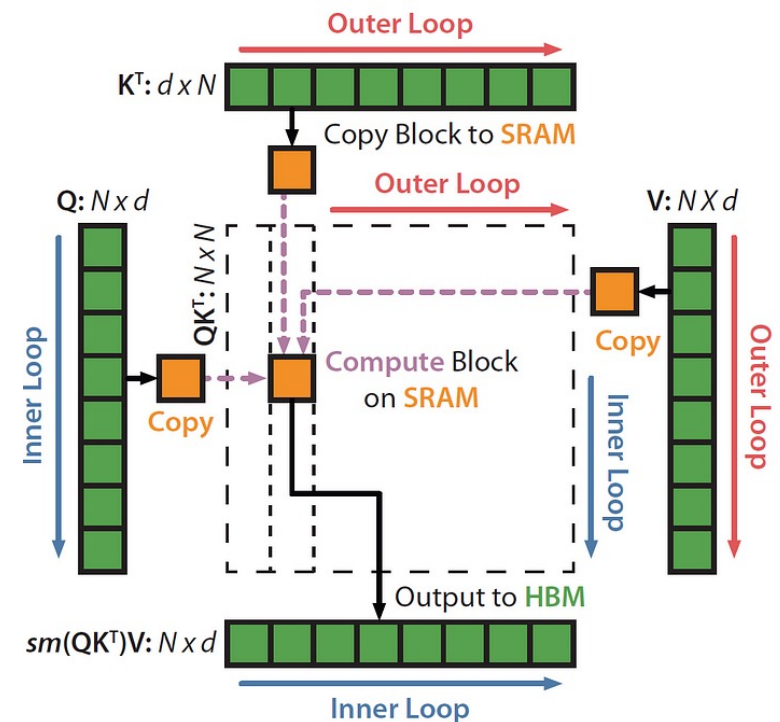
After finishing all K/V tiles for Q-block 1:

Move to the next tile of Q

Repeat inner loop

Produce next chunk of the output

Outer loop = “move to the next group of queries.”



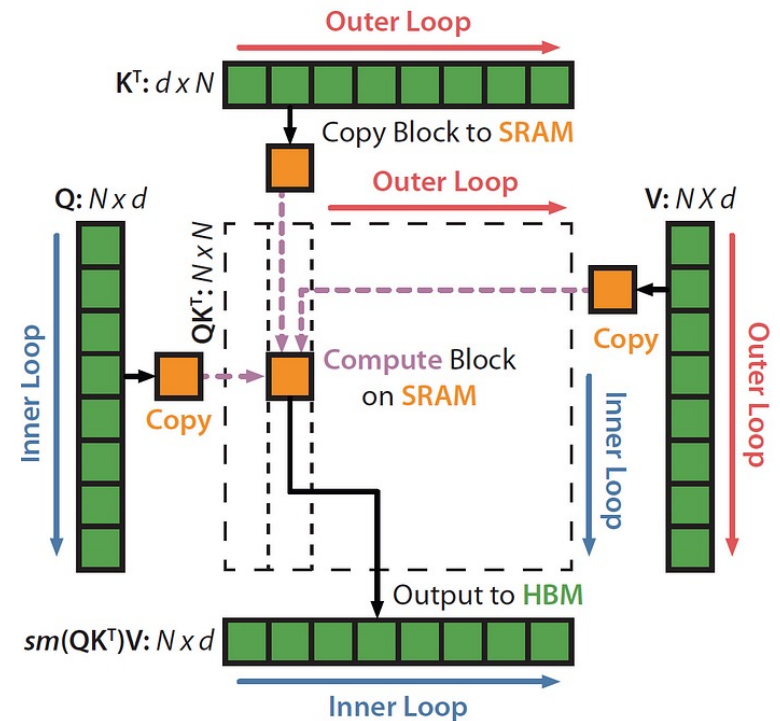
Tiling Idea

Think of attention as a nested loop:
 for each block of Q: $\leftarrow$ outer loop
 for each block of K,V: $\leftarrow$ inner loop
 compute partial attention

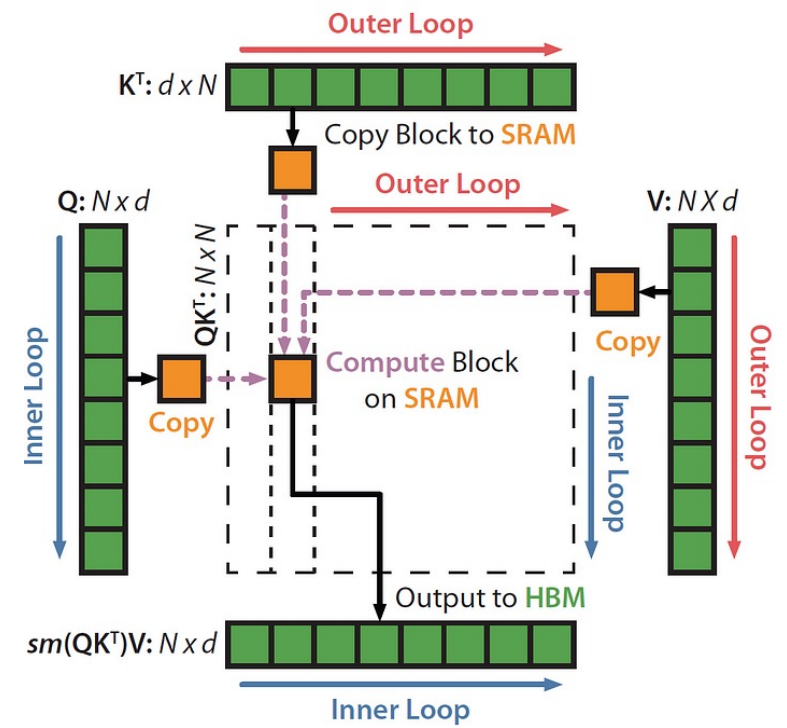
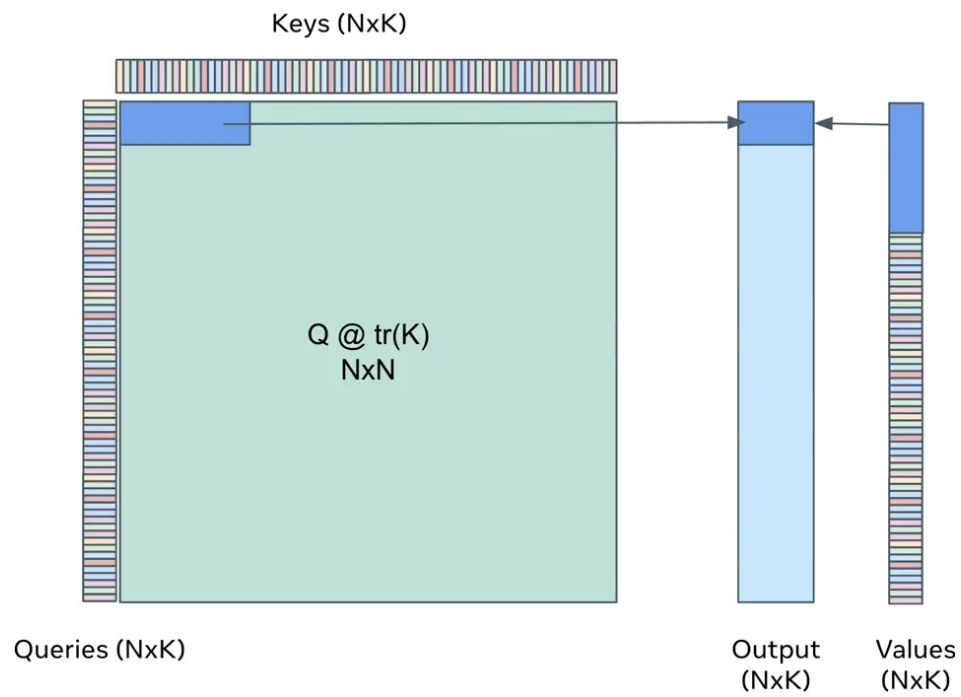
- Decomposing large softmax into smaller ones:

$$\text{softmax}([A_1, A_2]) = [\alpha \text{softmax}(A_1), \beta \text{softmax}(A_2)] .$$

$$\text{softmax}([A_1, A_2]) \begin{bmatrix} V_1 \\ V_2 \end{bmatrix} = \alpha \text{softmax}(A_1) V_1 + \beta \text{softmax}(A_1) V_2 .$$



Illustration

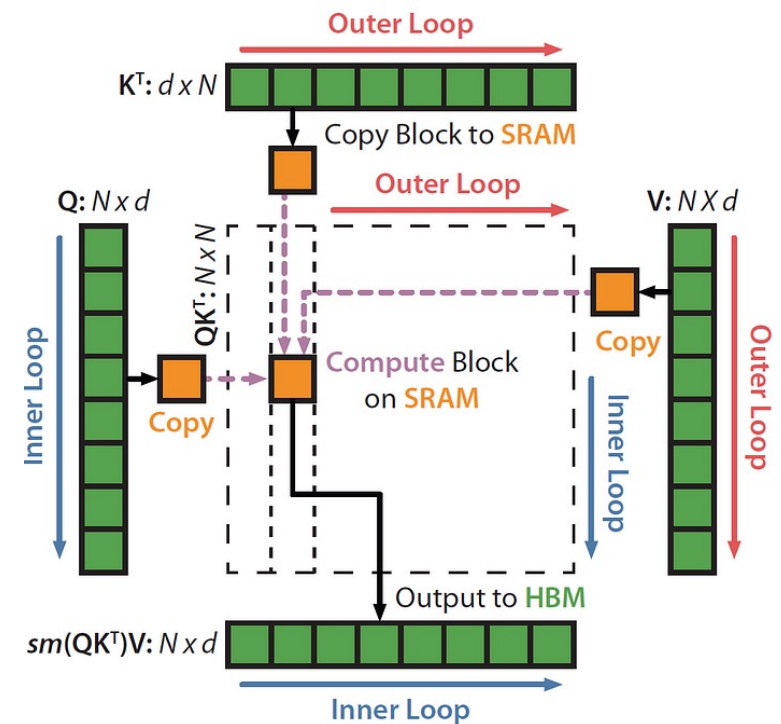


Recomputation Technique

- By storing softmax normalization factors from forward pass, we can quickly recompute attention in the backward from inputs in stored in SRAM

Attention	Standard	FLASHATTENTION
GFLOPs	66.6	75.2
HBM R/W (GB)	40.3	4.4
Runtime (ms)	41.7	7.3

Forward and backward runtime of standard attention vs FlashAttention for GPT-2 medium (seq. Len 1024, head dim. 64. batch size 64 on A100 GPU)



<https://arxiv.org/pdf/2205.14135.pdf>

How to deal with Softmax Challenge (1/2)

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

- Chop the Softmax computation into smaller blocks while producing precisely the same result.
- We can take the first B scores (x_1 through x_B) and compute their softmax.
- **Partial computation of softmax (will iteratively get to the correct softmax numbers).**

$$m(x) := \max_i x_i, \quad f(x) := [e^{x_1 - m(x)} \quad \dots \quad e^{x_B - m(x)}], \quad \ell(x) := \sum_i f(x)_i, \quad \text{softmax}(x) := \frac{f(x)}{\ell(x)}.$$

FlashAttention Algorithm

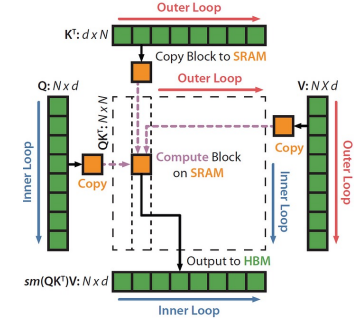
Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

Notation: d — attention head dimension, B_r (**row** block size), B_c (**column** block size)

HPML



Step 1: Compute the row/column block sizes.

Why $\text{ceil}(M/4d)$? Because query, key, and value vectors are d -dimensional, we must combine them into the output d -dimensional vector (o). So, this size allows us to max out SRAM capacity with q , k , v , and o vectors.

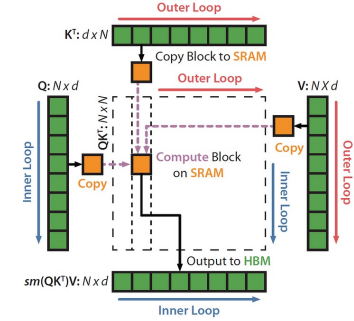
FlashAttention Algorithm

Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

Notation: d — attention head dimension, B_r (**row** block size), B_c (**column** block size)



Step 2:

- initialize the output matrix $\mathbf{O}$ with all 0s as it acts as an accumulator
- Initialize the ℓ to 0. Its purpose is to hold the cumulative denominator for the softmax (sum of exp of the scores)
- Initialize m to inf because it holds the row-wise max scores.

FlashAttention Algorithm

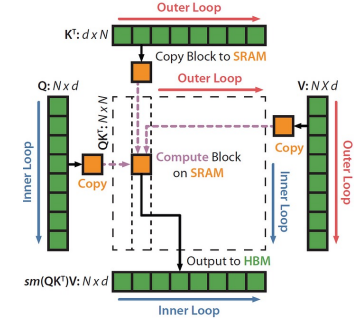
Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

Notation: d — attention head dimension, B_r (**row** block size), B_c (**column** block size)

HPML



Step 3:

- split the $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ into blocks using the block sizes from Step 1.

FlashAttention Algorithm

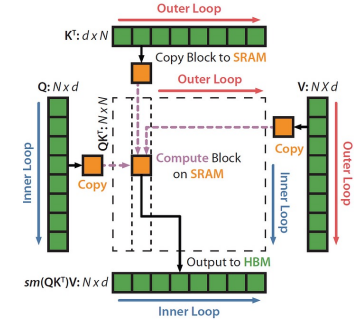
Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$ of size $B_r \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

Notation: d — attention head dimension, B_r (row block size), B_c (column block size)

HPML



Step 4:

- split $\mathbf{O}$, ℓ , m into blocks (same block size as $\mathbf{Q}$).

FlashAttention Algorithm

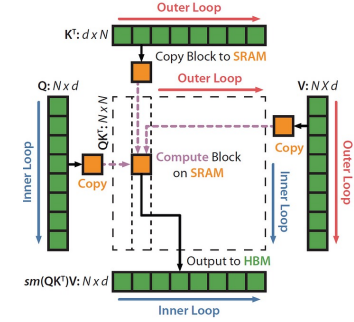
Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

Notation: d — attention head dimension, B_r (row block size), B_c (column block size)

HPML



Step 5:

- Loop across the columns (key/value vectors) as shown in the outer loop in the diagram

FlashAttention Algorithm

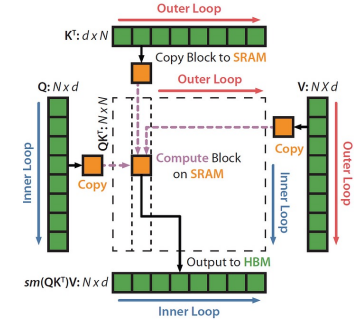
Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

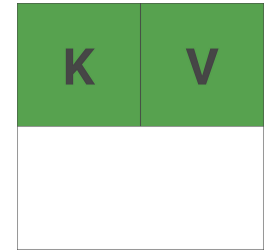
Notation: d — attention head dimension, B_r (**row** block size), B_c (**column** block size)

HPML



Step 6:

- Load the $\mathbf{K}_j$ and $\mathbf{V}_j$ blocks from HBM to SRAM.
- Because of the way we constructed the block sizes, 50% of the SRAM is still unoccupied at this point (dedicated to $\mathbf{Q}$ and $\mathbf{O}$).



FlashAttention Algorithm

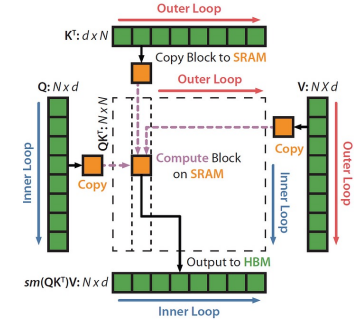
Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

Notation: d — attention head dimension, B_r (**row** block size), B_c (**column** block size)

HPML



Step 8:

- Load $\mathbf{Q}_i$ ($B_r \times d$) and $\mathbf{O}_i$ ($B_r \times d$) blocks, as well as ℓ_i (B_r) & m_i (B_r) into SRAM.

FlashAttention Algorithm

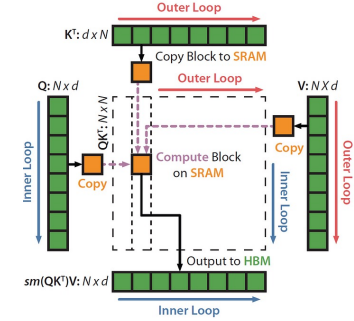
Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

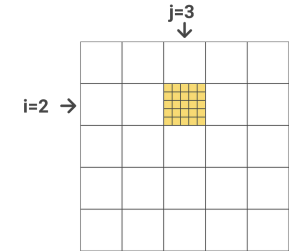
Notation: d — attention head dimension, B_r (row block size), B_c (column block size)

HPML



Step 9:

- Compute the dot product between $\mathbf{Q}_i$ ($B_r \times d$) and $\mathbf{K}_j$ transposed ($d \times B_c$) to get the scores ($B_r \times B_c$). As you can see we don't have the whole $N \times N$ S(scores) matrix "materialized". Only a fraction of it ($\mathbf{S}_{ij}$).



FlashAttention Algorithm

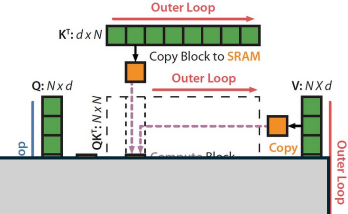
Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

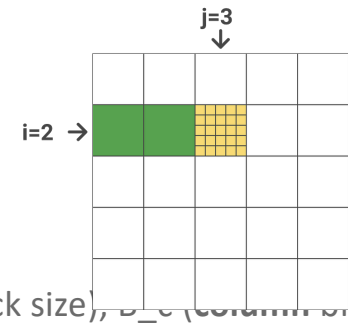
Notation: d — attention head dimension, B_r (row block size), B_c (column block size)

HPML



Step 11:

- Compute m_{new_i} and ℓ_{new_i}
- m_i contains row-wise maximums for all of the blocks that came before ($j=1$ & $j=2$, colored in green).
- $m_{\sim i_j}$ contains the row-wise maximums for the current block (colored in yellow).
- To get the m_{new_i} we just have to apply a max between $m_{\sim i_j}$ & m_i .



FlashAttention Algorithm

Algorithm

Require

- 1: Set
- 2: Init
- 3: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_1, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_1, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 4: Divide $\mathbf{K}$ into T_c blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$ of size $B_c \times d$ each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}, m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

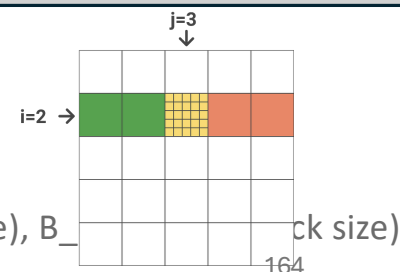
$$\text{Write } \mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$$

$$f(x) = \begin{bmatrix} e^{m(x^{(1)}) - m(x)} f(x^{(1)}) & e^{m(x^{(2)}) - m(x)} f(x^{(2)}) \end{bmatrix}$$

blocks

Step 12:

- Implement the partial softmax sum
- The first part**, underlined in green, updates the current softmax estimate for the blocks before the current block in the same row of blocks.
- The second part**, underlined in yellow, multiplies the $\tilde{\mathbf{P}}_{ij}$ matrix with the block of $\mathbf{V}$ vectors ($\mathbf{V}_j$).
- The e^x terms are there to modify the matrix $\tilde{\mathbf{P}}_{ij}$ & $\mathbf{O}_i$ by canceling out the m from the previous iteration and instead updating it with the latest estimate (m_{new_i}) that contains the row-wise max so far.



Notation: d — attention head dimension, B_r (row block size), B_c (column block size)

HPML

FlashAttention Algorithm

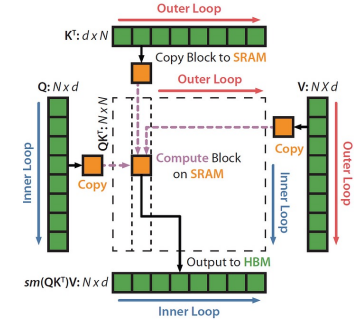
Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

Notation: d — attention head dimension, B_r (**row** block size), B_c (**column** block size)

HPML



Step 13:

- Write the newest cumulative statistics (ℓ_i & m_i) back to HBM. Notice these are of dimension B_r .

FlashAttention Algorithm

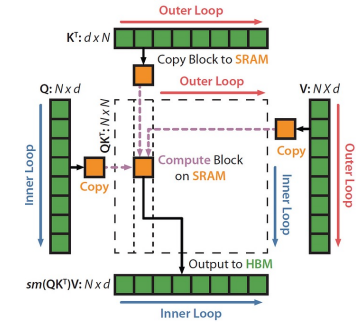
Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
- 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
- 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
- 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
- 5: **for** $1 \leq j \leq T_c$ **do**
- 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
- 7: **for** $1 \leq i \leq T_r$ **do**
- 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
- 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
- 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
- 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
- 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
- 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
- 14: **end for**
- 15: **end for**
- 16: Return $\mathbf{O}$.

Notation: d — attention head dimension, B_r (row block size), B_c (column block size)

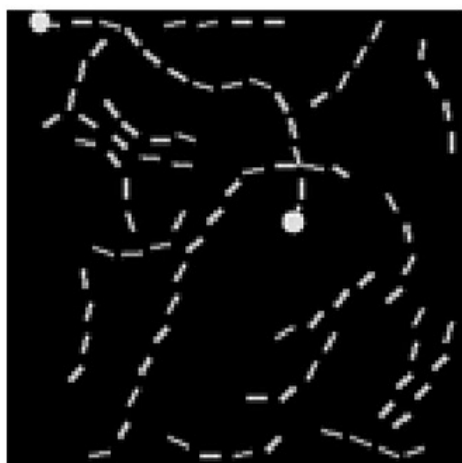
HPML



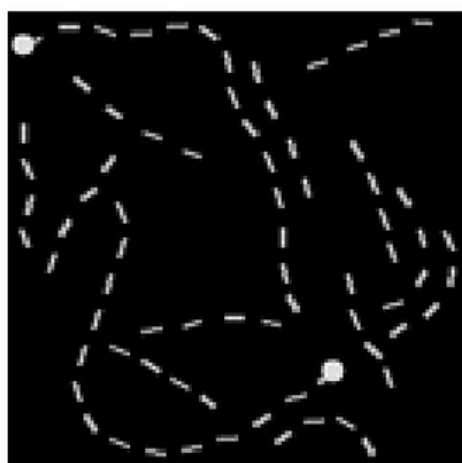
Step 14, 15, and 16:

- Once the nested for loop is over, $\mathbf{O}$ ($N \times d$) will contain the final result: **attention-weighted value vectors for each of the input tokens**

Solving Path-X and Path-256 benchmarks



Positive Example



Negative Example

This is a challenging benchmark where the task is to classify whether two points in a black and white 128×128 (or 256×256) image have a path connecting them.

Path-X and Path-256 require sequence lengths of 16k/64k

Model	Path-X	Path-256
Transformer	✗	✗
Linformer [84]	✗	✗
Linear Attention [50]	✗	✗
Performer [12]	✗	✗
Local Attention [80]	✗	✗
Reformer [51]	✗	✗
SMYRF [19]	✗	✗
FLASHATTENTION	61.4	✗
Block-sparse FLASHATTENTION	56.0	63.1

FlashAttention is the first transformer model that achieves non-random performance on Path-X and Path-256 benchmarks.

Massive Adoption

<https://github.com/Dao-AILab/flash-attention?tab=readme-ov-file>

Wide adoption and various implementations:

<https://github.com/Dao-AILab/flash-attention/blob/main/usage.md>

FlashAttention adoption

We've been very happy to see FlashAttention being adopted by many organizations and research labs to speed up their training / inference (within 6 months after FlashAttention's release, at the time of writing). This page contains a partial list of places where FlashAttention is being used. If you'd like to add links to your organization / product / codebase, please open a PR or email us. We'd very much like to hear from you!

Integrated into machine learning frameworks

- Pytorch: [integrated](#) into core Pytorch in nn.Transformer.
- Huggingface's [transformers](#) library. [On-going](#), blogpost coming soon.
- Microsoft's [DeepSpeed](#): FlashAttention is [integrated](#) into DeepSpeed's inference engine.
- Nvidia's [Megatron-LM](#). This library is a popular framework on training large transformer language models at scale.
- MosaicML [Composer](#) library. Composer is a library for efficient neural network training.
- EleutherAI's [GPT-NeoX](#). This is a research library for training large language transformer models at scale based on NVIDIA's Megatron-LM and Microsoft's DeepSpeed.
- PaddlePaddle: integrated into the framework with [API](#) `paddle.nn.functional.flash_attention`.

